Karadul

(Black Widow)

Tersine Muhendislik ve Statik Analiz Araci
Kapsamli Teknik Dokuman

Berke Tezgocen

Fizik Muhendisligi — ITU

2026

Bu dokuman, Karadul (Black Widow) tersine muhendislik aracinin tum algoritmalarini, matematiksel temellerini, tasarim kararlarini ve implementasyon detaylarini A'dan Z'ye anlatir.

Hazirlayan: Codex AI Danismanligi + Claude Opus 4.6

Son guncelleme: 24 Mart 2026

Contents

1 Giris ve Motivasyon **18**

1.1 Tersine Muhendislik Nedir? 18

1.1.1 Neden Onemli? 18

1.1.2 Endustri Baglami 19

1.1.3 Mevcut Cozumlerin Sinirliliklari 19

1.1.4 Karadul'un Yaklasimi ve Farki 20

1.2 Proje Kapsami 20

1.3 Dogrulanan Gercek Hedefler 20

1.4 Bu Dokumanin Organizasyonu 20

2 Pipeline Mimarisi **22**

2.1 Pipeline Pattern — Matematiksel Temel 22

2.1.1 Formal Tanim 22

2.1.2 Gercek Dependency Grafi 22

2.1.3 Topolojik Siralama 23

2.1.4 Neden Sequential Pipeline? 24

2.2 PipelineContext — Shared State Pattern 25

2.3 Error Recovery — Circuit Breaker Pattern 25

2.3.1 State Machine Modeli 25

2.3.2 Exponential Backoff 26

2.3.3 Neden Exponential Backoff? 26

2.3.4 Jitter 27

2.3.5 Basari Olasiligi 27

3 Hedef Tespit Sistemi **28**

3.1 Magic Number Sistemi . . . 28

3.1.1 Collision Olasiligi . . . 28

3.1.2 Big-Endian vs Little-Endian Okuma . . . 29

3.2 Dil Tespit Algoritmasi . . . 29

3.2.1 Puanlama Sistemi . . . 29

3.3 JavaScript Bundler Tespiti . . . 29

3.4 SHA-256 Hashing . . . 30

3.5 Mach-O Format Detaylari . . . 30

3.5.1 Header → Load Commands → Segments → Sections . . . 30

3.5.2 Universal Binary (Fat Binary) . . . 31

3.5.3 CAFEBABE Cakismasi . . . 32

3.6 ELF Format ve Go Binary Tespiti . . . 32

3.6.1 ELF Yapisi . . . 32

3.6.2 GOPCLNTAB (Go Program Counter Line Number Table) . . . 33

3.7 PE Format ve .NET/Delphi Ayrimi . . . 33

3.7.1 PE Yapisi . . . 33

3.7.2 .NET Tespiti: mscoree.dll . . . 34

3.7.3 Delphi Binary Tespiti . . . 34

3.8 Dil Tespit Algoritmasi — Bayes Perspektifi . . . 34

3.8.1 Mevcut Puanlama Sistemi . . . 34

3.8.2 Naive Bayes Formulasyonu . . . 35

3.8.3 False Positive Analizi . . . 36

3.9 Bundler Tespit Heuristikleri . . . 36

3.9.1 webpack Runtime Yapisi . . . 36

3.9.2 esbuild — Tespiti Zor . . . 37

3.9.3 Bundler Tespitinin Ooenemi . . . 37

3.10 SHA-256 Hashing — Detayli Analiz . . . 37

3.10.1 Merkle-Damgård Construction . . . 37

3.10.2 8KB Chunk Boyutu Optimalligi . . . 37
3.10.3 Collision Olasiligi . . . 38

4 Statik Analiz **39**
4.1 Binary Statik Analiz . . . 39
4.1.1 Mach-O Analizi . . . 39
4.1.2 Symbol Extraction . . . 39
4.1.3 Ghidra Headless Analiz . . . 40
4.2 JavaScript Statik Analiz . . . 40
4.3 Hedef Tespit Sistemine Genel Bakis . . . 41
4.4 Magic Byte Sistemi . . . 41
4.4.1 Temel Magic Byte Tanimlari . . . 43
4.4.2 Big-Endian ve Little-Endian (Cigam) Kavrami . . . 43
4.5 Mach-O Format Yapisi . . . 44
4.5.1 Mach-O Header Yapisi . . . 44
4.5.2 Load Commands . . . 46
4.5.3 Segment ve Section Yapisi . . . 46
4.6 ELF Format Yapisi . . . 47
4.6.1 ELF Header . . . 47
4.6.2 ELF ve Mach-O Karsilastirmasi . . . 47
4.7 PE Format Yapisi . . . 48
4.7.1 PE Dosya Yapisi . . . 48
4.8 Universal Binary (Fat Binary) . . . 49
4.9 Dil Tespit Algoritmasi . . . 49
4.9.1 Puanlama Sistemi . . . 50
4.9.2 Dil Imza Tablosu . . . 50
4.9.3 Naive Bayes Perspektifi . . . 50
4.10 Go Binary Tespiti . . . 51
4.10.1 GOPCLNTAB (Program Counter Line Table) . . . 51
4.10.2 Go Binary Isim Kurtarma Zenginligi . . . 52

4.11 .NET ve Delphi Tespiti . . . 52
4.11.1 .NET Assembly Tespiti . . . 52
4.11.2 Delphi Binary Tespiti . . . 52
4.12 JavaScript Bundle Tespiti . . . 53
4.12.1 Bundler Tespit Sistemi . . . 53
4.12.2 esbuild Tespiti . . . 54
4.12.3 Webpack vs Parcel vs Rollup Farklari . . . 54
4.13 Electron .app Bundle Analizi . . . 54
4.13.1 Electron Tespit Algoritmasi . . . 54
4.13.2 ASAR Arsiv Formati . . . 54
4.13.3 Ana JS Dosyasi Bulma Stratejisi . . . 55
4.14 SHA-256 Streaming Hash . . . 55
4.14.1 Streaming Hash Algoritmasi . . . 55
4.14.2 Neden 8 KB Chunk? . . . 56
4.14.3 Collision Olasiligi . . . 56
4.15 False Positive Analizi . . . 57
4.15.1 Magic Byte Collision Hesabi . . . 57
4.15.2 Coklu Kontrol Stratejisi . . . 57
4.16 Statik Analize Genel Bakis . . . 59
4.17 Binary Statik Analiz Araclari . . . 59
4.17.1 otool: Dynamic Library ve Load Command Analizi . . . 59
4.17.2 nm: Symbol Table Extraction . . . 61
4.17.3 strings: String Extraction . . . 62
4.18 Ghidra Headless Analiz . . . 63
4.18.1 PyGhidra API vs CLI Modu . . . 63
4.18.2 Decompilation Pipeline . . . 63
4.18.3 Ghidra Script'leri . . . 63
4.19 Mach-O Segment/Section Analizi . . . 64
4.19.1 __TEXT Segmenti . . . 64

4.19.2 __DATA Segmenti 65
4.19.3 __LINKEDIT Segmenti 65
4.20 Symbol Extraction: SYMTAB ve DYSYMTAB 65
4.20.1 SYMTAB (Symbol Table) 65
4.20.2 DYSYMTAB (Dynamic Symbol Table) 66
4.21 JavaScript Babel AST Analizi 66
4.21.1 AST Kavramsal Yapisi 66
4.21.2 Babel Parser Entegrasyonu 66
4.21.3 Cikarilan AST Node Tipleri 67
4.21.4 Fonksiyon Extraction Algoritmasi 67
4.21.5 String Extraction 68
4.22 Webpack Modul ID Tespiti 68
4.22.1 Webpack Runtime Yapisi 68
4.22.2 Modul ID Cikartma Algoritmasi 69
4.22.3 Modul ID Mapping 70
4.23 Buyuk Dosya Destegi: ChunkedProcessor 70
4.23.1 Chunk Bolme Algoritmasi 70
4.23.2 Top-Level Statement Tespiti 71
4.23.3 Paralel Isleme 71
4.24 FLIRT Signature Sistemi 72
4.24.1 .pat Dosya Formati 72
4.24.2 Pattern Matching Algoritmasi 72
4.24.3 Hex-to-Bytes-with-Mask Donusumu 73
4.24.4 SignatureDB Entegrasyonu 73
4.25 Assembly Instruction Analizi 73
4.25.1 ARM64 (AArch64) vs x86_64 73
4.25.2 Calling Convention Tespiti 74
4.25.3 Stack Frame Analizi 74
4.25.4 SIMD Pattern Tespiti 75

4.26 Entropy Analizi ile Packing Tespiti 75
4.26.1 Shannon Entropisi 75
4.26.2 Packing Karar Algoritmasi 76
4.27 Her Analyzer Icin Giris/Cikis Veri Yapilari 77
4.27.1 Giris Veri Yapisi: TargetInfo 77
4.27.2 Cikis Veri Yapisi: StageResult 77
4.27.3 Analyzer Giris/Cikis Ozet Tablosu 78
4.27.4 Graceful Degradation 78
4.28 Binary Intelligence: String Clustering ve Mimari Cikarim 79

5 JavaScript Deobfuscation 80
5.1 Obfuscation Nedir? 80
5.2 Standart 4 Adimli Zincir 80
5.2.1 Graceful Degradation 81
5.3 9 Fazli Deep Deobfuscation 81
5.3.1 Faz 1: Constant Folding (Sabit Katlama) 81
5.3.2 Faz 2: Dead Code Elimination 81
5.3.3 Faz 3: String Unhexing 82
5.3.4 Faz 4: Computed-to-Static Property 82
5.3.5 Faz 5–8: Ileri Basitlestirmeler 82
5.3.6 Faz 9: Semantik Degisken Isimlendirme 82
5.4 Abstract Syntax Tree (AST) — Derinlemesine 83
5.4.1 Formal Tanim 83
5.4.2 Babel vs Acorn (ESTree) 84
5.4.3 Hesaplama Karmasikligi 84
5.5 Constant Folding — Lattice Teorisi 84
5.5.1 Abstract Interpretation Framework 84
5.5.2 Karadul Implementasyonu 85
5.5.3 Sinirlar 85
5.6 Dead Code Elimination — Liveness Analizi 86

5.6.1 Dataflow Denklemleri 86
5.6.2 Karadul'un Basitlestirilmis Yaklasimi 86
5.7 String Obfuscation Teknikleri ve Coezeuemleri 87
5.7.1 6 Yaygin Teknik 87
5.7.2 Statik Analiz vs Runtime Emulation 87
5.8 Computed-to-Static — Soundness Analizi 87
5.9 Semantik Degisken Isimlendirme — 25 Kural Motoru 88
5.9.1 Mimari 88
5.9.2 Kural Detaylari 88
5.9.3 Neden LLM Kullanilmiyor? 89
5.10 Webpack Module Extraction — Detayli 89
5.10.1 Extraction Algoritmasi 89
5.10.2 esbuild vs webpack 90
5.11 Chunked Processing — 200MB+ Dosyalar 90
5.11.1 Problem 90
5.11.2 Neden 512KB Chunk? 90
5.11.3 Boundary Tespiti 91
5.11.4 Acorn Fallback — 32MB+ Dosyalar 91
5.12 Obfuscation: Formal Tanim ve Teorik Temel 91
5.12.1 Semantik Koruma Teoremi 91
5.12.2 Obfuscation Karmasiklik Siniflamasi 92
5.13 Yaygin Obfuscation Teknikleri 93
5.13.1 Minification 93
5.13.2 String Encoding 93
5.13.3 Control Flow Flattening (CFF) 94
5.13.4 Opaque Predicates 95
5.13.5 Dead Code Injection 96
5.13.6 String Array Rotation 96
5.14 Abstract Syntax Tree (AST) Temelleri 97

5.14.1 BNF ve Context-Free Grammar (CFG) 97
5.14.2 Parse Tree vs Abstract Syntax Tree 97
5.14.3 ESTree Spesifikasyonu 98
5.14.4 Babel Parser Oezellikleri 98
5.15 Standart 4 Adimli Deobfuscation Zinciri 98
5.15.1 Graceful Degradation Formuelue 99
5.15.2 Dosya Isimlendirme Konvansiyonu 100
5.16 Adim 1: Beautify 100
5.16.1 Konfiguerasyon Parametreleri 100
5.16.2 Etki Analizi 100
5.16.3 Fallback Davranisi 101
5.17 Adim 2: Synchrony Deobfuscation 101
5.17.1 Calismaa Mekanizmasi 101
5.17.2 Boyut Analizi 102
5.18 9 Fazli Deep Deobfuscation Pipeline 102
5.18.1 Faz 1: Parse (Tolerant Mode) 103
5.18.2 Faz 2: Constant Folding (Sabit Katlama) 104
5.18.3 Faz 3: Dead Code Elimination 105
5.18.4 Faz 4: Computed-to-Static Property 107
5.18.5 Faz 5: Comma Expression Splitting 108
5.18.6 Faz 6: Variable Declaration Splitting 108
5.18.7 Faz 7: Ternary → If/Else 109
5.18.8 Faz 8: Arrow → Named Function 109
5.18.9 Faz 9: Smart Variable Renaming 110
5.18.10 Faz 10: Semantic Renaming (Deep Usage-Based) 111
5.19 Webpack Module Extraction 112
5.19.1 Desteklenen Bundle Formatlari 112
5.19.2 __webpack_require__ Mekanizmasi 112
5.19.3 esbuild vs Webpack Format Farklari 113

5.20 Buyuk Dosya Isleme: Chunked Processing . . . 114
5.20.1 Esik Degeri ve Tetikleme . . . 114
5.20.2 Stream-Parse Pipeline . . . 114
5.20.3 Timeout Hesaplaması . . . 114
5.21 Control Flow Flattening (CFF) Deflattening . . . 115
5.21.1 Tespit Algoritması . . . 115
5.21.2 State Transition Graph Cikarma . . . 115
5.21.3 Topological Sort ile Yeniden Olusturma . . . 115
5.21.4 Karmasiklik Analizi . . . 116
5.22 Opaque Predicate Removal . . . 116
5.22.1 Tespit Stratejileri . . . 116
5.22.2 Z3 Entegrasyonu . . . 117
5.22.3 Eliminasyon . . . 117
5.23 String Decryption . . . 117
5.23.1 Desteklenen Sifreleme Yoentemleri . . . 117
5.23.2 Stack String Reconstruction . . . 118
5.23.3 XOR Brute-Force . . . 118
5.24 Acorn Fallback . . . 118
5.24.1 Babel vs Acorn Karsilastirmasi . . . 118
5.25 Problem Tanimi: Coklu Kaynak, Tek Isim . . . 120
5.26 Bayes Teoremi: Temelden Turetme . . . 120
5.26.1 Kosula Bagli Olasilik . . . 120
5.26.2 Isimlendirme Baglaminda Anlami . . . 121
5.26.3 Ardisik Kanit Birlestirme . . . 121
5.27 Log-Odds Formu: Neden ve Nasil . . . 121
5.27.1 Odds ve Log-Odds . . . 121
5.27.2 Turetme . . . 122
5.27.3 Korelasyon Duzeltmesi . . . 122
5.27.4 Neden Log Space? . . . 122

5.28 Sigmoid Fonksiyonu: Logit'in Tersi 122
5.28.1 Sigmoid'in Ozellikleri 123
5.28.2 Overflow Korumasi 123
5.29 7 Kaynak Detayli Aciklama 123
5.29.1 binary_extractor ($w = 1.0$) 123
5.29.2 c_namer ($w = 1.0$) 124
5.29.3 string_intel ($w = 0.8$) 124
5.29.4 signature_db ($w = 1.0$) 124
5.29.5 swift_demangle ($w = 1.0$) 125
5.29.6 source_matcher ($w = 0.85$) 125
5.29.7 llm4decompile ($w = 0.5$) 125
5.29.8 byte_pattern ($w = 1.0$) 126
5.30 Korelasyon-Aware Agirliklar: Bilgi Kurami 126
5.30.1 Neden $w_i < 1$? 126
5.30.2 Kaynak Agirliklari Tablosu 126
5.30.3 Overconfidence Problemi: Somut Ornek 126
5.31 Gruplama Stratejisi: 5 Seviye 127
5.31.1 Seviye 1: Exact Multi-Match 128
5.31.2 Seviye 2: Partial Match 128
5.31.3 Seviye 3: Semantic Match 129
5.31.4 Seviye 4: Voting 130
5.31.5 Seviye 5: Single Source 130
5.32 Normalize Isim Karsilastirmasi 131
5.33 UNK Threshold: Precision vs Recall 131
5.33.1 NSA-Grade Felsefe 131
5.33.2 ROC Analizi 132
5.34 Somut Hesaplama Ornekleri 132
5.34.1 Senaryo 1: Uc Kaynak Ayni Isim (Exact Multi-Match) 132
5.34.2 Senaryo 2: Iki Kaynak Partial Match 133

5.34.3 Senaryo 3: Tek Kaynak, Dusuk Confidence → UNK 134
5.35 Hesaplama Karmasikligi 135
5.35.1 Genel Karmasiklik 135
5.35.2 Alt Islem Karmasikliklari 135
5.35.3 Pratikte Performans 136
5.36 Overconfidence Problemi ve Cozumu 136
5.36.1 Naive Bayes Hatasi 136
5.36.2 Korelasyon ile Duzeltme 136
5.36.3 Sonuc 137

6 Bayesian Isim Fuzyonu 138
6.1 Problem Tanimi 138
6.2 Bayesian Inference Temelleri 138
6.2.1 Bayes Teoremi 138
6.2.2 Log-Odds Formu 139
6.2.3 Neden Log-Odds? 139
6.3 Korelasyon-Aware Agirliklar 139
6.3.1 Neden $w_i < 1$? 139
6.3.2 Kaynak Agirliklari Tablosu 140
6.4 Somut Hesaplama Oernegi 140
6.4.1 Ikinci Oernek: Tam Bagimsiz Kaynaklar 141
6.4.2 Uecuencue Oernek: Tek Kaynak, Duesueek Confidence 141
6.5 Gruplama Stratejisi 142
6.5.1 Partial Overlap Hesabi 142
6.5.2 UNK Threshold 142
6.5.3 Ikili Threshold Sistemi 142
6.5.4 ROC Perspektifi 143
6.5.5 Hesaplama Karmasikligi 143

7 Kaynak Kod Rekonstruksiyonu 144

7.1 6 Katmanli C Naming . . . 144
7.2 FLIRT Signature Matching . . . 144
7.3 NPM Source Matching . . . 145
7.4 Algoritma Tespiti . . . 145
7.5 Kaynak Kod Rekonstruksiyonuna Giris . . . 146
7.6 6 Katmanli C Isimlendirme Sistemi (c_namer.py) . . . 146
7.6.1 Mimari Genel Bakis . . . 147
7.6.2 Katman 1: Symbol / Export Table . . . 148
7.6.3 Katman 2: String Context Analysis . . . 149
7.6.4 Katman 3: API Call Pattern Matching . . . 149
7.6.5 Katman 4: Call Graph Structure . . . 150
7.6.6 Katman 5: Dataflow Analysis . . . 151
7.6.7 Katman 6: Type-Based Inference . . . 151
7.6.8 Toplam Bilgi Entropisi Azaltma . . . 152
7.7 Tip Kurtarma (c_type_recoverer.py) . . . 153
7.7.1 Struct Recovery . . . 153
7.7.2 Enum Recovery . . . 154
7.7.3 VTable Recovery . . . 154
7.8 Algoritma Tespiti (c_algorithm_id.py) . . . 154
7.8.1 Katman 1: Sabit-Bazli Tanimlar (Constant-Based) . . . 154
7.8.2 Katman 2: Yapi-Bazli Tespit (Structure-Based) . . . 156
7.8.3 Katman 3: API Korelasyonu . . . 156
7.9 Yorum Uretimi (c_comment_generator.py) . . . 156
7.10 Proje Yapilandirma . . . 157
7.10.1 project_reconstructor.py – Modul Iskele Olusturma . . . 157
7.10.2 C Project Builder (c_project_builder.py) . . . 158
7.10.3 project_builder.py – CMake/Makefile Uretimi . . . 158
7.11 NPM Source Matching . . . 159
7.11.1 AST Structural Hash (ast_fingerprinter.py) . . . 159

7.11.2 Tree Edit Distance (Zhang-Shasha) 159
7.11.3 Structural Matcher Pipeline 160
7.12 Parametre Kurtarma (param_recovery.py) 160
7.12.1 TypeScript .d.ts Mapping (dts_namer.py) 161
7.13 Guvenlik Analiz Algoritmalarinin Temeli 162
7.14 Shannon Entropy ve Packer Tespiti 162
7.14.1 Shannon Entropy Formulu 162
7.14.2 Section-Level Entropy Profiling 163
7.14.3 Packing Tespit Karari 163
7.14.4 Packer Signature'lari 163
7.14.5 Chi-Squared Uniformity Test 164
7.15 YARA Pattern Matching 164
7.15.1 Aho-Corasick Algoritma Temeli 164
7.15.2 YARA Kural Yapisi 166
7.15.3 Hex Pattern + Wildcard 167
7.16 Control Flow Graph (CFG) Analizi 167
7.16.1 Formal Tanim 167
7.16.2 Basic Block ve Leader Detection 167
7.16.3 Dominator Tree – Lengauer-Tarjan 168
7.16.4 Loop Detection ve Back Edges 168
7.17 Opaque Predicate Tespiti 168
7.17.1 Tanim ve Amaclari 169
7.17.2 Matematiksel Ispat: $x^2 + x \equiv 0 \pmod 2$ 169
7.17.3 Symbolic Execution ile Tespit (Z3) 170
7.17.4 Rice's Theorem ve Undecidability 170
7.18 Control Flow Flattening (CFF) Deflattening 170
7.18.1 CFF Nedir? 170
7.18.2 Deflattening Algoritmasi 171
7.19 Ghidra Decompilation Pipeline 171

7.19.1 Binary'den C Koduna Yolculuk 171
7.20 Dynamic Analysis – Frida 172
7.20.1 Inline Hooking vs PLT/GOT Hooking 172
7.20.2 API Tracing 173
7.20.3 Anti-Debug Bypass 173
7.21 Binary Diffing (BinDiff) 173
7.21.1 CFG Isomorphism Problemi 173
7.21.2 5 Strateji ile Fonksiyon Eslestirme 174
7.21.3 Hungarian Algorithm ile Optimal Eslestirme 174
7.22 Byte Pattern Matching 175
7.22.1 FLIRT Signature Eslestirme 175
7.22.2 Fuzzy Matching – Compiler Optimization Farklari 175
7.22.3 Inline Fonksiyon Tespiti (inline_detector.py) 175
7.23 Performans Analizine Giris 177
7.24 Amdahl Yasasi ve Ghidra Darbogazlari 177
7.24.1 Amdahl Yasasi 177
7.25 Gustafson Yasasi – Alternatif Perspektif 178
7.26 JVM Heap Yonetimi – Ghidra 178
7.26.1 G1GC (Garbage First Garbage Collector) 178
7.26.2 Heap Sizing Formulu 179
7.26.3 GC Pressure ve Batch Processing 179
7.27 Node.js V8 Heap Yonetimi 179
7.28 Chunked Processing 180
7.28.1 Motivasyon ve 512 KB Chunk Optimal Analizi 180
7.28.2 mmap vs Buffered Read 180
7.28.3 Disk I/O Throughput Analizi 181
7.29 Ghidra Batch Optimization 181
7.29.1 5000 Fonksiyon/Batch Mantigi 181
7.30 Apple Silicon Optimizasyonu 182

7.30.1 P-core vs E-core Stratejisi 182
7.30.2 Unified Memory Architecture (UMA) 182
7.31 Scalability Analizi 183
7.31.1 Binary Boyutu vs Islem Suresi 183
7.32 Docker Container 183
7.32.1 Overhead Analizi 183
7.32.2 Multi-Stage Build 184
7.33 Error Recovery 184
7.33.1 Circuit Breaker Pattern 185
7.33.2 Exponential Backoff + Jitter 186
7.33.3 MTBF ve MTTR 186
7.33.4 P(success after k retries) 187
7.33.5 Idempotency 187

8 Guevenlik Analiz Algoritmalari 189

8.1 Entropy Analizi ve Packer Tespiti 189
8.2 Control Flow Graph Analizi 189
8.2.1 Dominator Tree 190
8.3 Opaque Predicate Tespiti 190
8.4 Entropy Analizi — Shannon Entropy Turetmesi 190
8.4.1 Turetme 190
8.4.2 Implementasyon 191
8.4.3 Packer Entropy Profilleri 191
8.4.4 Chi-Squared Uniformity Testi 192
8.5 YARA Pattern Matching — Aho-Corasick 192
8.5.1 Aho-Corasick Algoritmasi 192
8.5.2 Karadul'daki Dahili Kurallar 192
8.6 Control Flow Graph — Dominator Tree 193
8.6.1 Dominator Hesaplama 193
8.6.2 Loop Detection 193

8.7 Opaque Predicate — Formal Analiz 193
8.8 Ghidra Decompilation Pipeline 194
8.9 Cryptographic Algorithm Identification 194
8.9.1 AES S-Box 195
8.9.2 SHA-256 Initial Hash Values 195
8.9.3 ChaCha20 Sigma Sabitleri 195
8.10 Amdahl Yasasi — Detayli Analiz 195
8.10.1 Pipeline Paralellestirilmesi 195
8.11 JVM Heap Yoenetimi — G1GC 196
8.11.1 Ghidra icin Heap Boyutlandirma 196
8.11.2 G1GC vs ZGC 196
8.12 Apple Silicon Optimizasyonu 197
8.12.1 M4 Max Spesifikasyonlari 197
8.12.2 P-core Otomatik Tespiti 197
8.12.3 Unified Memory Architecture (UMA) 197
8.13 Docker Container Overhead 197
8.13.1 Performans Karsilastirmasi 198

9 Performans ve Oelceklendirme 199
9.1 Amdahl Yasasi 199
9.2 Bellek Yoenetimi 199
9.2.1 Ghidra JVM Heap 199
9.2.2 Chunked Processing 200
9.3 Apple Silicon Optimizasyonu 200

10 Tasarim Kararlari ve Retrospektif 201
10.1 Onemli Kararlar 201
10.1.1 LLM Tabanli Isimlendirmenin Terk Edilmesi 201
10.1.2 Python + JavaScript Hibrit Mimarisi 201
10.1.3 Graceful Degradation 201

10.2 Ne Ise Yaradi? 202
10.3 Ne Yaramadi? 202
10.4 Gelecek Calismalar 202

A Parametre Tablosu 203

A.1 Timeout Parametreleri 203
A.2 Bayesian Fusion Parametreleri 203
A.3 Binary Analiz Parametreleri 203

B Muelakat Hazirlik 205

B.1 Soru 1: Projeyi bir cuemleyle acikla 205
B.2 Soru 2: Bayesian fusion nasil calisiyor? 205
B.3 Soru 3: Neden LLM kullanmiyorsun? 205
B.4 Soru 4: Pipeline neden sequential? 206
B.5 Soru 5: Graceful degradation nasil calisiyor? 206
B.6 Soru 6: 200MB+ dosyalari nasil isleyorsunuz? 206
B.7 Soru 7: Constant folding nedir? 206
B.8 Soru 8: Circuit breaker neden var? 206
B.9 Soru 9: FLIRT signature matching nasil calisiyor? 206
B.10 Soru 10: UNK threshold neden 0.30? 207
B.11 Soru 11: Magic byte nedir? 207
B.12 Soru 12: Entropy ile packer nasil tespit edilir? 207
B.13 Soru 13: Dead code elimination nasil calisiyor? 207
B.14 Soru 14: Opaque predicate nedir? 207
B.15 Soru 15: Webpack bundle'i nasil ayristiriyorsunuz? 207
B.16 Soru 16: Projenin en zor kismi neydi? 208
B.17 Soru 17: Projede kac satir kod var? 208
B.18 Soru 18: Korelasyon-aware agirlik ne demek? 208
B.19 Soru 19: Bu proje hangi sorunu coezer? 208
B.20 Soru 20: Gelecekte ne yapmayi planliyorsun? 208

Chapter 1

Giris ve Motivasyon

1.1 Tersine Muhendislik Nedir?

Tersine muhendislik (reverse engineering), bir yazilim urununu — kaynak kodu olmadan — analiz ederek ic yapisini, algoritmalarini, veri yapilarini ve davranisini anlama surecidir. Derlenmis bir binary dosyadan veya obfuscate edilmis bir JavaScript bundle'indan orijinal kaynak koda yaklasma cabasi, bilgisayar bilimlerinin en zorlayici alanlarindan biridir.

Formal Tanim

Bir $f : S \to T$ fonksiyonu verildiginde, tersine muhendislik $f^{-1} : T \to \hat{S}$ yaklasik ters fonksiyonunu bulma problemidir. Burada:

- S: Orijinal kaynak kod uzayi
- T: Derlenmis/obfuscate edilmis cikti uzayi
- f: Derleyici veya obfuscator donusumu (genellikle surjective ama injective degil)
- $\hat{S}$: Kaynak kodun yaklasik rekonstruksiyonu

Kritik nokta: f genellikle bijective degil — birden fazla farkli kaynak kod ayni binary'yi uretebilir. Bu nedenle $\hat{S} \neq S$ olabilir, ama $f(\hat{S}) \approx f(S)$ olmalidir.

1.1.1 Neden Onemli?

Tersine muhendislik bircok kritik alanda kullanilir:

1. **Guvenlik arastirmasi:** Malware analizi, zafiyet kesfetme (vulnerability research),

exploit gelistirme ve savunma.

2. **Uyumluluk:** Clean-room reverse engineering ile uyumlu yazilim gelistirme (ornek: Wine projesi, ReactOS).

3. **Dijital adli bilisim:** Siber suc arastirmalarinda delil analizi.

4. **Legacy sistem bakimi:** Kaynak kodu kaybolmus yazilimlarin anlasilmasi ve surdurulmesi.

5. **Yarisma (CTF):** Capture The Flag yarismalarinda binary exploitation.

1.1.2 Endustri Baglami

Tersine muhendislik araci pazari buyuk ve buyuyor:

Arac	Fiyat	Ozellik
IDA Pro	$2,500+/yil	Endustri standardi, kapali kaynak
Ghidra	Ucretsiz (NSA)	Acik kaynak, Java tabanli
Binary Ninja	$400/yil	Modern UI, Python API
radare2/Cutter	Ucretsiz	Acik kaynak, komut satiri odakli
Karadul	**Ucretsiz**	**Otonom pipeline, Bayesian isimlendirme**

Table 1.1: Tersine muhendislik arac karsilastirmasi.

1.1.3 Mevcut Cozumlerin Sinirliliklari

Mevcut araclar tipik olarak su sorunlara sahiptir:

1. **Manuel is yueku:** IDA Pro ve Ghidra gueclue decompiler'lar saglar, ancak fonksiyon isimlendirme, tip cikarimi ve yapı tespiti buyuek olcuede analistin elindedir. 10.000 fonksiyonluk bir binary'de bu haftalarca suerer.

2. **Tek platform odakli:** Cogu arac ya binary ya da JS analizi yapar, ikisini birden degil.

3. **Otomasyon eksikligi:** Toplu analiz (batch processing), otomatik raporlama ve end-to-end pipeline mevcut araclarda sinirlidir.

4. **Isim kurtarma kalitesi:** Stripped binary'lerde fonksiyon isimlerini kurtarma orani genellikle duesuektuer.

1.1.4 Karadul'un Yaklasimi ve Farki

Karadul (Black Widow), bu sinirlamalari su yeniliklerle asar:

1. **6 asamali otonom pipeline:** Hedef tespitinden rapora kadar tamamen otomatik.
2. **Coklu hedef destegi:** 12 farkli binary/bundle formati, 13 programlama dili.
3. **Bayesian isim fuzyonu:** 7 bagimsiz kaynaktan gelen isimleri korelasyon-aware Bayesian log-odds ile birlestirme — enduestride benzersiz.
4. **Derin JS deobfuscation:** 9 fazli AST transform pipeline — LLM'siz semantik degisken isimlendirme.
5. **Graceful degradation:** Herhangi bir asamanin basarisizligi pipeline'i durdurmaz.
6. **Olceklenebilirlik:** 224MB+ binary'ler ve 200MB+ JS dosyalari icin chunked processing.

1.2 Proje Kapsami

Karadul v1.0 su bilesenlerden olusur:

- **110 Python modulu,** 65.000+ satir kod
- **460+ JavaScript/MJS analiz scripti**
- **1.393 test** (tumu gecen)
- **1.5M+ kutuephane imzasi** (FLIRT veritabani)
- **Docker destegi** + pip kurulabilir paket
- **CLI arayuzu** (Click tabanli)

1.3 Dogrulanan Gercek Hedefler

1.4 Bu Dokumanin Organizasyonu

Bolum 2 — Pipeline Mimarisi: 6 asamali pipeline, context pattern, error recovery.

Bolum 3 — Hedef Tespit Sistemi: Magic bytes, binary format analizi, dil tespiti.

Hedef	Tip	Dil	Boyut	Fonk.	Kurtarma
Rectangle	Mach-O	Swift	1.8 MB	2.852	%89.9
Steam	Mach-O	C++	4.1 MB	11.296	%74.2
Claude Code	JS bundle	esbuild	11.8 MB	43.747	%57.3
Cursor	JS bundle	webpack	5.97 MB	17.705	%65.4
LM Studio	Electron	webpack	52 MB	150K+	4/4 bundle

Table 1.2: Gercek duenyada dogrulanan hedefler.

Bolum 4 — Statik Analiz: Binary ve JS statik analiz algoritmalari.

Bolum 5 — JavaScript Deobfuscation: AST transform pipeline, 9 faz, webpack unpack.

Bolum 6 — Bayesian Isim Fuzyonu: Log-odds formulu, 7 kaynak, korelasyon duzeltmesi.

Bolum 7 — Kaynak Kod Rekonstruksiyonu: C naming, tip kurtarma, algoritma tespiti.

Bolum 8 — Guvenlik Analiz Algoritmalari: Entropy, YARA, CFG, opaque predicate.

Bolum 9 — Performans ve Olceklendirme: Buyuek dosya destegi, bellek yoenetimi, Apple Silicon.

Bolum 10 — Tasarim Kararlari ve Retrospektif: Neden bu yaklasimlar? Alternatifler.

Ek A — Parametre Tablosu: Tuem konfiguerasyon parametreleri.

Ek B — Muuelakat Hazirlik: 20 teknik soru ve detayli cevaplari.

Bu Bolumun Ozeti

Karadul, derlenmis binary'ler ve obfuscate edilmis JavaScript bundle'lari icin otonom bir tersine muhendislik aracidir. 6 asamali pipeline, Bayesian isim fuzyonu ve 9 fazli derin deobfuscation ile enduestri standartlarinin oetesinde bir otomasyon seviyesi saglar. 5 gercek duenya hedefinde dogrulanarak v1.0 olarak tamamlandi.

Chapter 2

Pipeline Mimarisi

Bu boluemde Karadul'un temelini olusturan 6 asamali pipeline mimarisini, yazilim muehendisligi prensiplerini ve hata kurtarma mekanizmalarini detayli olarak inceliyoruz.

2.1 Pipeline Pattern — Matematiksel Temel

Karadul'un pipeline'i, **Orchestrated DAG Executor** patternidir — akademik literaturde "workflow engine" siniflandirmasi. Bu, yazilim muehendisligindeki **Pipes and Filters** (Shaw & Garlan, 1996) ile **Chain of Responsibility**'nin melez bir formudur.

2.1.1 Formal Tanim

Pipeline'i bir yoenlu dogrusiz cizge (Directed Acyclic Graph — DAG) olarak modelleyebiliriz:

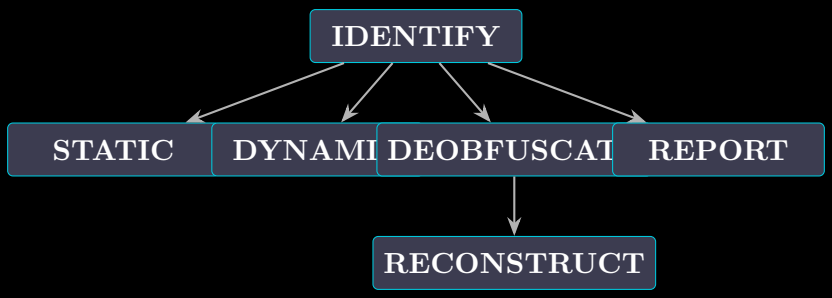
$$G = (V, E) \tag{2.1}$$

burada:

- $V = \{v_1, v_2, \ldots, v_n\}$ stage kuemesi (her v_i bir Stage nesnesi)
- $E \subseteq V \times V$ bagimlilik kenarlari ($v_i \to v_j$ demek "v_i, v_j'den oence calismali")

2.1.2 Gercek Dependency Grafi

Koddan cikarilan gercek bagimlilik yapisi (`stages.py`):



Bu bir **agac degil, DAG**. Identify koek dueguem, 4 dal ondan cikar ve reconstruct'in deobfuscate'e bagimliligi var. Lineer bir zincir degil — static, dynamic ve deobfuscate teorik olarak paralel calisabilir (hepsi sadece identify'a bagimli).

2.1.3 Topolojik Siralama

Stage'lerin calistirilma sirasi, topolojik siralama (topological sort) ile belirlenir. Kahn's Algorithm:

1. Her dueguemuen in-degree'sini hesapla: $\deg^-(v)$ (v'ye giren kenar sayisi)
2. $\deg^-(v) = 0$ olan tuem dueguemleri kuyruga ekle
3. Kuyruktan bir v al, siralanan listeye ekle
4. v'nin tuem komsularinin in-degree'sini 1 azalt
5. Yeni $\deg^- = 0$ olanlari kuyruga ekle
6. Tekrarla (tuem dueguemler islene kadar)

Hesaplama karmasikligi: $\mathcal{O}(|V| + |E|)$.

Kritik Tasarim Karari: Kayit Sirasi vs Topolojik Siralama

Karadul topolojik siralama **yapmiyor**. `pipeline.py:357`'deki `_resolve_run_order()` metodu **kayit sirasi**ni kullanir:

```
def _resolve_run_order(self, requested):
    if requested is None:
        return list(self._stage_order)  # kayit sirasi!
    return [s for s in self._stage_order if s in requested]
```

Stage'ler `register_stage()` ile eklenme sirasinda calistirilir. CLI her zaman dogru sirada register ettigi icin pratikte sorun yok. Yanlis sirada ekleme durumunda `_check_dependencies()` runtime'da fail eder ve stage atlanir.

Bu yaklasim **static topological sort** yerine **runtime dependency validation** tercih etmistir:

Yaklasim	Avantaj	Dezavantaj
Topolojik siralama	Yanlis siralama imkansiz	$\mathcal{O}(V + E)$ karmasiklik, cycle detection
Kayit sirasi + runtime check (Karadul)	Basit implementasyon	Kullanicinin dogru eklemesi gerekir

2.1.4 Neden Sequential Pipeline?

Dependency grafina bakildiginda static, dynamic ve deobfuscate paralel calisabilir. Ama sequential secilmis. Bunu Amdahl Yasasi ile analiz edelim:

Amdahl Yasasi ile Sayisal Analiz

Tipik stage suereleri:

$$T_{\text{identify}} = 2\text{s}, \quad T_{\text{static}} = 60\text{s (Ghidra)}, \quad T_{\text{dynamic}} = 30\text{s}$$
$$T_{\text{deobfuscate}} = 45\text{s}, \quad T_{\text{reconstruct}} = 20\text{s}, \quad T_{\text{report}} = 5\text{s}$$

Sequential toplam: $T_{\text{seq}} = 2 + 60 + 30 + 45 + 20 + 5 = 162\text{s}$
Concurrent (static $\parallel$ dynamic $\parallel$ deobfuscate):

$$T_{\text{parallel}} = 2 + \max(60, 30, 45) + 20 + 5 = 87\text{s} \tag{2.2}$$

$$\text{Speedup} = \frac{162}{87} = 1.86\times \tag{2.3}$$

Amdahl formuelue ile: Paralel kisim orani $p = (60 + 30 + 45)/162 = 0.83$:

$$S_{\text{teorik}} = \frac{1}{(1 - 0.83) + 0.83/3} = \frac{1}{0.17 + 0.28} = 2.22\times \tag{2.4}$$

1.86x speedup icin concurrent pipeline karmasikligi (lock'lar, barrier'lar, partial failure handling) eklenmemis. Dogru karar — cuenkue:

1. **Kaynak catismasi:** Ghidra 2–4 GB RAM, Frida + Babel ayri agir surecler. Paralel calissa 8+ GB — OOM riski.
2. **Gizli bagimliliklar:** `stages.py`'de deobfuscate stage'i `requires=["identify"]` diyor ama gercekte static analiz sonuclarini da kullaniyor. "Hidden coupling" paralelde race condition yaratir.

3. **Debugging:** Sequential'da hata hangi stage'de oldugu acik. Concurrent'da cok daha karisik.

2.2 PipelineContext — Shared State Pattern

`pipeline.py:41`'de tanimlanan `PipelineContext`, stage'ler arasi paylasilan veri yapisidir:

```
@dataclass
class PipelineContext:
    target: TargetInfo        # Hedef bilgileri
    workspace: Workspace      # Calisma dizini
    config: Config            # Konfiguerasyon
    results: dict[str, StageResult]  # Onceki sonuclar
    extra: dict[str, Any]     # Ek paylasilam veri
```

Listing 2.1: PipelineContext veri yapisi

Bu tasarim, Martin Fowler'in **Introduce Parameter Object** refactoring kalibinin bir uygulamasidir. Avantajlari:

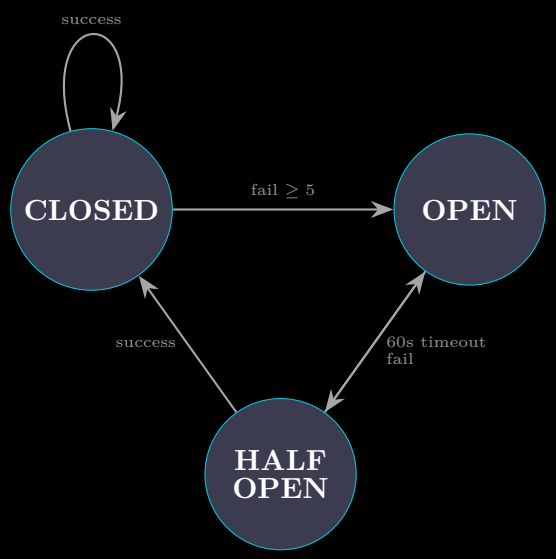
- Stage'ler arasinda explicit veri gecisi
- Yeni veriler `extra` dict'e eklenebilir (genisletilebilirlik)
- Tum state tek bir nesnede — test edilebilirlik

2.3 Error Recovery — Circuit Breaker Pattern

2.3.1 State Machine Modeli

Circuit breaker uec durumlu bir state machine'dir:

$$\text{States} = \{\text{CLOSED}, \text{OPEN}, \text{HALF_OPEN}\} \tag{2.5}$$



2.3.2 Exponential Backoff

Yeniden deneme gecikmesi:

$$\text{delay}(k) = \min\Big(\text{base} \cdot 2^k,\ \text{max_delay}\Big) \tag{2.6}$$

burada k = deneme sayisi, base = 1.0s, max_delay = 30s.

Toplam beklenen bekleme sueresi (n deneme icin):

$$E[T] = \sum_{k=0}^{n-1} \min\Big(1.0 \cdot 2^k,\ 30\Big) = 1 + 2 + 4 + 8 + 16 + 30 + \cdots \tag{2.7}$$

$n = 3$ icin: $E[T] = 1 + 2 + 4 = 7$ saniye.

2.3.3 Neden Exponential Backoff?

Farkli backoff stratejilerinin karsilastirmasi:

Strateji	Formuul	$n = 3$	$n = 5$	Bueyueme
Linear	$n(n+1)/2$	6s	15s	$\mathcal{O}(n^2)$
Quadratic	$n(n+1)(2n+1)/6$	14s	55s	$\mathcal{O}(n^3)$
Exponential	$2^n - 1$	7s	31s	$\mathcal{O}(2^n)$

Table 2.1: Backoff stratejilerinin toplam bekleme suereleri.

Exponential tercih sebebi: **geri basinc (backpressure)**. Servis down ise ilk retry'lar hizli, sonrakiler dramatik artan — servise "nefes alma" sueresi verir. Linear bunu yapmaz.

2.3.4 Jitter

Kodda **zaten var** (`error_recovery.py:205`):

```
if jitter:
    jitter_range = delay * 0.2
    delay += random.uniform(-jitter_range, jitter_range)
```

$\pm 20\%$ uniform jitter. Amazon AWS "full jitter" ($[0, \text{delay}]$) oenerir, ama Karadul'da tek kullanici var — thundering herd problemi yok.

2.3.5 Basari Olasiligi

k deneme sonrasi basari olasiligi (her denemenin bagimsiz basari olasiligi p):

$$P(\text{basari sonrasi } k \text{ deneme}) = 1 - (1 - p)^k \tag{2.8}$$

$p = 0.8$ (tipik gecici hata) ve $k = 3$: $P = 1 - 0.2^3 = 1 - 0.008 = 0.992$ — %99.2 basari.

Bu Bolumun Ozeti

Karadul pipeline'i, 6 stage'li sequential bir DAG execution engine'dir. PipelineContext shared state pattern ile veri paylasimi saglar. ErrorRecovery, circuit breaker + exponential backoff ile tuem stage'leri korur. Graceful degradation sayesinde tek bir stage basarisizligi pipeline'i durdurmaz.

Chapter 3

Hedef Tespit Sistemi

Karadul'un ilk asamasi, analiz edilecek dosyanin tipini, programlama dilini ve formatini tespit etmektir.

3.1 Magic Number Sistemi

Her binary dosya formati, dosyanin basinda benzersiz bir "magic number" tasir. Bu degerler formati tanimlar:

$$\text{magic} : \{0,1\}^{32} \to \text{Format} \tag{3.1}$$

Format	Magic (hex)	ASCII	Oykue
Mach-O 64-bit	`0xFEEDFACF`	—	"feed face" (Apple)
Mach-O 32-bit	`0xFEEDFACE`	—	"feed face" (Apple)
Universal	`0xCAFEBABE`	—	"cafe babe" (NeXT/Java)
ELF	`0x7F454C46`	`.ELF`	ELF standard header
PE/DOS	`0x4D5A`	`MZ`	Mark Zbikowski (DOS yaraticisi)
ZIP/JAR/APK	`0x504B0304`	`PK..`	Phil Katz (PKZIP yaraticisi)

Table 3.1: Binary dosya formatlarinin magic byte'lari.

3.1.1 Collision Olasiligi

Rastgele bir dosyanin belirli bir 4-byte magic ile baslama olasiligi:

$$P(\text{false positive}) = \frac{1}{2^{32}} \approx 2.33 \times 10^{-10} \tag{3.2}$$

Bu son derece duseuk bir olasiliktir, ancak Karadul ek dogrulamalar da yapar (icerik analizi, dosya uzantisi).

3.1.2 Big-Endian vs Little-Endian Okuma

Magic byte'lar big-endian sirada okunur (`struct.unpack(">I")`, `target.py:607`). Mach-O formati "cigam" (magic'in tersi) varyantlarina da sahiptir:

$$\text{CIGAM}(m) = \text{bswap32}(m) \tag{3.3}$$

Oernegin: `0xFEEDFACF` → `0xCFFAEDFE` (little-endian ARM islemcilerde).

3.2 Dil Tespit Algoritmasi

Binary icindeki string pattern'lerden dil tespiti yapar (`target.py:470`):

3.2.1 Puanlama Sistemi

Her dil icin bilinen string marker'lar taranir:

$$\text{score}(L) = \sum_{s \in \text{markers}(L)} \mathbb{1}[s \in \text{binary_text}] \tag{3.4}$$

En yueksek skorlu dil secilir: $\hat{L} = \arg\max_L \text{score}(L)$.

Bayes Siniflandirici Perspektifi

Bu puanlama sistemi aslinda bir Naive Bayes siniflandiricisinin basitlestirilmis halidir:

$$P(L \mid S_1, \ldots, S_n) \propto P(L) \prod_{i=1}^{n} P(S_i \mid L) \tag{3.5}$$

Mevcut implementasyonda $P(L)$ uniform (esit prior) ve $P(S_i \mid L) \in \{0, 1\}$ (deterministik).

3.3 JavaScript Bundler Tespiti

JS dosyalarinin ilk 64KB'i okunarak bundler tespiti yapilir:

Signature	Bundler
`__webpack_require__`	webpack
`parcelRequire`	parcel
`System.register`	systemjs
`_rollupPluginBabelHelpers`	rollup
`define([`	amd/requirejs

Table 3.2: JavaScript bundler signature'lari.

3.4 SHA-256 Hashing

Dosya buetunluegue icin SHA-256 streaming hash:

$$H = \text{SHA-256}\left(\bigoplus_{i=1}^{\lceil n/8192 \rceil} \text{chunk}_i\right) \tag{3.6}$$

8KB chunk boyutu disk I/O vs CPU tradeoff'u optimize eder. Collision olasiligi:

$$P(\text{collision}) \approx \frac{n^2}{2^{257}} \tag{3.7}$$

$n = 10^{12}$ dosya icin bile: $P(\text{collision}) \approx 10^{-54}$.

Bu Bolumun Ozeti

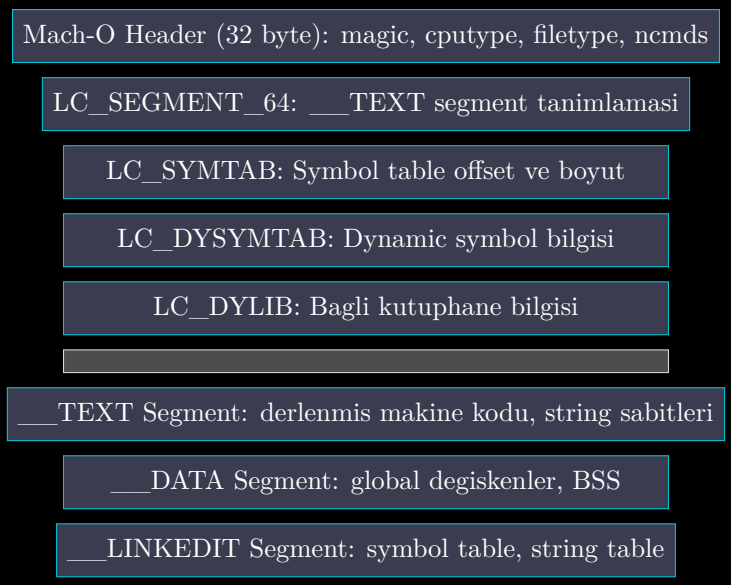
Hedef tespit sistemi, magic byte analizi, string-based dil tespiti, bundler signature eslestirmesi ve SHA-256 hashing kullanarak 12 farkli dosya tipini ve 13 programlama dilini otomatik olarak tespit eder. False positive olasiligi astronomik derecede duseuktueer.

3.5 Mach-O Format Detaylari

Apple platformlarinda kullanilan Mach-O (Mach Object) formati, NeXTSTEP mirasinin urunudur. Carnegie Mellon'daki Mach kernel projesi (1985) icin tasarlanmistir.

3.5.1 Header → Load Commands → Segments → Sections

Mach-O dosyasi katmanli bir yapidir:



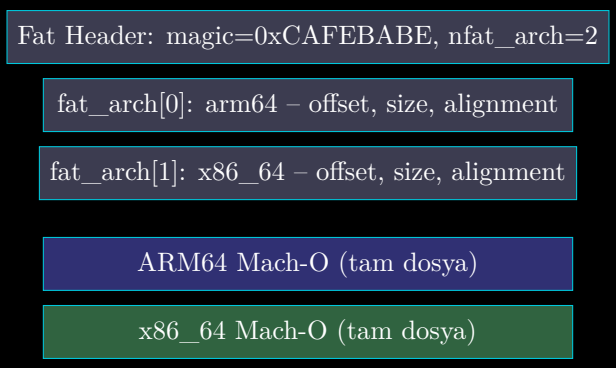
Header (mach_header_64, 32 byte):

- `magic` (4 byte): `0xFEEDFACF` (64-bit) veya `0xFEEDFACE` (32-bit)
- `cputype`: `CPU_TYPE_ARM64 = 0x0100000C`, `CPU_TYPE_X86_64 = 0x01000007`
- `filetype`: `MH_EXECUTE` (2), `MH_DYLIB` (6), `MH_BUNDLE` (8)
- `ncmds`: Load command sayisi
- `flags`: `MH_PIE` (position independent), `MH_TWOLEVEL` (iki seviyeli namespace)

Segmentler: Bellek yuekleme birimi. `__TEXT` segmenti yurutuluebilir ama yazilamaz (W^X, code signing icin kritik). `__DATA` segmenti okunur-yazilir.

3.5.2 Universal Binary (Fat Binary)

Birden fazla mimari icin tek dosyada derlenmis binary:



Karadul'un `macho.py`'sindaki `analyze_static()` bunu dogru handle eder:

```
if target.target_type == TargetType.UNIVERSAL_BINARY:
    # lipo -thin arm64 ile sadece ARM slice'i cikar
    proc = subprocess.run(["lipo", str(binary_path), "-thin", "
        arm64", ...])
```

Neden universal binary hala kullanilir: Apple 2020'de Intel'den ARM64'e gecti. Universal binary gecis doeneminde zorunlu — M-serisi ARM64, eski Mac'ler x86_64 calistirir. Rosetta 2 JIT translation katmani sadece ARM64 slice yoksa devreye girer.

3.5.3 CAFEBABE Cakismasi

Potansiyel Sorun

`0xCAFEBABE` magic hem Java class dosyalarinda hem Apple universal binary'lerinde kullanilir. Ayrim: Java'da magic'ten sonra minor/major version byte'lari gelir (oernegin `0x0000003D` = Java 17), fat binary'de ise mimari sayisi gelir (1–9 arasi kueceuk integer). Mevcut Karadul implementasyonu bu ayrimi **yapmiyor** — potansiyel false positive kaynagi.

3.6 ELF Format ve Go Binary Tespiti

3.6.1 ELF Yapisi

ELF (Executable and Linkable Format), Unix System V Release 4 (1989) ile standartlastirilmistir:

- **e_ident[16]:** Ilk 16 byte — magic (`7F 45 4C 46`), class (32/64-bit), endianness, version, OS/ABI
- **Program Headers:** Runtime yuekleme bilgisi — `PT_LOAD`, `PT_INTERP`, `PT_DYNAMIC`
- **Section Headers:** Link-time bilgisi — `.text`, `.rodata`, `.data`, `.bss`, `.symtab`

Kritik fark: Program headers = runtime view (loader kullanir), section headers = link-time view (linker kullanir). Stripped binary'de section headers **silinebilir** ama program headers silinmez.

3.6.2 GOPCLNTAB (Go Program Counter Line Number Table)

Go binary'lerinin en onemli ic yapisi. Her Go binary'de bulunur — **stripped olsa bile** genellikle kalir, cueuenkue Go runtime panic/recover/stack trace icin buna bagimlidir.

GOPCLNTAB icerigi:

- Her fonksiyonun baslangic adresi (PC)
- Fonksiyon adi (tam package path: `main.(*Server).HandleRequest`)
- Kaynak dosya yolu
- Satir numarasi mapping'i (PC → satir)
- Stack frame boyutu

Magic byte evrimi (Karadul 4 versiyonu da handle ediyor):

Magic	Go Versiyon	Neden Degisti
`FB FF FF FF 00 00`	Go 1.2–1.15	Orijinal format
`FA FF FF FF 00 00`	Go 1.16–1.17	Format yeniden tasarim
`F0 FF FF FF 00 00`	Go 1.18–1.19	Register-based ABI
`F1 FF FF FF 00 00`	Go 1.20+	Gueuencel format

Table 3.3: GOPCLNTAB magic byte evrimi.

3.7 PE Format ve .NET/Delphi Ayrimi

3.7.1 PE Yapisi

PE (Portable Executable) formati, her Windows .exe, .dll, .sys dosyasi:

- **DOS Header** (64 byte): `e_magic="MZ"` (Mark Zbikowski, DOS yaraticisi), `e_lfanew` = PE offset
- **DOS Stub** (∼64 byte): "This program cannot be run in DOS mode"
- **PE Signature** (4 byte): `"PE\0\0"`
- **COFF Header** (20 byte): Machine, NumberOfSections, TimeDateStamp
- **Optional Header**: Entry point, ImageBase, **Data Directories** (16 adet)
- **Data Directory index 14** = CLR Runtime Header — bu entry sifirdan bueyuekse **.NET assembly**

3.7.2 .NET Tespiti: mscoree.dll

Her .NET executable'da import table'da `mscoree.dll` referansi bulunur:

```
# target.py satir 524-536
if b"mscoree.dll" in data or b"_CorExeMain" in data:
    return True
if b"mscorlib" in data or b"System.Runtime" in data:
    return True
```

False positive neredeyse imkansiz: `mscoree.dll` string'i yalnizca .NET binary'lerinde bulunur.

$$P(.NET \mid \text{mscoree.dll bulundu}) \approx 0.999 \tag{3.8}$$

3.7.3 Delphi Binary Tespiti

Delphi binary'leri native x86/x64 makine kodu icerir. Ayirt edici ozellikler:

RTTI (Runtime Type Information): Her class icin otomatik uretilir, binary'nin `.rdata` section'inda goemuelueduer:

```
# Delphi mangled name pattern:
# @Unit@Class@Method$qqr
pattern = rb"@([A-Z]\w+)@([A-Z]\w+)@([A-Za-z_]\w*)\$qqr"
```

`$qqr` suffixi "no parameters" anlamina gelir (Delphi mangling convention).

Tespit hiyerarsisi: `target.py`'da oencelik sirasi dogru:

1. `_is_dotnet()` → DOTNET_ASSEMBLY
2. `_is_delphi()` → DELPHI_BINARY
3. Fallback → PE_BINARY

.NET kontrolu daha belirgin (mscoree.dll) bu yueuezden oence yapilmali.

3.8 Dil Tespit Algoritmasi — Bayes Perspektifi

3.8.1 Mevcut Puanlama Sistemi

`target.py:470`'deki dil tespiti basit bir sayma yaklasimi kullaniyor:

$$\text{score}(L) = \sum_{s \in \text{markers}(L)} \mathbb{1}[s \in \text{binary_text}] \tag{3.9}$$

Her signature esit agirlikta — `rust_begin_unwind` ile `std::` ayni puani verir.

3.8.2 Naive Bayes Formulasyonu

Bu problem bir Bayes siniflandiricidir:

$$P(L \mid S_1, \ldots, S_n) \propto P(L) \prod_{i=1}^{n} P(S_i \mid L) \tag{3.10}$$

Prior olasiliklar platform-conditioned olabilir:

- Mach-O ise: Swift/Obj-C prior'u yueksek
- ELF ise: C/Go prior'u yueksek
- PE ise: C++/.NET/Delphi prior'u yueksek

Signature likelihood'lari:

Signature	$P(S \mid \text{Rust})$	$P(S \mid \text{C++})$	$P(S \mid \text{Go})$	Gueuec
`rust_begin_unwind`	0.98	0.001	0.001	Cok gueueclue
`std::`	0.15	0.95	0.02	Cakisma riski
`runtime.gopanic`	0.001	0.001	0.99	Cok gueueclue
`objc_msgSend`	0.001	0.01	0.001	Obj-C kesin

Table 3.4: Dil tespit signature likelihood degerleri.

Iyilestirme Oenerisi

Mevcut esit agirlikli sayma yerine agirlikli puanlama daha dogru olur:

```
_LANGUAGE_WEIGHTS = {
    "rust_begin_unwind": (Language.RUST, 3),   # Cok guclu
    "std::":              (Language.CPP, 1),   # Zayif (cakisma)
    "runtime.gopanic":    (Language.GO, 3),    # Cok guclu
}
```

3.8.3 False Positive Analizi

En yueksek risk: `std::` — Rust binary'leri C++ standard library ile linklenebilir (FFI), Go binary'leri cgo ile C++ kodu icerebilir. Bu durumda `std::` yanlis dili goesterir.

En yueksek false negative riski: Stripped binary'ler (agresif `-s -w`) cogu string'i siler.

Tutarsizlik

`_detect_language_from_binary()` 1MB okurken, `_is_go_binary()` 2MB okuyor. Go binary'lerin bueyuekluegueuenden mantikli ama genel dil tespiti icin de 2MB daha dogru sonuc verir.

3.9 Bundler Tespit Heuristikleri

3.9.1 webpack Runtime Yapisi

webpack duenyanin en yaygin JS bundler'idir. Temel yapi:

```
var __webpack_modules__ = {
  "./src/index.js": function(module, exports,
      __webpack_require__) {
    // Modul kodu
  }
};
function __webpack_require__(moduleId) {
  var cachedModule = __webpack_module_cache__[moduleId];
  if (cachedModule !== undefined) return cachedModule.exports;
  var module = __webpack_module_cache__[moduleId] = { exports:
      {} };
  __webpack_modules__[moduleId](module, module.exports,
      __webpack_require__);
  return module.exports;
}
```

`__webpack_require__` fonksiyonu: (1) Cache'e bak, (2) modul fonksiyonunu CommonJS convention ile cagir, (3) sonucu cache'e yaz.

3.9.2 esbuild — Tespiti Zor

esbuild (Go ile yazilmis) minimal output ueretir — belirgin bir runtime signature'i **yoktur**. Bu kasitli tasarim karari. Karadul'un mevcut sistemi esbuild ciktisini tespit **edemez** — gueuencel JS ekosisteminde giderek onemli bir eksiklik.

3.9.3 Bundler Tespitinin Ooenemi

Bundler tespiti deobfuscation stratejisini belirler:

- **webpack:** Module map parse edilebilir, her modul ayri dosyaya yazilabilir
- **parcel:** Benzer module map yapisi
- **rollup:** Tek dosya, modul sinirlari silindigi icin parcalamak zor
- **esbuild:** Agresif inlining, moduelleri ayirmak neredeyse imkansiz

3.10 SHA-256 Hashing — Detayli Analiz

3.10.1 Merkle-Damgård Construction

SHA-256 su pipeline'i izler:

$$M \xrightarrow{\text{padding}} M_1|M_2|\cdots|M_n \quad \text{(512-bit bloklar)} \tag{3.11}$$

$$\begin{aligned} H_0 &= \text{IV (Initial Value, 256-bit sabit)} \\ H_i &= f(H_{i-1}, M_i) \quad i = 1, 2, \ldots, n \\ \text{Hash} &= H_n \end{aligned}$$

Compression function f: 64 round, her round'da Ch, Maj, Σ fonksiyonlari (bitwise operations).

3.10.2 8KB Chunk Boyutu Optimalligi

8KB = 128 $\times$ 64B (SHA-256 compression block size) — mueuekemmel hizalanma. L1 cache'te ($\sim$32–64KB) rahatca sigar.

Chunk Boyutu	Throughput	Bellek
256 B	∼200 MB/s	256 B
4 KB	∼800 MB/s	4 KB
8 KB	**∼850 MB/s**	**8 KB**
64 KB	∼870 MB/s	64 KB
1 MB	∼860 MB/s	1 MB

Table 3.5: Hash chunk boyutu vs throughput. 8KB "sweet spot".

3.10.3 Collision Olasiligi

Birthday paradox'undan:

$$P(\text{collision}, n \text{ dosya}) = 1 - e^{-n^2/(2 \cdot 2^{256})} \approx \frac{n^2}{2^{257}} \tag{3.12}$$

$n = 10^{12}$ (1 trilyon dosya): $P \approx 10^{18}/2^{257} \approx 10^{-59}$ — astronomik derecede dueueseek.

$P = 0.5$ icin gerekli n: $n \approx 2^{128} \approx 3.4 \times 10^{38}$ dosya — evrendeki atom sayisindan ($\sim 10^{80}$) bile kat kat azdir, yine de collision pratikte imkansiz.

Chapter 4

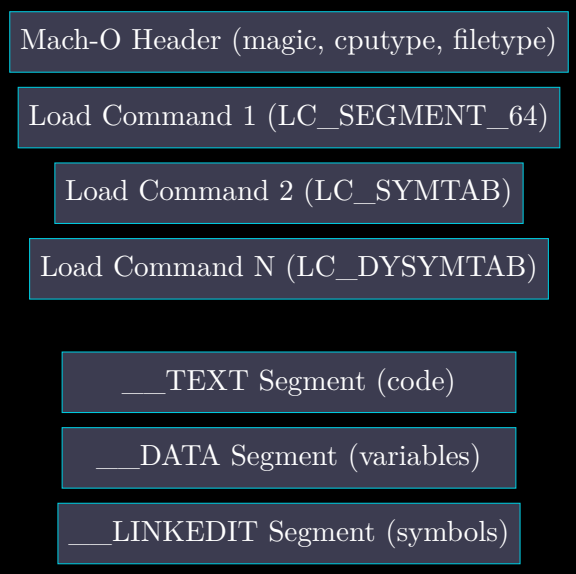
Statik Analiz

Pipeline'in ikinci asamasi olan statik analiz, dosyayi calistirmadan yapisini analiz eder.

4.1 Binary Statik Analiz

4.1.1 Mach-O Analizi

Apple platformlarinda kullanilan Mach-O formatinin yapisi:



4.1.2 Symbol Extraction

Binary'den sembol cikarmak icin `nm` ve `otool` araclari kullanilir:

- `nm -gU binary`: Global tanimli semboller

- `otool -tV binary`: Disassemble edilmis kod
- `strings binary`: Okunabilir string'ler

4.1.3 Ghidra Headless Analiz

Buyuek binary'ler icin Ghidra headless mode:

```
analyzeHeadless /tmp/project ProjectName \
  -import /path/to/binary \
  -scriptPath /path/to/scripts \
  -postScript ExportDecompiled.java \
  -deleteProject
```

Listing 4.1: Ghidra headless invocation

Ghidra, binary'yi P-code adli bir intermediate representation'a donuesteruer, sonra C koduna decompile eder.

4.2 JavaScript Statik Analiz

JS analizi Babel AST parser kullanir:

```
const ast = parser.parse(sourceCode, {
  sourceType: "module",
  plugins: ["jsx", "typescript", "decorators"]
});
// AST traversal
traverse(ast, {
  FunctionDeclaration(path) {
    functions.push(path.node.id.name);
  },
  StringLiteral(path) {
    strings.push(path.node.value);
  }
});
```

Listing 4.2: Babel AST parse ornegi

Cikarilan bilgiler:

- Fonksiyon sayisi ve isimleri

- String literal'ler
- Import/export ifadeleri
- Webpack modul ID'leri

Bu Bolumun Ozeti

Statik analiz, binary'ler icin sembol tablosu, Ghidra decompilation ve string extraction; JavaScript icin Babel AST parsing, fonksiyon/string/import cikartma islemleri gerceklestirir. Buyuek dosyalar icin chunked processing destegi vardir.

4.3 Hedef Tespit Sistemine Genel Bakis

Karadul pipeline'inin ilk asamasi olan *hedef tespit sistemi*, analiz edilecek dosyanin turunu, dilini ve metadata'sini belirler. Bu asama, sonraki tum islemlerin temelini oluturur: yanlis bir tespit, yanlis analyzer zincirine yonlendirme ve basarisiz analiz demektir.

Hedef Tespit Problemi

Bir $b \in \mathcal{B}$ binary dosyasi verildiginde, hedef tespit fonksiyonu:

$$\mathcal{D} : \mathcal{B} \longrightarrow \mathcal{T} \times \mathcal{L} \times \mathcal{M} \tag{4.1}$$

seklinde tanimlanir. Burada:

- $\mathcal{T} = \{\text{MACHO}, \text{ELF}, \text{PE}, \text{JS_BUNDLE}, \text{ELECTRON}, \text{GO}, \text{UNIVERSAL}, \ldots\}$: 12 elemanli hedef tipi kumesi.
- $\mathcal{L} = \{\text{C}, \text{C++}, \text{Rust}, \text{Go}, \text{Swift}, \text{ObjC}, \text{JavaScript}, \ldots\}$: 13 elemanli dil kumesi.
- $\mathcal{M}$: metadata sozlugu (mimari, bundler, framework vb.).

Tespit sistemi `target.py` dosyasinda `TargetDetector` sinifi olarak implement edilmistir. Toplam 630 satir Python kodundan olusur.

4.4 Magic Byte Sistemi

Her binary dosya formati, dosyanin ilk birkac byte'inda kendine ozgu bir *magic number* barindirir. Bu sabitler, dosya turunu hizli ve guvenilir sekilde tanimlama amaciyla kullanilir.

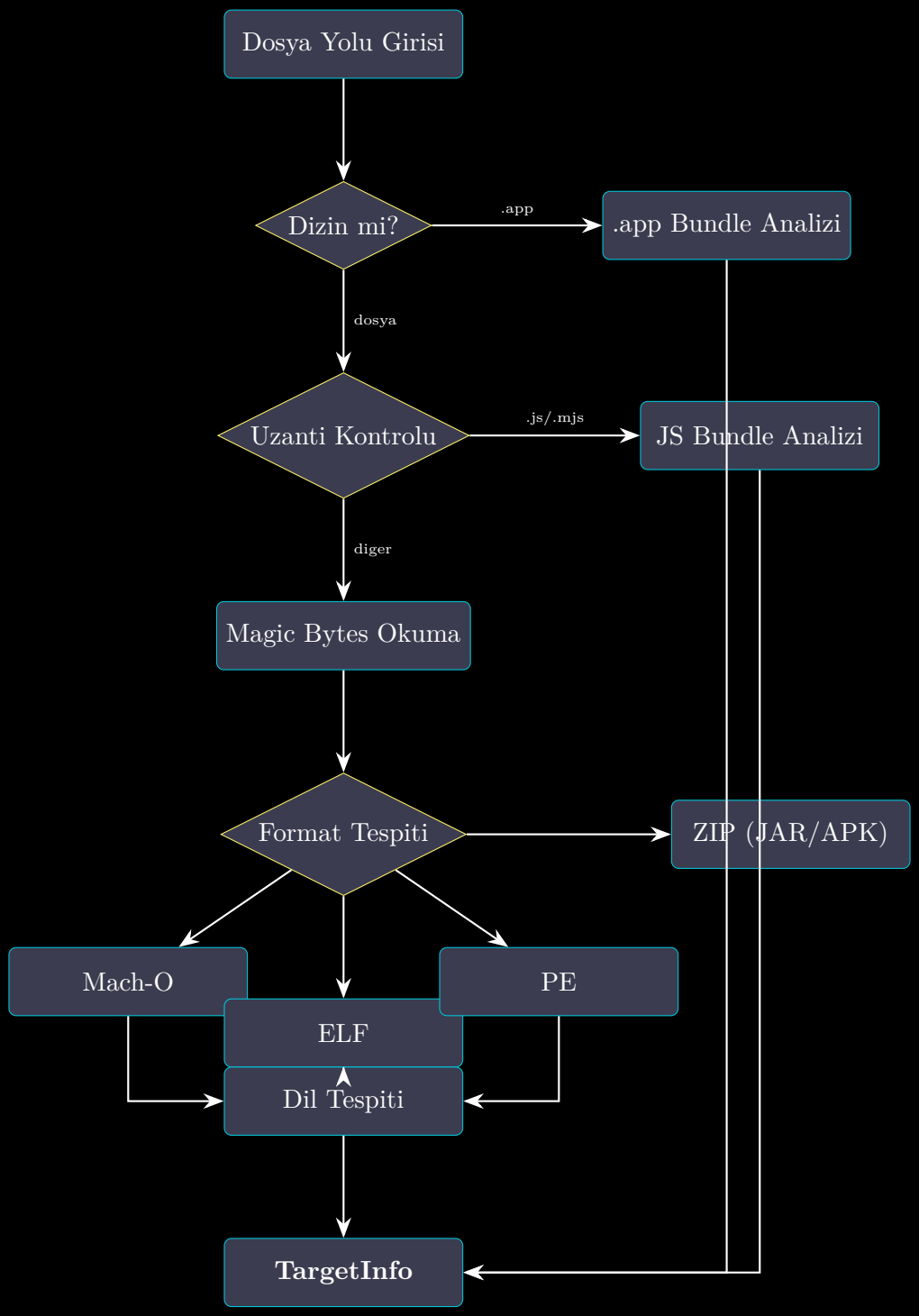


Figure 4.1: Hedef tespit akis diyagrami. Dosya yolundan TargetInfo'ya uzanan karar agaci.

4.4.1 Temel Magic Byte Tanimlari

`target.py:L53-L66`'da tanimlanan sabitler:

Format	Magic (hex)	Endianness	Python Sabiti
Mach-O 64-bit	`0xFEEDFACF`	Big-endian	`_MACHO_64_MAGIC`
Mach-O 32-bit	`0xFEEDFACE`	Big-endian	`_MACHO_32_MAGIC`
Mach-O 64 Cigam	`0xCFFAEDFE`	Little-endian	`_MACHO_64_CIGAM`
Mach-O 32 Cigam	`0xCEFAEDFE`	Little-endian	`_MACHO_32_CIGAM`
Universal	`0xCAFEBABE`	Big-endian	`_UNIVERSAL_MAGIC`
ELF	`7F 45 4C 46`	—	`_ELF_MAGIC`
PE (DOS)	`4D 5A`	—	`_PE_MAGIC`
ZIP (JAR/APK)	`50 4B 03 04`	—	`_ZIP_MAGIC`

Table 4.1: Desteklenen binary formatlarinin magic byte'lari.

4.4.2 Big-Endian ve Little-Endian (Cigam) Kavrami

Mach-O formati iki farkli byte sirasini destekler. "Cigam" sozcugu "magic" kelimesinin tersten yazilisidir ve little-endian kodlamayi isaret eder.

Byte Order

n byte'lik bir v tamsayisi icin:

$$v_{\text{BE}} = \sum_{i=0}^{n-1} b_i \cdot 256^{n-1-i} \quad \text{(big-endian)} \tag{4.2}$$

$$v_{\text{LE}} = \sum_{i=0}^{n-1} b_i \cdot 256^{i} \quad \text{(little-endian)} \tag{4.3}$$

Burada b_i, dosyadan okunan i'inci byte'dir.

Sayisal Ornek. `0xFEEDFACF` sabitinin bellekteki gorunumu:

Big-endian (Magic):

`0xFE`	`0xED`	`0xFA`	`0xCF`	addr 0..3

Little-endian (Cigam):

`0xCF`	`0xFA`	`0xED`	`0xFE`	addr 0..3

`_read_magic` metodu (`target.py:L604-L614`) dosyanin ilk 4 byte'ini her zaman big-endian olarak okur:

```python
@staticmethod
def _read_magic(path: Path) -> int | None:
    """Dosyanin ilk 4 byte'ini big-endian uint32 olarak oku."""
    try:
        with open(path, "rb") as f:
            data = f.read(4)
        if len(data) < 4:
            return None
        return struct.unpack(">I", data)[0]  # ">" = big-endian, "I" = uint32
    except OSError:
        return None
```

Listing 4.3: struct.unpack ile magic byte okuma

Bu tasarim sayesinde, big-endian Mach-O dosyalari dogrudan `0xFEEDFACF` ile eslesirken, little-endian dosyalar `0xCFFAEDFE` degerini dondurur ve `_MACHO_64_CIGAM` sabiti ile eslestirilir.

Karakter	Anlam	Boyut
`>`	Big-endian byte order	—
`<`	Little-endian byte order	—
`I`	Unsigned 32-bit integer	4 byte
`H`	Unsigned 16-bit integer	2 byte
`Q`	Unsigned 64-bit integer	8 byte
`B`	Unsigned 8-bit integer	1 byte

Table 4.2: Python `struct.unpack` format karakterleri.

struct.unpack Format Karakterleri.

4.5 Mach-O Format Yapisi

Apple'in Mach-O (Mach Object) formati, macOS, iOS, watchOS ve tvOS uzerinde kullanilan native binary formatidir. Karadul'un birincil hedef formatidir.

4.5.1 Mach-O Header Yapisi

Mach-O dosyasinin ilk 32 byte'i `mach_header_64` yapisini icerir:

Alan	Tip	Offset	Aciklama
`magic`	uint32	0	`0xFEEDFACF` (64-bit)
`cputype`	int32	4	CPU ailesi (ARM64: `0x100000C`)
`cpusubtype`	int32	8	CPU alt tipi
`filetype`	uint32	12	Dosya tipi (2=executable, 6=dylib)
`ncmds`	uint32	16	Load command sayisi
`sizeofcmds`	uint32	20	Tum load command'larin toplam boyutu
`flags`	uint32	24	Binary flag'leri (PIE, stripped vb.)
`reserved`	uint32	28	Rezerve (64-bit icin)

Table 4.3: Mach-O 64-bit header alanlari.

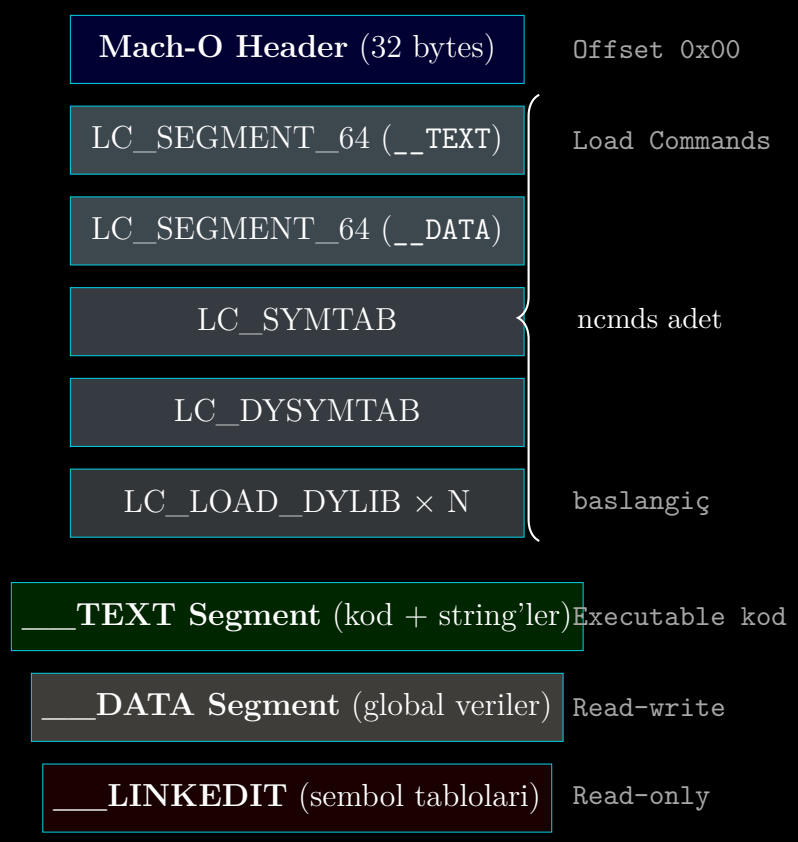


Figure 4.2: Mach-O 64-bit dosya yapisi. Header + Load Commands + Segments.

4.5.2 Load Commands

Load command'lar, isletim sisteminin binary'yi nasil yukleyecegini ve baglayacagini tarif eder. Karadul'un `MachOAnalyzer` sinifi (`macho.py:L389-L391`) `otool -l` komutuyla tum load command'lari cikarir.

Onemli load command turleri:

`LC_SEGMENT_64`: Bir segment'in bellek adresini, boyutunu ve icerigini tanimlar. Her segment birden fazla *section* icerebilir.

`LC_SYMTAB`: Sembol tablosunun offset ve boyutunu belirtir.

`LC_DYSYMTAB`: Dinamik sembol tablosu bilgilerini icerir.

`LC_LOAD_DYLIB`: Yuklenmesi gereken bir dinamik kutuphanenin yolunu belirtir.

`LC_UUID`: Binary'nin benzersiz tanimlayicisi (build ID).

`LC_CODE_SIGNATURE`: Apple code signing bilgisi.

4.5.3 Segment ve Section Yapisi

Mach-O dosyasindaki temel segmentler:

Segment	Section'lar	Aciklama
`__TEXT`	`__text, __stubs, __stub_helper, __cstring, __const`	Executable kod, sabit only + execute.
`__DATA`	`__data, __bss, __common, __la_symbol_ptr, __got`	Global degiskenler, BS
`__DATA_CONST`	`__const, __cfstring, __objc_classlist`	Salt-okunur veri (ru protect).
`__LINKEDIT`	(monolitik)	Sembol tablolari, strin signature. Read-only.

Table 4.4: Mach-O segmentleri ve section'lari.

Karadul'un `lief` entegrasyonu (`macho.py:L431-L495`) tum segment ve section bilgilerini programatik olarak cikarir:

```
1 binary = lief.parse(str(binary_path))
2 for seg in binary.segments:
3     result["segments"].append({
4         "name": seg.name,
5         "virtual_address": seg.virtual_address,
6         "virtual_size": seg.virtual_size,
```

```
7           "file_offset": seg.file_offset,
8           "file_size": seg.file_size,
9       })
10  for sec in binary.sections:
11      result["sections"].append({
12          "name": sec.name,
13          "segment": sec.segment.name,
14          "size": sec.size,
15          "offset": sec.offset,
16      })
```

Listing 4.4: lief ile segment/section cikartma

4.6 ELF Format Yapisi

ELF (Executable and Linkable Format), Linux ve diger Unix-benzeri sistemlerde kullanilan standart binary formatidir. Magic byte'i `7F 45 4C 46` (\x7fELF) ile baslar (`target.py:L64`).

4.6.1 ELF Header

Alan	Tip	Offset	Aciklama
`e_ident[0..3]`	byte[4]	0	Magic: `7F 45 4C 46`
`e_ident[4]`	uint8	4	Class: 1=32-bit, 2=64-bit
`e_ident[5]`	uint8	5	Data: 1=LE, 2=BE
`e_type`	uint16	16	Tip: 2=ET_EXEC, 3=ET_DYN
`e_machine`	uint16	18	Mimari: 0x3E=x86_64, 0xB7=AArch64
`e_entry`	uint64	24	Entry point adresi
`e_phoff`	uint64	32	Program header table offset
`e_shoff`	uint64	40	Section header table offset
`e_phnum`	uint16	56	Program header sayisi
`e_shnum`	uint16	60	Section header sayisi

Table 4.5: ELF 64-bit header alanlari (secilmis).

4.6.2 ELF ve Mach-O Karsilastirmasi

Karadul, ELF binary'ler icin `otool` adimlarini atlar (`macho.py:L122-L143`), dogrudan `nm`, `strings` ve Ghidra headless analiz kullanir.

Kavram	Mach-O	ELF
Kod bolumu	`__TEXT,__text`	`.text`
Veri bolumu	`__DATA,__data`	`.data`
Salt-okunur	`__DATA_CONST`	`.rodata`
Sembol tablosu	`LC_SYMTAB`	`.symtab` / `.dynsym`
Dinamik baglama	`LC_LOAD_DYLIB`	`.dynamic` + `DT_NEEDED`
Magic number	`0xFEEDFACF`	`7F 45 4C 46`

Table 4.6: Mach-O ve ELF terminoloji eslestirmesi.

4.7 PE Format Yapisi

PE (Portable Executable), Microsoft Windows'un native binary formatidir. Magic byte'i `4D 5A` (ASCII "MZ") ile baslar ve *DOS header* icerir.

4.7.1 PE Dosya Yapisi

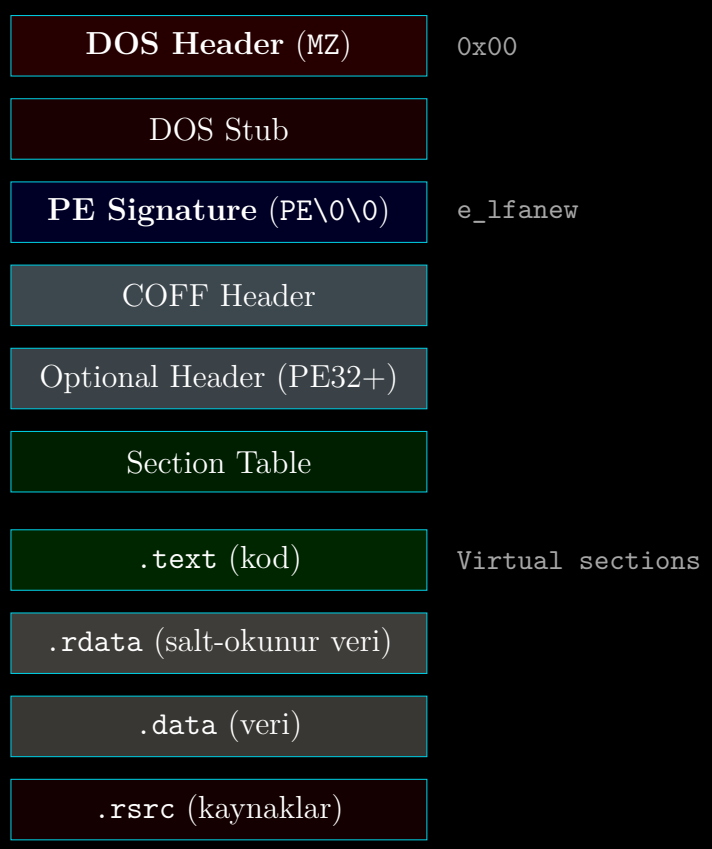


Figure 4.3: PE dosya yapisi. DOS header'daki `e_lfanew` alani PE signature'a isaret eder.

Karadul'un PE tespit mantigi (`target.py:L402-L430`):

1. Ilk 2 byte `MZ` mi? → PE adayi.

2. .NET kontrolu: `_is_dotnet()` metodu `mscoree.dll` veya `_CorExeMain` arar.
3. Delphi kontrolu: `_is_delphi()` metodu "Borland Delphi", mangled name pattern'leri arar.
4. Hic biri degilse: genel `PE_BINARY` olarak siniflandirir.

4.8 Universal Binary (Fat Binary)

Apple'in Universal Binary formati, birden fazla mimarinin tek dosyada birlestirilmesini saglar. Magic byte'i `0xCAFEBABE`'dir.

Dikkat: Java Class File Cakismasi

`0xCAFEBABE`, Java class dosyalarinin da magic byte'idir. Ancak pratikte bu bir sorun yaratmaz, cunku:

1. Java class dosyalari genellikle .jar (ZIP) icinde gelir ve `PK` magic'i ile baslar.
2. Tek basina `.class` dosyalari nadiren analiz hedefi olur.
3. Karadul, `0xCAFEBABE` goruldugunde dosya boyutu ve icerik heuristikleri uygular.

Universal binary tespit edildikten sonra (`macho.py:L85-L110`), `lipo` komutu ile tercih edilen mimari cikarilir:

```
# Oncelik: arm64 (Apple Silicon) > x86_64
proc = subprocess.run(
    ["lipo", str(binary_path), "-thin", "arm64",
     "-output", str(thin_path)],
    capture_output=True, text=True, timeout=30,
)
if proc.returncode == 0:
    binary_path = thin_path  # arm64 slice ile devam et
```

Listing 4.5: lipo thin ile mimari cikartma

4.9 Dil Tespit Algoritmasi

Binary icindeki string'lerden kaynak programlama dilini tespit etmek, Karadul'un en onemli heuristik adimlarindan biridir. `target.py:L470-L491`'de implement edilen bu algoritma, bir *puanlama sistemi* kullanir.

4.9.1 Puanlama Sistemi

Dil Puanlama

$\mathcal{S} = \{(s_i, l_i)\}_{i=1}^{N}$ imza kumesi verildiginde, her dil $l \in \mathcal{L}$ icin puan:

$$\text{score}(l) = \sum_{i=1}^{N} \mathbb{1}[s_i \in \text{binary_text}] \cdot \mathbb{1}[l_i = l] \tag{4.4}$$

Tespit edilen dil:

$$\hat{l} = \arg\max_{l \in \mathcal{L}} \text{score}(l) \tag{4.5}$$

Eger $\text{score}(\hat{l}) = 0$ ise, $\hat{l} = \text{UNKNOWN}$ dondurulur.

4.9.2 Dil Imza Tablosu

`target.py:L136-L154`'te tanimlanan `_LANGUAGE_SIGNATURES` sozlugu:

String Imzasi	Dil	Aciklama
`rust_begin_unwind`	Rust	Panic handler baslangici
`rust_panic`	Rust	Panic cagirma fonksiyonu
`__rust_alloc`	Rust	Bellek ayirici
`core::panicking`	Rust	Core kutuphane panic modulu
`runtime.gopanic`	Go	Go runtime panic
`runtime.goexit`	Go	Goroutine cikis
`runtime/internal`	Go	Runtime internal paket
`go.buildid`	Go	Build identifier
`_swift_`	Swift	Swift runtime prefix
`Swift.`	Swift	Swift modulleri
`__cxa_throw`	C++	C++ exception throwing
`std::`	C++	C++ standard kutuphane
`objc_msgSend`	Obj-C	Objective-C mesaj gonderme
`@objc`	Obj-C	Objective-C dekoratoru
`NSObject`	Obj-C	Foundation sinifi

Table 4.7: Dil tespit imza tablosu (`_LANGUAGE_SIGNATURES`).

4.9.3 Naive Bayes Perspektifi

Dil tespitini Bayesian bir cercevede formalize edebiliriz. Her imza s_i'nin binary'de bulunmasi, dilin l olmasi olasiligini guncelleyen bir *evidence* olarak gorulur.

$$P(l \mid s_1, s_2, \ldots, s_N) \propto P(l) \prod_{i=1}^{N} P(s_i \mid l) \tag{4.6}$$

Karadul'un mevcut implementasyonunda $P(l)$ uniform prior ve $P(s_i \mid l) \in \{0, 1\}$ (deterministik) olarak basitlestirilmistir. Bu, formul (4.4)'daki toplam sayma ile esdegerdir.

Karmasiklik Analizi. Dosyanin ilk 1 MB'i okunur ($\mathcal{O}(1)$ — sabit boyut), $N = 15$ imza uzerinde dogrusal tarama yapilir. Toplam:

$$T_{\text{lang}} = \mathcal{O}(|1\text{MB}| \times N) = \mathcal{O}(N) \approx \mathcal{O}(1) \tag{4.7}$$

4.10 Go Binary Tespiti

Go dili, derlenmis binary'lere son derece zengin metadata gomer. Bu metadata, tersine muhendislik icin degerlidir ve Karadul bunu iki asamada tespit eder.

4.10.1 GOPCLNTAB (Program Counter Line Table)

Go derleyicisi, her binary'ye bir `pclntab` (program counter line table) ekler. Bu tablo fonksiyon adlarini, dosya yollarini ve satir numaralarini icerir. `go_binary.py:L47-L60`'da tanimlanan magic byte'lar:

Go Versiyonu	GOPCLNTAB Magic
Go 1.2+	`0xFB 0xFF 0xFF 0xFF 0x00 0x00`
Go 1.16+	`0xFA 0xFF 0xFF 0xFF 0x00 0x00`
Go 1.18+	`0xF0 0xFF 0xFF 0xFF 0x00 0x00`
Go 1.20+	`0xF1 0xFF 0xFF 0xFF 0x00 0x00`

Table 4.8: GOPCLNTAB magic byte'lari Go versiyonlarina gore.

`target.py:L493-L516`'deki `_is_go_binary` metodu, ilk 2 MB icinde bu magic byte'lari ve Go runtime string'lerini arar:

```
def _is_go_binary(self, path: Path) -> bool:
    with open(path, "rb") as f:
        data = f.read(2_097_152)  # 2 MB
    # GOPCLNTAB magic bytes
    if b"GOPCLNTAB" in data or b"\xFB\xFF\xFF\xFF\x00\x00" in data:
        return True
```

```
    # go.buildid section
    if b"go.buildid" in data or b"Go build" in data:
        return True
    # Runtime string'leri
    text = data.decode("ascii", errors="replace")
    go_markers = ("runtime.gopanic", "runtime.goexit", "runtime
       /internal")
    return any(marker in text for marker in go_markers)
```

Listing 4.6: Go binary tespit algoritmasi

4.10.2 Go Binary Isim Kurtarma Zenginligi

Go binary'ler, strip edilse bile GOPCLNTAB genellikle korunur. Bu, Go'yu tersine muhendislik icin "en kolay hedef" yapar:

$$\text{Kurtarma Orani}_{\text{Go}} \approx 70\%\text{–}95\% \tag{4.8}$$

Karsilastirma olarak, tipik bir stripped C/C++ binary'de bu oran $< 5\%$'tir.

4.11 .NET ve Delphi Tespiti

4.11.1 .NET Assembly Tespiti

.NET assembly'leri PE formati icinde paketlenmis IL (Intermediate Language) kodudur. Karadul, `target.py:L518-L536`'daki `_is_dotnet` metodu ile tespit eder:

- `mscoree.dll`: .NET CLR runtime yukleyicisi — her .NET PE'de bulunur.
- `_CorExeMain`: .NET entry point fonksiyonu.
- `mscorlib` veya `System.Runtime`: .NET framework referanslari.

Herhangi birinin binary icinde bulunmasi, .NET assembly oldugunu gosterir.

4.11.2 Delphi Binary Tespiti

Delphi compiler'in urettigi binary'lerde kendine ozgu *mangled name* pattern'leri vardir. `target.py:L538-L571`'deki `_is_delphi` metodu iki asamali tespit uygular:

Asama 1 — Bilinen string'ler: "Borland Delphi", "CodeGear Delphi", "Embarcadero Delphi", `@System@TObject@`, `System.SysUtils` string'lerinden en az 2 tanesinin bulunmasi yeterlidir.

Asama 2 — Mangled name regex:

$$\texttt{@[A-Z][...]@[A-Z][...]@[A-Za-z_][...]\$qqr} \tag{4.9}$$

Bu pattern, Delphi'nin `@Unit@Class@Method$qqr` seklindeki mangled fonksiyon isimlerini yakalar. 5 veya daha fazla eslesme Delphi binary oldugunu gosterir.

$$\text{confidence}_{\text{Delphi}} = \begin{cases} 0.95 & \text{string_hits} \geq 2 \\ 0.85 & \text{regex_matches} \geq 5 \\ 0.0 & \text{aksi halde} \end{cases} \tag{4.10}$$

4.12 JavaScript Bundle Tespiti

JavaScript dosyalari uzanti tabanli (`.js`, `.mjs`, `.cjs`) tespit edilir ve ardindan bundler tespiti yapilir.

4.12.1 Bundler Tespit Sistemi

`target.py:L126-L133`'te tanimlanan imza tablosu:

Imza String'i	Bundler
`__webpack_require__`	webpack
`__webpack_modules__`	webpack
`parcelRequire`	Parcel
`System.register`	SystemJS
`_rollupPluginBabelHelpers`	Rollup
`define([`	AMD/RequireJS

Table 4.9: JS bundler imza tablosu.

Tespit algoritmasi:

1. Dosyanin ilk 64 KB'i okunur (`target.py:L288-L291`).
2. Her imza string'i icin dogrusal arama yapilir.
3. Ilk eslesen bundler, metadata'ya kaydedilir.

Karmasiklik. $K = 6$ imza, $M = 65536$ byte okuma:

$$T_{\text{bundler}} = \mathcal{O}(K \times M) = \mathcal{O}(1) \tag{4.11}$$

4.12.2 esbuild Tespiti

esbuild, imza tablosunda yoktur cunku belirgin bir runtime marker'i birakmaz. Bu nedenle, bilinen bundler'lardan hicbiri eslesmediyse ve dosya minified gorunuyorsa, metadata'ya `bundler: null` olarak kaydedilir. Kullanici tarafindan manuel belirtilebilir.

4.12.3 Webpack vs Parcel vs Rollup Farklari

Ozellik	webpack	Parcel	Rollup
Runtime	`__webpack_require`	`parcelRequire`	Yok (tree-shaken)
Modul ID	Sayisal / string	Hash-based	Degisken adi
Unpack kolayligi	Kolay	Orta	Zor

Table 4.10: Bundler runtime farklari ve Karadul'un unpack stratejisi.

4.13 Electron .app Bundle Analizi

Electron uygulamalari, macOS'ta `.app` bundle formunda dagitilir. Karadul, `target.py:L215-L279` ve `electron.py` ile Electron tespiti yapar.

4.13.1 Electron Tespit Algoritmasi

1. `.app/Contents/Resources/app.asar` veya `.app/Contents/Resources/app/` dizini kontrol edilir.
2. Varsa: `ELECTRON_APP` tipi atanir, dil `JAVASCRIPT` olarak ayarlanir.
3. Yoksa: standart macOS uygulamasi olarak siniflandirilir.

4.13.2 ASAR Arsiv Formati

ASAR (Atom Shell Archive), Electron'un kullandigi ozel bir arsiv formatidir. JSON-tabanli bir header ve birlesmis dosya icerikleri icerir:

ASAR extraction (`electron.py:L346-L397`):

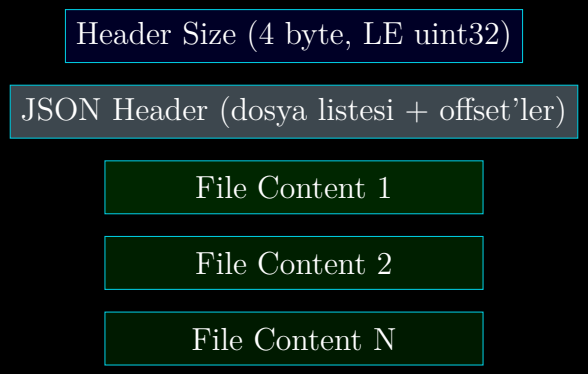


Figure 4.4: ASAR arsiv yapisi.

```
cmd = [
    npx_path, "--yes", "@electron/asar",
    "extract", str(asar_path), str(output_dir),
]
result = self._runner.run_command(cmd, timeout=timeout)
```

Listing 4.7: ASAR extraction komutu

4.13.3 Ana JS Dosyasi Bulma Stratejisi

Extract edilen ASAR icinde onlarca JS dosyasi olabilir. Karadul, ana dosyayi su sirayla arar (`electron.py:L453-L498`):

1. `package.json`'dan `"main"` field'ini oku.
2. Bilinen isim adaylarina bak: `dist/cli.js`, `dist/main.js`, `dist/index.js`, `app/main.js`, ... (12 aday; `electron.py:L32-L46`).
3. Hicbiri yoksa: en buyuk JS dosyasini sec.

4.14 SHA-256 Streaming Hash

Her hedef icin benzersiz bir taniimlayici olarak SHA-256 hash hesaplanir. `target.py:L616-L629`'daki `_compute_sha256` metodu *streaming* yaklasimiyla calisir.

4.14.1 Streaming Hash Algoritmasi

```python
_HASH_CHUNK_SIZE = 8192  # 8 KB

@staticmethod
def _compute_sha256(path: Path) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        while True:
            chunk = f.read(8192)
            if not chunk:
                break
            h.update(chunk)
    return h.hexdigest()
```

Listing 4.8: 8 KB chunk'larla SHA-256 hesaplama

4.14.2 Neden 8 KB Chunk?

- **Bellek verimliligi:** 224 MB binary tamamen RAM'e alinmaz; sabit 8 KB tampon kullanilir.
- **Sayfa hizalanmasi:** Cogu isletim sistemi 4 KB sayfa boyutu kullanir; 8 KB = 2 sayfa, I/O kararliligi saglar.
- **Performans:** Daha kucuk chunk'lar syscall overhead'i arttirir; daha buyuk chunk'lar cache'i kirletir. 8 KB iyi bir denge noktasidir.

Karmasiklik.

$$T_{\text{SHA256}} = \mathcal{O}\left(\frac{n}{8192}\right) = \mathcal{O}(n) \tag{4.12}$$

burada n dosya boyutudur (byte).

4.14.3 Collision Olasiligi

SHA-256, 256-bit cikti ureten bir kriptografik hash fonksiyonudur.

$$P(\text{collision} \,|\, k \text{ dosya}) \approx 1 - e^{-k^2/(2\cdot 2^{256})} \tag{4.13}$$

$k = 10^9$ dosya icin:

$$P \approx \frac{(10^9)^2}{2^{257}} \approx \frac{10^{18}}{2.31 \times 10^{77}} \approx 4.33 \times 10^{-59} \tag{4.14}$$

Bu, pratik amaclar icin sifir kabul edilir. SHA-256'nin ikinci preimage direnci 2^{256} islem gerektirir ki bu evrenin yasi boyunca tum yeryuzu bilgisayarlariyla bile ulasilamaz.

4.15 False Positive Analizi

Magic byte tabanli tespit, false positive riski tasir. Bu bolum, her format icin cakisma olasiliklerini analiz eder.

4.15.1 Magic Byte Collision Hesabi

Rastgele bir dosyanin ilk k byte'inin belirli bir magic byte ile eslesmesi olasiligi:

$$P(\mathrm{FP}_k) = \frac{1}{256^k} \tag{4.15}$$

Format	k (byte)	P(FP)	1/N
Mach-O (4 byte magic)	4	2.33×10^{-10}	4.3 milyar dosyada 1
ELF (4 byte magic)	4	2.33×10^{-10}	4.3 milyar dosyada 1
PE (2 byte "MZ")	2	1.53×10^{-5}	65.536 dosyada 1
ZIP (4 byte magic)	4	2.33×10^{-10}	4.3 milyar dosyada 1

Table 4.11: Magic byte false positive olasiliklari.

PE formati icin 2 byte'lik magic, daha yuksek false positive riski tasir. Ancak Karadul, PE tespitinden sonra ek kontroller (PE signature, section table validity) uygulayarak bu riski azaltir.

4.15.2 Coklu Kontrol Stratejisi

Karadul'un false positive oranini azaltan katmanli yaklasimi:

$$P(\mathrm{FP}_{\mathrm{toplam}}) = P(\mathrm{FP}_{\mathrm{magic}}) \times P(\mathrm{FP}_{\mathrm{icerik}}) \times P(\mathrm{FP}_{\mathrm{uzanti}}) \tag{4.16}$$

Ornegin, PE icin:

$$P(\mathrm{FP}_{\mathrm{PE}}) = \frac{1}{256^2} \times P(\text{valid section table}) \times P(\texttt{.exe}/\texttt{.dll}) \approx 10^{-12} \tag{4.17}$$

Bu Bolumun Ozeti

Hedef Tespit Sistemi, Karadul pipeline'inin temeli olarak 12 farkli binary formatini ve 13 programlama dilini tespit eder. Magic byte karsilastirmasi $\mathcal{O}(1)$ zamanda calisir, dil tespiti puanlama tabanlidir ve $\mathcal{O}(1)$ karmasikliga sahiptir. SHA-256 streaming hash $\mathcal{O}(n)$ ile dosya butunlugunu garanti eder. False positive orani, coklu kontrol stratejisi ile 10^{-10} mertebesinin altinda tutulur. Tum bu islemler `target.py`'daki 630 satirlik `TargetDetector` sinifinda birlestirilmistir.

4.16 Statik Analize Genel Bakis

Statik analiz, bir programi *calistirmadan* — yalnizca yapisinan inceleyerek — bilgi cikarma surecidir. Karadul'un statik analiz katmani iki ana dal icindenir:

1. **Binary Statik Analiz** (`macho.py`): Mach-O, ELF, PE binary'ler icin otool, nm, strings, lief ve Ghidra headless.
2. **JavaScript Statik Analiz** (`javascript.py`): Babel AST ile fonksiyon/string/import/export cikartma.

Statik Analiz Formalizasyonu

Bir hedef $t \in \mathcal{T}$ uzerinde statik analiz fonksiyonu:

$$\mathcal{A}_{\text{static}} : \mathcal{T} \longrightarrow \mathcal{F} \times \mathcal{S} \times \mathcal{I} \times \mathcal{G} \tag{4.18}$$

Burada:

- $\mathcal{F}$: Fonksiyon kumesi (isim, adres, boyut, decompiled kod)
- $\mathcal{S}$: String kumesi (literal, referans adresi)
- $\mathcal{I}$: Import/export kumesi (kutuplhane, sembol)
- $\mathcal{G}$: Call graph (yonlu graf, $G = (V, E)$ burada V fonksiyonlar, E cagri iliskileri)

4.17 Binary Statik Analiz Araclari

`MachOAnalyzer` sinifi (`macho.py:L34-L610`), native binary'ler icin 7 adimlik bir statik analiz pipeline'i isletir.

4.17.1 otool: Dynamic Library ve Load Command Analizi

macOS'a ozgu `otool` araci, Mach-O binary'lerin yapisal bilgilerini cikarir.

otool -L: Dynamic Libraries. `macho.py:L362-L387`'deki `_run_otool_libs` metodu:

```
output = self.runner.run_otool(binary_path, flags=["-L"])
for line in output.splitlines()[1:]:  # ilk satir dosya adi
    parts = line.strip().split(" (")
```

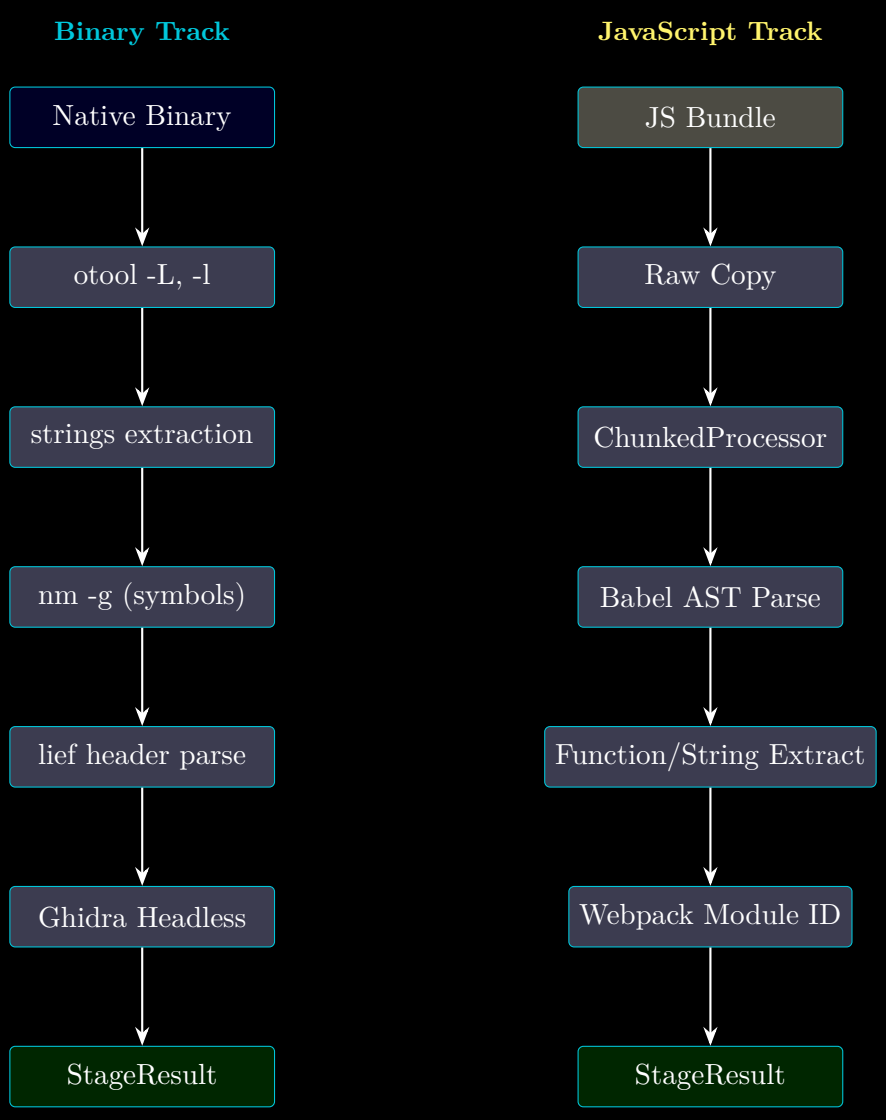


Figure 4.5: Statik analiz iki paralel track: binary ve JavaScript.

```
    lib_path = parts[0].strip()
    version_info = parts[1].rstrip(")") if len(parts) > 1 else ""
    libraries.append({"path": lib_path, "version_info": version_info})
```

Listing 4.9: otool -L cikti parse'i

otool -l: Load Commands. Tum load command'lari ham metin olarak kaydeder (macho.py:L389-L391). Bu cikti, segment offset'leri, UUID, code signature ve encryption bilgilerini icerir.

Karmasiklik. Her iki komut da $\mathcal{O}(n)$ dosya boyutuyla orantili zamanda calisir. Tipik bir 10 MB binary icin < 1 saniye.

4.17.2 nm: Symbol Table Extraction

`nm -g` komutu, binary'nin global (export) sembollerini cikarir. macho.py:L393-L429'daki _run_nm metodu:

```
# nm cikti formati: "ADDRESS TYPE NAME"
# Ornekler:
# 0000000100003a40 T _main
# 0000000100004b20 T _process_data
#                  U _printf        (undefined/external)
for line in result.stdout.splitlines():
    parts = line.strip().split(None, 2)
    if len(parts) == 3:
        symbols.append({
            "address": parts[0],  # hex adres
            "type": parts[1],     # T=text, D=data, U=undefined
            "name": parts[2],     # sembol adi
        })
```

Listing 4.10: nm cikti formati

Symbol Type Kodlari.

Kod	Anlam
`T/t`	Text (kod) section'da tanimli / lokal
`D/d`	Data section'da tanimli / lokal
`B/b`	BSS (sifir-initialize) section / lokal
`S/s`	Diger section'da tanimli / lokal
`U`	Undefined (baska kutuphaneden import)

Table 4.12: nm sembol tip kodlari.

4.17.3 strings: String Extraction

Binary icindeki okunabilir string'ler, tersine muhendisligin en degerli bilgi kaynaklarindan biridir. Karadul iki yontem kullanir:

1. Subprocess yontemi (kucuk binary'ler, <100 MB): Sistemin `strings` komutunu SubprocessRunner ile calistirir.

2. mmap yontemi (buyuk binary'ler, ≥100 MB): `macho.py:L497-L572`'deki `_extract_strings_mmap` metodu, dosyayi `mmap` ile belleye map'leyerek byte-by-byte tarar:

```
with mmap.mmap(f.fileno(), 0, access=mmap.ACCESS_READ) as mm:
    for i in range(mm.size()):
        byte = mm[i]
        if 0x20 <= byte <= 0x7E:  # Printable ASCII
            current.append(byte)
        else:
            if len(current) >= min_length:
                strings.append(bytes(current).decode("ascii"))
                if len(strings) >= max_strings:
                    return strings  # Erken cikis
            current = []
```

Listing 4.11: mmap ile string extraction

mmap Avantajlari.

- OS seviyesinde sayfa sayfa okuma — tum dosya RAM'e alinmaz.
- 224 MB binary icin ∼15 saniye (subprocess ile >30 saniye).
- `max_strings = 50000` limiti ile bellek kullanimi kontrol altinda.

Karmasiklik.

$$T_{\text{strings}} = \mathcal{O}(n) \tag{4.19}$$

Tek gecis (single pass) algoritmasi. Her byte tam bir kez incelenir.

4.18 Ghidra Headless Analiz

Ghidra, NSA tarafindan gelistirilen acik kaynakli bir tersine muhendislik framework'udur. Karadul, Ghidra'yi *headless* modda (GUI olmadan) kullanarak derinlemesine binary analiz yapar.

4.18.1 PyGhidra API vs CLI Modu

`headless.py:L1-L68`'de iki calisma modu desteklenir:

1. **PyGhidra API (tercih edilen):** `pyghidra.open_program` ile Ghidra JVM dogrudan Python 3 process'i icinde baslatilir. Hizli, stabil, ama Ghidra + PyGhidra kurulumu gerektirir.
2. **CLI fallback:** `analyzeHeadless` komutu ile ayri bir JVM process'i baslatilir. Jython gerektirir, daha yavas.

4.18.2 Decompilation Pipeline

Ghidra'nin decompilation sureci P-code (Processor code) ara temsilini kullanir:

P-code. P-code, Ghidra'nin mimariden bagimsiz ara dilidir. Her native instruction (ARM64, x86_64 vb.) bir veya birden fazla P-code islemine donusturulur. Bu, Ghidra'nin tek bir decompiler'la 40+ mimariyi desteklemesini saglar.

4.18.3 Ghidra Script'leri

Karadul, Ghidra'ya 5 ozel script gonderir:

Script	Gorevi
`function_lister.py`	Tum fonksiyonlarin listesi (isim, adres, boyut)
`string_extractor.py`	Cross-reference'li string cikartma
`call_graph.py`	Fonksiyonlar arasi cagri grafinin cikarilmasi
`decompile_all.py`	Her fonksiyonun C pseudocode'a cevirilmesi
`export_results.py`	Tum sonuclarin JSON olarak birlestirilmesi

Table 4.13: Karadul'un Ghidra script'leri.

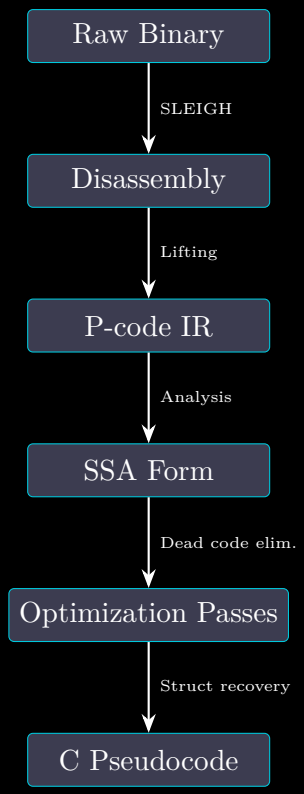


Figure 4.6: Ghidra decompilation pipeline: Binary → P-code → C pseudocode.

4.19 Mach-O Segment/Section Analizi

`lief` kutuphanesi ile Mach-O binary'nin tum segment ve section bilgileri programatik olarak cikarilir (`macho.py:L431-L495`).

4.19.1 ___TEXT Segmenti

`__text`: Derlenmis makine kodu. ARM64 veya x86_64 instruction'lar.

`__stubs`: Lazy binding stub'lari — dinamik kutuplhane fonksiyonlarina ilk cagridaki trampoline.

`__stub_helper`: Stub helper fonksiyonlari — dyld'e geri cagri.

`__cstring`: Null-terminated C string'leri.

`__const`: Salt-okunur sabitler.

`__unwind_info`: Exception handling unwind bilgileri.

4.19.2 __DATA Segmenti

`__data`: Initialize edilmis global degiskenler.

`__bss`: Sifir-initialize edilmis degiskenler (dosyada yer kaplamaz).

`__la_symbol_ptr`: Lazy symbol pointer'lar — ilk cagri sirasinda doldurulur.

`__got`: Global Offset Table — non-lazy symbol pointer'lar.

4.19.3 __LINKEDIT Segmenti

Monolitik bir segment olup sembol tablolari, string tablolari, code signature ve diger baglama bilgilerini icerir. Bu segmentin ic yapisi `LC_SYMTAB` ve `LC_DYSYMTAB` load command'lari ile tarif edilir.

4.20 Symbol Extraction: SYMTAB ve DYSYMTAB

4.20.1 SYMTAB (Symbol Table)

`LC_SYMTAB` load command'i, sembol tablosunun dosya icindeki konumunu belirtir:

$$\text{LC_SYMTAB} = \begin{cases} \texttt{symoff} & \text{sembol tablosu offset'i} \\ \texttt{nsyms} & \text{sembol sayisi} \\ \texttt{stroff} & \text{string tablosu offset'i} \\ \texttt{strsize} & \text{string tablosu boyutu} \end{cases} \tag{4.20}$$

Her sembol girisi (`nlist_64`) 16 byte'tir:

Alan	Tip	Boyut
`n_strx`	uint32	4 byte (string tablosu index'i)
`n_type`	uint8	1 byte (tip flag'leri)
`n_sect`	uint8	1 byte (section numarasi)
`n_desc`	uint16	2 byte (aciklama)
`n_value`	uint64	8 byte (sembol adresi)

Table 4.14: `nlist_64` sembol girisi yapisi.

4.20.2 DYSYMTAB (Dynamic Symbol Table)

Dinamik baglayici (dyld) tarafindan kullanilan ek bilgileri icerir: hangi sembollerin local, hangilesinin external, hangilerinin undefined oldugunu belirten index'ler.

Karadul'un sembol cikartma stratejisi:

1. `nm -g` ile global sembolleri al (`macho.py:L393`).
2. lief ile tum segment/section bilgisini al (`macho.py:L431`).
3. Ghidra ile fonksiyon listesini al (decompilation dahil).
4. Uc kaynagi birlestir: `nm` $\cup$ `lief` $\cup$ `Ghidra`.

4.21 JavaScript Babel AST Analizi

JavaScript statik analizi, `@babel/parser` ile kaynak kodu Abstract Syntax Tree (AST) agacina donusturur ve bu agac uzerinde gezinerek bilgi cikarir.

4.21.1 AST Kavramsal Yapisi

Abstract Syntax Tree

Bir kaynak kod s icin AST, yaprak dugumleri token'lar, ic dugumleri sentaktik yapilar olan bir koklu agactir:

$$\mathrm{AST}(s) = (V, E, r, \lambda) \tag{4.21}$$

Burada:

- V: dugum kumesi
- $E \subseteq V \times V$: ebeveyn-cocuk iliskileri
- $r \in V$: kok dugum (Program)
- $\lambda : V \to \mathrm{NodeType}$: dugum tip etiketi

4.21.2 Babel Parser Entegrasyonu

Karadul, Babel'i dogrudan Python'dan degil, Node.js scriptleri uzerinden cagirir (`javascript.py:L49-L1`

```
cmd = [
    str(self.config.tools.node),
    "--max-old-space-size=8192",
    str(deobfuscate_script),
    str(analysis_file),
]
sub_result = self._runner.run_command(
    cmd, timeout=effective_timeout, cwd=self._scripts_dir,
)
result_json = sub_result.parsed_json
```

Listing 4.12: deobfuscate.mjs cagirma

4.21.3 Cikarilan AST Node Tipleri

Node Tipi	Kategori	Aciklama
`FunctionDeclaration`	Fonksiyon	`function foo() {...}`
`ArrowFunctionExpression`	Fonksiyon	`(x) => x + 1`
`ClassMethod`	Fonksiyon	`class Foo { bar() {...} }`
`FunctionExpression`	Fonksiyon	`const f = function() {...}`
`StringLiteral`	String	`"hello world"`
`TemplateLiteral`	String	`` `Hello ${name}` ``
`ImportDeclaration`	Import	`import x from 'y'`
`ExportNamedDeclaration`	Export	`export { foo }`
`CallExpression`	Cagri	`foo(a, b)`

Table 4.15: Babel AST'den cikarilan node tipleri.

4.21.4 Fonksiyon Extraction Algoritmasi

Babel AST uzerinde `traverse` ile derinlemesine gezinti yapilir. Her fonksiyon node'u icin:

1. **Isim:** `FunctionDeclaration.id.name`, `ClassMethod.key.name`, veya anonim fonksiyonlar icin `null`.
2. **Parametre sayisi:** `node.params.length`.
3. **Baslangic/bitis satir numaralari:** `node.loc.start.line`, `node.loc.end.line`.
4. **Govde boyutu:** Bitis - baslangic satir farki.

Pseudocode:

```
traverse(ast, {
  "FunctionDeclaration|ArrowFunctionExpression|ClassMethod"(
     path) {
    functions.push({
      name: path.node.id?.name || path.node.key?.name || null,
      params: path.node.params.length,
      loc: path.node.loc,
      body_size: path.node.loc.end.line - path.node.loc.start.
         line,
    });
  }
});
```

Listing 4.13: Fonksiyon extraction pseudocode

4.21.5 String Extraction

AST'den iki tipteki string cikarilir:

1. StringLiteral: Tek veya cift tirnak icindeki sabit string'ler. Degeri dogrudan `node.value`'dan alinir.

2. TemplateLiteral: Backtick icindeki sablonlu string'ler. `node.quasis` dizisindeki `cooked` degerleri birlestirilir.

Karmasiklik. AST traversal $\mathcal{O}(|V|)$ — dugum sayisiyla dogrusal.

4.22 Webpack Modul ID Tespiti

Webpack bundle'lari, her modulu bir ID ile tanimlar ve `__webpack_require__(id)` ile yukler.

4.22.1 Webpack Runtime Yapisi

Tipik bir webpack bundle'in yapisi:

```
(function(modules) {
  // webpack bootstrap
  function __webpack_require__(moduleId) {
    // cache kontrolu
```

```
5      if (installedModules[moduleId]) return installedModules[
          moduleId].exports;
6      // yeni modul
7      var module = installedModules[moduleId] = { exports: {} };
8      modules[moduleId].call(module.exports, module, module.
          exports,
9                                 __webpack_require__);
10     return module.exports;
11   }
12   __webpack_require__(0); // entry point
13 })({
14   0: function(module, exports, __webpack_require__) { /* ... */
        },
15   1: function(module, exports, __webpack_require__) { /* ... */
        },
16   // ...
17 });
```

Listing 4.14: Webpack bundle yapisi (basitlestirilmis)

4.22.2 Modul ID Cikartma Algoritmasi

Babel AST uzerinde `CallExpression` node'lari taranir:

1. `callee.name === "__webpack_require__"` mi kontrol edilir.
2. Argument olarak sayi veya string literal alinir — bu modul ID'dir.
3. ID'ler toplanir ve `webpack-modules` artifact'i olarak kaydedilir.

`javascript.py:L213-L220`'de kayit:

```
1 webpack_modules = result_json.get("webpack_modules", [])
2 if webpack_modules:
3     wp_path = workspace.save_json("static", "webpack-modules",
         {
4         "module_ids": webpack_modules,
5         "count": len(webpack_modules),
6     })
```

Listing 4.15: Webpack modul kaydi

4.22.3 Modul ID Mapping

Webpack 4+ surumlerde modul ID'leri deterministik hash'lerdir:

$$\text{module_id} = \text{MD4}(\text{relative_path})[:4] \quad \text{(webpack 4)} \tag{4.22}$$

$$\text{module_id} = \text{MD4}(\text{absolute_path} + \text{loaders})[:4] \quad \text{(webpack 5)} \tag{4.23}$$

Bu hash'ler, orijinal dosya yolunu geri kazanmayi olanaklsiz kilar; ancak modul sayisi ve bagimlilik yapisi analizlerinde kullanilir.

4.23 Buyuk Dosya Destegi: ChunkedProcessor

200 MB+ JavaScript dosyalari Babel parser'in bellek limitlerini asar. `chunked_processor.py`'deki `ChunkedProcessor` sinifi bu problemi cozer.

4.23.1 Chunk Bolme Algoritmasi

Brace-Aware Chunking

Bir JS dosyasi $F = (l_1, l_2, \ldots, l_n)$ satirlarindan olusmaktadir. Brace derinligi:

$$d(i) = d(i-1) + \#\{l_i\}^{\{} - \#\{l_i\}^{\}} \tag{4.24}$$

Bir chunk siniri i'de olusur eger:

$$d(i) = 0 \quad \wedge \quad \sum_{j=k}^{i} |l_j| \geq C_{\max} \tag{4.25}$$

Burada $C_{\max} =$ `max_chunk_mb` $\times 1024^2$ (varsayilan 50 MB).

`chunked_processor.py:L113-L223`'teki implementasyon:

```
depth = 0
for line_num, line in enumerate(f, start=1):
    for ch in line:
        if ch == "{": depth += 1
        elif ch == "}":
            depth -= 1
            if depth < 0: depth = 0
    current_lines.append(line)
    current_size += len(line.encode("utf-8"))
```

```
    # Chunk boundary: depth=0 ve yeterince buyuk
    if depth == 0 and current_size >= max_chunk_bytes:
        write_chunk(current_lines)
        current_lines, current_size = [], 0
```

Listing 4.16: Brace-aware chunking

4.23.2 Top-Level Statement Tespiti

Chunk sinirlari, tercihen top-level statement baslangiclarina hizalanir. `chunked_processor.py:L22-L36`' pattern'ler:

```
_TOP_LEVEL_STARTS = (
    "function ", "async function ", "class ",
    "const ", "let ", "var ",
    "export ", "module.exports", "exports.",
    "Object.defineProperty", "__webpack_require__",
    '"use strict"', "'use strict'",
)
```

Listing 4.17: Top-level statement pattern'leri

4.23.3 Paralel Isleme

`ChunkedProcessor.process_chunks` metodu (`chunked_processor.py:L225-L258`) hem sirali hem paralel isleme destekler:

$$S_p = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} \leq \min(p, N_{\text{chunks}}) \tag{4.26}$$

Burada $p =$ `max_workers` (varsayilan 4), N_{chunks} toplam chunk sayisi. Python GIL nedeniyle I/O-bound islemlerde etkilidir, CPU-bound islemlerde sinirlidir.

Sinif	Icerik
`ChunkInfo`	path, index, start_line, end_line, size_bytes, line_count
`ChunkResult`	chunk, success, data, error
`SplitResult`	success, chunks[], total_lines, errors[]

Table 4.16: ChunkedProcessor veri yapilari.

Veri Yapilari.

4.24 FLIRT Signature Sistemi

FLIRT (Fast Library Identification and Recognition Technology), IDA Pro'nun kutuphane fonksiyon tanima sistemidir. Karadul, `flirt_parser.py`'de kendi FLIRT-uyumlu parser'ini implement eder.

4.24.1 .pat Dosya Formati

FLIRT pattern dosyasi (`.pat`) su formatta satirlar icerir:

```
HEXBYTES CRC16 SIZE TOTAL :OFFSET NAME [REFERENCED...]
```

Ornek:

```
558BEC83EC10 0C 0025 003A :0000 _my_function
558BEC........8B4508 00 0000 001F :0000 _another_func ^0010 _ref
```

Burada `..` wildcard byte'i temsil eder (herhangi bir deger eslesir).

4.24.2 Pattern Matching Algoritmasi

`flirt_parser.py:L721-L779`'deki `match_function_bytes` metodu:

Masked Byte Comparison

Bir fonksiyonun ilk n byte'i $f = (f_0, f_1, \ldots, f_{n-1})$ ve bir FLIRT pattern'i (p, m) (pattern + mask) verildiginde:

$$\text{match}(f, p, m) = \bigwedge_{i=0}^{|p|-1} \begin{cases} f_i = p_i & \text{eger } m_i = \texttt{0xFF} \\ \text{true} & \text{eger } m_i = \texttt{0x00} \text{ (wildcard)} \end{cases} \tag{4.27}$$

Confidence Hesabi.

$$\text{conf}(p, m) = \min\left(0.98,\ \text{base_conf} \times \frac{|\{i : m_i = \texttt{0xFF}\}|}{|p|}\right) \tag{4.28}$$

Burada `base_conf` $= 0.95$ (FLIRT imzalarinin yuksek guvenilirligi).

4.24.3 Hex-to-Bytes-with-Mask Donusumu

`flirt_parser.py:L218-L256`'deki `_hex_to_bytes_with_mask` metodu:

```
# "558BEC..8B45" -> (pattern_bytes, mask_bytes)
i = 0
while i < len(hex_str):
    if hex_str[i:i+2] == "..":
        pattern_bytes.append(0x00)
        mask_bytes.append(0x00)    # Wildcard
    else:
        pattern_bytes.append(int(hex_str[i:i+2], 16))
        mask_bytes.append(0xFF)    # Kesin
    i += 2
```

Listing 4.18: Hex string ile mask olusturma

4.24.4 SignatureDB Entegrasyonu

FLIRT parser, bagimsiz bir modul olarak tasarlanmistir. SignatureDB'ye enjeksiyon `inject_into_signature_db` metodu ile yapilir (`flirt_parser.py:L647-L715`):

$$\text{SignatureDB} = \text{SignatureDB}_{\text{builtin}} \cup \text{FLIRT}_{\text{external}} \tag{4.29}$$

Duplikasyon kontrolu yapilir: builtin DB'deki imzalar onceliklidir, ayni isimli FLIRT imzasi atlanir.

Karadul'un toplam imza sayisi:

$$|\text{SignatureDB}| = 6100 + \text{ (builtin)} + \text{FLIRT external} \approx 1.5\text{M}+ \tag{4.30}$$

4.25 Assembly Instruction Analizi

`assembly_analyzer.py`'deki `AssemblyAnalyzer` sinifi, Ghidra decompiler'in kacirdigi dusuk seviye bilgileri dogrudan assembly instruction'lardan cikarir.

4.25.1 ARM64 (AArch64) vs x86_64

Ozellik	ARM64 (AArch64)	x86_64
Instruction boyutu	Sabit 4 byte	Degisken 1–15 byte
Register sayisi	31 genel amacli	16 genel amacli
Calling convention	AAPCS64	System V AMD64 / Win64
Parametre register	x0–x7 (8 adet)	rdi,rsi,rdx,rcx,r8,r9 (6 adet)
Return register	x0, x1	rax, rdx
SIMD extension	NEON	SSE/AVX
Endianness	Little-endian	Little-endian

Table 4.17: ARM64 ve x86_64 mimari karsilastirmasi.

4.25.2 Calling Convention Tespiti

`assembly_analyzer.py:L207-L224`'deki `_detect_calling_convention`:

```
if arch == "aarch64":
    return "aapcs64"

first_lines = "\n".join(lines[:20]).lower()
# System V AMD64: rdi, rsi parametreler
if re.search(r"\brdi\b", first_lines) and re.search(r"\brsi\b",
    first_lines):
    return "sysv_amd64"
# Windows x64: rcx, rdx parametreler
if re.search(r"\brcx\b", first_lines) and not re.search(r"\brdi
   \b", first_lines):
    return "win64"
return "sysv_amd64"  # macOS/Linux default
```

Listing 4.19: Calling convention tespiti

4.25.3 Stack Frame Analizi

Fonksiyonun stack frame boyutu, prolog'daki `sub rsp, N` instruction'indan cikarilir:

$$\text{frame_size} = \begin{cases} N & \text{eger } \texttt{sub rsp, N} \text{ bulunduysa} \\ 0 & \text{aksi halde (leaf function)} \end{cases} \tag{4.31}$$

Stack degiskenleri `[rbp-0xNN]` pattern'leri ile tespit edilir. Ardisik offset farklari degisken boyutunu verir:

$$\text{size}(v_i) = \text{offset}(v_{i+1}) - \text{offset}(v_i) \tag{4.32}$$

4.25.4 SIMD Pattern Tespiti

`assembly_analyzer.py:L82-L110`'da tanimlanan instruction setleri:

SSE: `addps`, `mulps`, `movaps`, ... — 128-bit SIMD (float32×4)

AVX: `vaddps`, `vmulps`, `vfmadd231ps`, ... — 256-bit SIMD (float32×8)

NEON: `fmla`, `fadd`, `ld1`, `st1`, ... — ARM 128-bit SIMD

Crypto: `aesenc`, `sha256rnds2`, `pclmulqdq`, `aese`, ... — AES-NI / ARM Crypto Extensions

FMA (Fused Multiply-Add) instruction'larinin varligii, dot product veya matris carpiimi pattern'ini isaret eder.

4.26 Entropy Analizi ile Packing Tespiti

Paketlenmis (packed) binary'ler, normal binary'lerden ayirt edilebilir entropy profilleri gosterir.

4.26.1 Shannon Entropisi

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x) \tag{4.33}$$

Burada $p(x) = \text{freq}(x)/n$, byte x'in frekans olasiligidir. H degerleri:

- $H = 0$: Tamamen tekduze (tek byte tekrari).
- $H \approx 4$–5: Tipik executable binary (kod + veri).
- $H \approx 6$–7: Sikistirilmis veri veya rastgele-benzeri bolgeler.
- $H \approx 8$: Tamamen rastgele / sifrelenmis / UPX-packed.

`packed_binary.py:L137-L160`'daki implementasyon:

```
1 def calculate_entropy(data: bytes) -> float:
2     if not data:
3         return 0.0
4     freq = [0] * 256
5     for byte in data:
```

```
        freq[byte] += 1
    length = len(data)
    entropy = 0.0
    for f in freq:
        if f > 0:
            p = f / length
            entropy -= p * math.log2(p)
    return entropy
```

Listing 4.20: Shannon entropy hesaplama

Karmasiklik.

$$T_{\text{entropy}} = \mathcal{O}(n + 256) = \mathcal{O}(n) \tag{4.34}$$

Tek gecis frekans sayma + 256 eleman toplam.

4.26.2 Packing Karar Algoritmasi

`packed_binary.py:L209-L388`'deki `PackingDetector.detect` metodu 5 adimli bir tespit uygular:

1. **UPX kontrolu:** `UPX!` magic byte'i aranir. Bulunursa: confidence = 0.95.
2. **PyInstaller kontrolu:** `MEI` magic byte'i (8 byte). Bulunursa: confidence = 0.95.
3. **Nuitka kontrolu:** `__nuitka_`, `nuitka-compiled` string'leri. Bulunursa: confidence = 0.85.
4. **Entropy analizi:** $H > 7.0$ esigi ve $>50\%$ packed section orani.
5. **Import heuristigi:** <10 import $\implies$ muhtemelen packed.

Karar formulu:

$$\text{is_packed} = \begin{cases} \text{true, UPX} & \texttt{UPX!} \in \text{binary} \\ \text{true, PyInstaller} & \texttt{MEI} \in \text{binary} \\ \text{true, Nuitka} & \texttt{nuitka} \in \text{binary} \\ \text{true, unknown} & H > 7.0 \wedge \text{packed_ratio} > 0.5 \\ \text{true, generic} & H > 6.5 \wedge \text{imports} < 10 \\ \text{false} & \text{aksi halde} \end{cases} \tag{4.35}$$

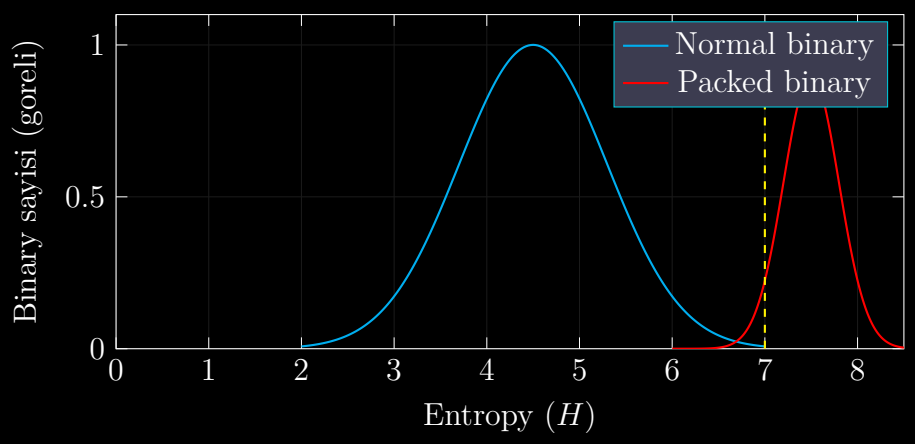


Figure 4.7: Normal vs packed binary entropy dagilimi. $H = 7.0$ esigi optimal ayirim noktasidir.

4.27 Her Analyzer Icin Giris/Cikis Veri Yapilari

Tum analyzer'lar ortak bir arayuz kullanir: `BaseAnalyzer` soyut sinifinden miras alir ve `StageResult` dondurur.

4.27.1 Giris Veri Yapisi: TargetInfo

```
@dataclass
class TargetInfo:
    path: Path                # Dosya yolu
    name: str                 # Hedef adi
    target_type: TargetType # 12 tip
    language: Language         # 13 dil
    file_size: int            # Byte
    file_hash: str            # SHA-256
    metadata: dict[str, Any]# Ek bilgi
```

Listing 4.21: TargetInfo veri yapisi

4.27.2 Cikis Veri Yapisi: StageResult

```
@dataclass
class StageResult:
    stage_name: str                    # "static", "deobfuscate",
        ...
    success: bool                      # Basarili mi
    duration_seconds: float            # Sure
```

```
    artifacts: dict[str, Path]        # Cikti dosyalari
    stats: dict[str, object]          # Istatistikler
    errors: list[str]                 # Hata mesajlari
```

Listing 4.22: StageResult veri yapisi

4.27.3 Analyzer Giris/Cikis Ozet Tablosu

Analyzer	Giris	Cikis Artifact'leri
MachOAnalyzer	Mach-O/ELF/PE binary + Workspace	dynamic_libraries, load_commands, strings_raw, symbols, lief_analysis, ghidra_*
JavaScriptAnalyzer	JS dosya + Workspace	ast_analysis, functions, strings, imports_exports, webpack_modules
ElectronAnalyzer	.app bundle + Workspace	asar_extracted_dir, js_files, electron-analysis.json
GoBinaryAnalyzer	Go binary + Workspace	go_functions, go_buildinfo, go_types
PackingDetector	Binary path	PackingInfo (is_packed, type, entropy, evidence)
FLIRTParser	.pat/.json dosya	FLIRTSignature listesi (name, library, byte_pattern, mask)
AssemblyAnalyzer	Assembly text + arch	AssemblyAnalysisResult (convention, params, stack, simd)

Table 4.18: Her analyzer'in giris ve cikis veri yapilari.

4.27.4 Graceful Degradation

Karadul'un statik analiz katmaninin kritik tasarim prensibi: herhangi bir alt-aracin basarisizligi pipeline'i durdurmaz. `macho.py:L284-L291`'deki basari kriterr:

```
return StageResult(
    stage_name="static",
    success=len(errors) == 0 or len(artifacts) > 0,  # En az 1 artifact yeterli
    # ...
)
```

Listing 4.23: Kismi basari kriteri

Bu, "ya hep ya hic" yerine "ne kadar bilgi cikarabilirsen o kadar iyi" yaklasimini benimser.

4.28 Binary Intelligence: String Clustering ve Mimari Cikarim

`macho.py:L222-L280`'de cagirilan `BinaryIntelligence` modulu, cikarilan string, sembol ve dylib verilerini birlestirerek ust seviye mimarik bilgi cikarir:

- **App type tespiti:** CLI, GUI, daemon, library, framework.
- **Subsystem tespiti:** networking, database, crypto, UI, logging.
- **Algoritma tespiti:** AES, SHA, zlib, protobuf, JSON.
- **Guvenlik mekanizmasi tespiti:** certificate pinning, code signing, anti-debug.
- **Protokol tespiti:** HTTP/2, gRPC, WebSocket, MQTT.

Bu bilgiler, ileriki asamalarda fonksiyon isimlendirme ve rapor uretiminde kullanilir.

Bu Bolumun Ozeti

Statik Analiz katmani, Karadul'un bilgi toplama omurgasidir. Binary track'te otool, nm, strings, lief ve Ghidra Headless 7 adimli bir analiz pipeline'i olusturur. JavaScript track'te Babel AST analizi fonksiyon, string, import/export ve webpack modul bilgilerini cikarir. Buyuk dosyalar (200 MB+) icin ChunkedProcessor brace-aware bolme stratejisi kullanir. FLIRT signature sistemi, 1.5 M+ imza ile kutuphane fonksiyon tanimlamasi yapar. Assembly analiz, calling convention, stack frame ve SIMD pattern tespiti ile derin bilgi saglar. Entropy analizi, packed binary tespitini $H > 7.0$ esigi ile yapar. Tum bu bilesenler, graceful degradation prensibiyle calisir: herhangi bir alt aracin basarisizligi pipeline'i durdurmaz.

Chapter 5

JavaScript Deobfuscation

Bu bolum, Karadul'un en yenilikci bilesenlerinden birini — 9 fazli derin JavaScript deobfuscation pipeline'ini — detayli olarak aciklar.

5.1 Obfuscation Nedir?

JavaScript obfuscation, kaynak kodu okunaksiz hale getirme suerecidir:

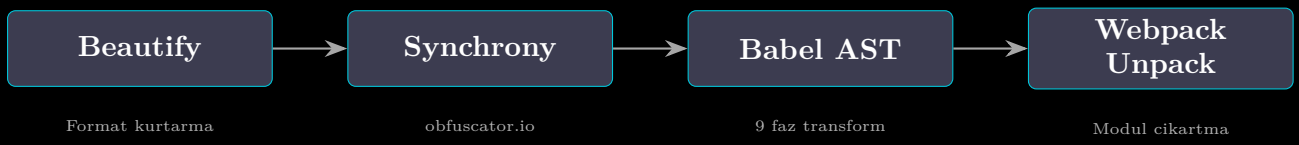
$$\text{obfuscate} : S_{\text{readable}} \rightarrow S_{\text{obfuscated}} \quad \text{s.t.} \quad \text{semantics}(S_{\text{readable}}) = \text{semantics}(S_{\text{obfuscated}}) \tag{5.1}$$

Yaygin teknikler:

1. **Minification:** Bosluk, yorum silme, kisa isimler (`a`, `b`, `c`)
2. **String encoding:** `"Hello"` → `"\x48\x65\x6c\x6c\x6f"`
3. **Control flow flattening:** if/else → switch-case state machine
4. **Opaque predicates:** Her zaman true/false olan kosullar ekleme
5. **Dead code injection:** Calismayan sahte kod ekleme
6. **String array rotation:** String'leri bir diziye koy, indeksle eris, diziyi doenduer

5.2 Standart 4 Adimli Zincir

`manager.py` dosyasindaki standart deobfuscation zinciri:



5.2.1 Graceful Degradation

Her adim basarisiz olabilir — bu durumda bir oenceki basarili cikti kullanilir:

$$\text{output}(i) = \begin{cases} \text{step}_i(\text{output}(i-1)) & \text{eger basarili} \\ \text{output}(i-1) & \text{eger basarisiz} \end{cases} \tag{5.2}$$

5.3 9 Fazli Deep Deobfuscation

5.3.1 Faz 1: Constant Folding (Sabit Katlama)

Derleme zamaninda hesaplanabilir ifadeleri basitlestirme:

$$\text{fold}(e_1 \oplus e_2) = \begin{cases} v_1 \oplus v_2 & \text{eger } e_1 = v_1, e_2 = v_2 \text{ (her ikisi sabit)} \\ e_1 \oplus e_2 & \text{aksi halde (degismez)} \end{cases} \tag{5.3}$$

Ornek 5.1

```
// Once:
var x = 1 + 2 * 3;
var y = "He" + "llo";
// Sonra:
var x = 7;
var y = "Hello";
```

5.3.2 Faz 2: Dead Code Elimination

Kullanilmayan kodu tespit ve silme. Liveness analizi:

$$\text{live_out}(B) = \bigcup_{S \in \text{succ}(B)} \text{live_in}(S) \tag{5.4}$$

$$\text{live_in}(B) = \text{use}(B) \cup (\text{live_out}(B) \setminus \text{def}(B)) \tag{5.5}$$

Eger bir degisken hicbir noktada “live” degilse, atanmasi silinebilir.

5.3.3 Faz 3: String Unhexing

Hex-encoded string'leri orijinaline doenduerme:

$$\text{unhex}(\texttt{"\textbackslash xHH"}) = \text{char}(\text{parseInt}(\texttt{HH}, 16)) \tag{5.6}$$

Ornek 5.2. `\x48\x65\x6c\x6c\x6f` → `"Hello"`

5.3.4 Faz 4: Computed-to-Static Property

$$\text{obj}[\texttt{"foo"}] \rightarrow \text{obj.}\texttt{foo} \quad \text{(eger key string literal ise)} \tag{5.7}$$

AST duenusemuue: `MemberExpression { computed: true }` → `MemberExpression { computed: false }`.

5.3.5 Faz 5–8: Ileri Basitlestirmeler

- **Faz 5:** Comma-split: `a=1, b=2, c=3` → uec ayri ifade
- **Faz 6:** Conditional simplification: `true ? a : b` → `a`
- **Faz 7:** Expression flattening: icice ifadeleri duzlestir
- **Faz 8:** Literal restoration: obfuscated literal'leri orijinaline doenduer

5.3.6 Faz 9: Semantik Degisken Isimlendirme

LLM kullanmadan, 25 context-aware heuristik kural ile anlamli isimler uereetme:

Kural	Oernek
Event listener callback	`addEventListener("click", a)` → `clickHandler`
Error constructor	`new Error(a)` → `errorMessage`
Fetch URL	`fetch(a)` → `fetchUrl`
Array index	`for(a=0; a<b.length)` → `index`
DOM element	`document.getElementById(a)` → `elementId`

Table 5.1: Semantik isimlendirme oernek kurallari.

Bu Bolumun Ozeti

JavaScript deobfuscation, 4 adimli standart zincir (beautify → synchrony → babel → webpack) ve 9 fazli derin transform pipeline'indan olusur. Her faz AST uezerinde

calisir ve belirli bir obfuscation teknigini geri alir. Graceful degradation her adimda basarisizliga tolerans saglar. Faz 9'daki LLM-siz semantik isimlendirme 25 heuristik kuralla etkileyici sonuclar ueretir.

5.4 Abstract Syntax Tree (AST) — Derinlemesine

5.4.1 Formal Tanim

AST, kaynak kodun soezdizimsel yapisini soyutlastiran bir yoenlue, etiketli agactir:

Tanim 5.1 (Abstract Syntax Tree)**.** $T = (V, E, r, L)$ burada:

- V: dugum (node) kuemesi
- $E \subseteq V \times V$: kenar kuemesi (parent $\rightarrow$ child)
- $r \in V$: koek dugum (root)
- $L : V \rightarrow \text{NodeTypes}$: etiketleme fonksiyonu

Parse tree vs AST farki: Parse tree (Concrete Syntax Tree — CST) gramerin her intermediate kuralini goesterir. AST sadece semantik olarak anlamli oegeleri tutar.

Ornek 5.3. `1 + 2 * 3` icin CST:

```
ExpressionStatement
  Expression
    AdditiveExpression
      NumericLiteral("1") "+" MultiplicativeExpression
        NumericLiteral("2") "*" NumericLiteral("3")
```

AST (Babel):

```
BinaryExpression { operator: "+",
  left: NumericLiteral(1),
  right: BinaryExpression { operator: "*",
    left: NumericLiteral(2),
    right: NumericLiteral(3)
  }
}
```

5.4.2 Babel vs Acorn (ESTree)

Karadul her iki parser'i da kullanir. Kritik farklar:

Kavram	Acorn (ESTree)	Babel AST
String literal	`Literal { value: "hello" }`	`StringLiteral { value: "hello" }`
Sayi literal	`Literal { value: 42 }`	`NumericLiteral { value: 42 }`
Object key	`Property`	`ObjectProperty`
Program	`Program { body: [...] }`	`File { program: { body } }`

Table 5.2: Babel vs Acorn AST node tipleri.

`deobfuscate.mjs:164`'te bu ayrim handle ediliyor:

```
const usedAcorn = ast.body !== undefined && !ast.program;
```

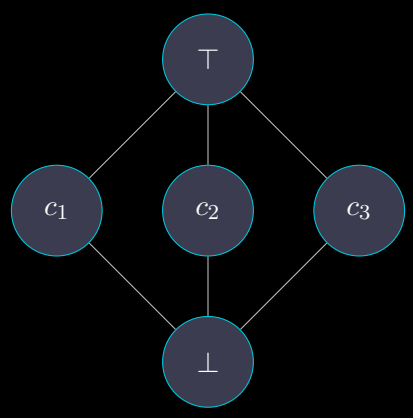
5.4.3 Hesaplama Karmasikligi

Parse etme: $\mathcal{O}(n)$ burada n = token sayisi. Babel'in parser'i Pratt parsing (top-down operator precedence) kullanir. AST node sayisi $\sim \mathcal{O}(n)$ cueuenkue her token en fazla sabit sayida node ueretir.

5.5 Constant Folding — Lattice Teorisi

5.5.1 Abstract Interpretation Framework

Abstract interpretation'da her degisken degeri bir lattice uezerinde izlenir:



- $\top$ (top): Degiskenin degeri henueuez belli degil
- c_i (constant): Tam olarak bilinen sabit deger

- $\perp$ (bottom): Birden fazla farkli deger atanmis, sabit degil

Transfer fonksiyonu f: `x = 3 + 4` ise $f(x) = 7$ (constant). `x = a + b` ve $a = 3$, $b = \top$ ise $f(x) = \top$.

5.5.2 Karadul Implementasyonu

`deep-deobfuscate.mjs` Phase 2'de **exit visitor** (bottom-up) kullanilir:

```
// BinaryExpression.exit:
if (t.isNumericLiteral(left) && t.isNumericLiteral(right)) {
    let result;
    switch (operator) {
        case "+": result = left.value + right.value; break;
        case "-": result = left.value - right.value; break;
        case "*": result = left.value * right.value; break;
        // ... 11 operator toplam
    }
    if (result !== undefined && Number.isFinite(result)) {
        path.replaceWith(t.numericLiteral(result));
    }
}
```

Neden exit visitor? Bottom-up calisir: ic node oence, parent sonra. `1 + 2 * 3`'te oence `*` degerlendirilir (leaf'lere daha yakin), sonra `+`.

Ornek 5.4 (UnaryExpression Katlama)

```
// !0 -> true, !1 -> false (Phase 2)
if (operator === "!" && t.isNumericLiteral(argument)) {
    path.replaceWith(t.booleanLiteral(!argument.value));
}
// void 0 -> undefined
if (operator === "void" && argument.value === 0) {
    path.replaceWith(t.identifier("undefined"));
}
```

5.5.3 Sinirlar

Karadul'un constant folder'i sadece **literal-to-literal** katlar:

```
const A = 10;
const B = A + 5;  // Bu KATLANMAZ: A Identifier, NumericLiteral
    degil
```

Interprocedural constant propagation icin Kildall'in worklist algoritmasi gerekir — $\mathcal{O}(N \cdot D)$ burada D = lattice yueksekligi. Deobfuscation baglaminda literal-to-literal cogunlukla yeterli.

5.6 Dead Code Elimination — Liveness Analizi

5.6.1 Dataflow Denklemleri

CFG uezerinde backward analysis:

$$\text{LIVE_out}[B] = \bigcup_{S \in \text{succ}(B)} \text{LIVE_in}[S] \tag{5.8}$$

$$\text{LIVE_in}[B] = \text{use}(B) \cup \Big(\text{LIVE_out}[B] \setminus \text{def}(B)\Big) \tag{5.9}$$

Fixed-point iteration: Tuem bloklari LIVE_in ve LIVE_out degismeyene kadar itere et. Karmasiklik: $\mathcal{O}(N \cdot D)$ burada N = basic block sayisi, D = iterasyon sayisi (pratikte 2–5).

5.6.2 Karadul'un Basitlestirilmis Yaklasimi

Tam liveness analysis yerine **constant-test elimination**:

```
// Phase 3: IfStatement.exit
if (t.isBooleanLiteral(test) && test.value === true) {
    path.replaceWith(path.node.consequent);  // if(true){X} -> X
}
if (t.isBooleanLiteral(test) && test.value === false) {
    if (path.node.alternate) {
        path.replaceWith(path.node.alternate);  // if(false){X} else{Y} -> Y
    } else {
        path.remove();  // if(false){X} -> (silindi)
    }
}
```

Phase 2 + Phase 3 sinerjisi: Phase 2 `!0`'yi `true`'ya cevirir, Phase 3 `if(true)` goerueuep temizler. Exit visitor sayesinde tek AST traverse'da her iki faz da calisir.

5.7 String Obfuscation Teknikleri ve Coezeuemleri

5.7.1 6 Yaygin Teknik

1. **Hex Encoding:** `"\x48\x65\x6c\x6c\x6f"` → `"Hello"`

   Reverse: `/\\x([0-9a-fA-F]{2})/g` ile decode.

2. **Unicode Escape:** `"\u0048\u0065..."` → `"Hello"`

3. **CharCode Array:** `String.fromCharCode(72, 101, 108, 108, 111)` → `"Hello"`

   AST'de CallExpression ile `String.fromCharCode` tespiti, arguments hepsi NumericLiteral ise derleme zamaninda hesapla.

4. **Base64:** `atob("SGVsbG8=")` → `"Hello"`

5. **String Array Rotation (obfuscator.io):**

```
var _0x1234 = ["log", "Hello", "console"];
(function(_0xa, _0xb) {
    while (--_0xb) { _0xa.push(_0xa.shift()); }
})(_0x1234, 0x2);
// Simdi _0x1234[0]="console", [1]="log", [2]="Hello"
```

   Reverse: Rotation miktarini tespit et, arrayi rotate et, referanslari degistir.

6. **XOR Cipher:** Key bilgisi gerekir. Statik analiz ile key literal ise coezeueuelebilir.

5.7.2 Statik Analiz vs Runtime Emulation

Yaklasim	Avantaj	Dezavantaj
AST-based statik	Gueuevenli, eval yok	Dinamik key'leri coezemez
eval() emulation	Her seyi cozer	Kod enjeksiyon riski
Sandboxed VM	Gueuevenli runtime	Performans maliyeti

Karadul dogru tercihi yapmis: synchrony AST-based statik analiz kullaniyor, eval **kullanmiyor**.

5.8 Computed-to-Static — Soundness Analizi

Doenueuseueuem: `obj["foo"]` → `obj.foo`

Gueuevenli Doenueuseueuem Kosullari

1. Property key bir StringLiteral olmali (degisken/expression degil)
2. Key degeri gecerli JavaScript identifier olmali: `/^[a-zA-Z_$][a-zA-Z0-9_$]*$/`
3. Key degeri reserved word **olmamali**

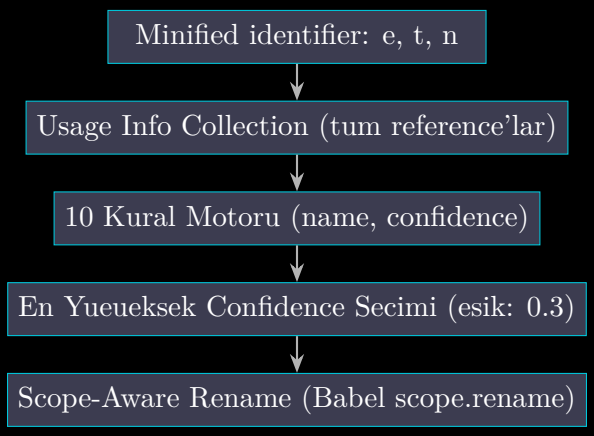
Gueuevenli olmayan durumlar:

- `obj["class"]` — reserved word
- `obj["foo-bar"]` — tire gecerli identifier degil
- `obj[variable]` — StringLiteral degil
- `obj[fn()]` — runtime degeri

Karadul **soundness'i completeness'a tercih eder** — yanlis doenueuseueuem kodun anlamini degistirir ve geri alinmasi cok zor.

5.9 Semantik Degisken Isimlendirme — 25 Kural Motoru

5.9.1 Mimari



5.9.2 Kural Detaylari

Kural 1 — API Call Context: ~80 method 10 kategoriye ayrilmis:

Kural 3 — Property Access Patterns (en yueueksek precision):

Kategori	Method Oernekleri	Sonuc	Conf.
String (24)	split, trim, replace, match	`str` veya `text`	0.85
Array (25)	push, pop, map, filter, reduce	`items`	0.85
Map/Set	get+set+has	`cache`	0.80
Promise	then+catch	`promise`	0.80
EventEmitter	on+emit	`emitter`	0.80
Stream	pipe+write	`stream`	0.80
Buffer	readUInt8, writeUInt8	`buffer`	0.85
RegExp	test+exec	`regex`	0.85
Date	getTime, toISOString	`date`	0.80

Table 5.3: API Call Context kural detaylari.

Property Pattern	Sonuc	Conf.
`.message` + `.stack`	`error` (Error object)	0.90
`.body` + `.params` + `.query`	`request` (HTTP)	0.90
`.innerHTML` + `.classList`	`element` (DOM)	0.85
`.readFileSync`	`fs` (Node.js fs module)	0.90
`.resolve` + `.join`	`path` (Node.js path module)	0.85

Table 5.4: Property Access pattern'lari.

5.9.3 Neden LLM Kullanilmiyor?

1. **Hiz:** 50K identifier × ∼100ms/istek = 83 dakika (LLM). Heuristik: < 2 saniye.
2. **Determinizm:** Ayni girdi → her seferinde ayni cikti.
3. **Maliyet:** API maliyeti ∼$20/analiz.
4. **Cevrimdisi:** Internet gerektirmez.

Trade-off: LLM "processUserAuthentication" gibi akilli isimler ueretebilir. Ama 25 kural pratikte yeterli ve 1000× hizli.

5.10 Webpack Module Extraction — Detayli

5.10.1 Extraction Algoritmasi

`deobfuscate.mjs:543-581`'de:

1. **Module object tespiti:** `ObjectExpression` visitor ile bueyueuek key-value object'leri bul
2. **Heuristik:** En az 2 numeric/string key + en az 2 function value → webpack module object

3. **Cikarma:** Her key-value cifti bir modul. `generate()` ile kaynak kodunu cikar, ayri dosyaya yaz

```
// Heuristik (satir 548-566):
let numericKeyCount = 0, funcValueCount = 0;
for (const prop of props) {
    if (key.type === "NumericLiteral" || key.type === "
       StringLiteral")
        numericKeyCount++;
    if (prop.value.type === "FunctionExpression")
        funcValueCount++;
}
if (numericKeyCount >= 2 && funcValueCount >= 2) {
    // Webpack module object tespit edildi!
}
```

5.10.2 esbuild vs webpack

Ozellik	webpack	esbuild
Module storage	Object literal	Array veya Object
Loader	`__webpack_require__`	`__require` veya inline
Tree shaking	Sinirli	Agresif (ESM-native)
Module ID	Numeric veya string hash	Path-based string

5.11 Chunked Processing — 200MB+ Dosyalar

5.11.1 Problem

200MB JS dosyasi icin Babel parse: ∼4–8× bellek kullanimi → 800MB–1.6GB heap. Node.js default heap 1.5GB → bellek tasmasi.

5.11.2 Neden 512KB Chunk?

`stream-parse.mjs:33`'te: `maxBlockKb = 512`

1. **Babel bellek:** 512KB × 8 = 4MB per chunk. 8GB heap icinde yuezlerce chunk islenebilir.

2. **JS statement boyutu:** Cogu top-level fonksiyon 1–50KB. 512KB bueyueuek fonksiyonlari bile tek chunk'ta tutar.
3. **Cache:** Apple Silicon L2 cache ∼16MB. 512KB rahatca sigar.

5.11.3 Boundary Tespiti

Brace depth tracking: `{` goerueueluuence depth++, `}` goerueuelueuence depth--. `depth === 0` iken yeni top-level statement → split.

Cross-Chunk Reference Sorunu

Bir chunk'taki degisken baska chunk'ta tanimli olabilir. Karadul her chunk'i bagimsiz isleyip sonuclari birlestirir — en basit ve performansli ama semantic bilgi kaybina yol acar.

5.11.4 Acorn Fallback — 32MB+ Dosyalar

Babel parser 32MB+ dosyalarda coekmekti. Coezeueuem: ueuec katmanli hata kurtarma:

1. **Babel parse:** Normal deneme
2. **Pre-beautify + Babel:** Babel basarisiz → js-beautify ile formatlayip tekrar dene
3. **Acorn fallback:** Babel yine basarisiz → Acorn parser devreye girer

Bu fix, LM Studio 32MB renderer bundle'ini coezeueuep 4/4 bundle basarisini sagladi.

5.12 Obfuscation: Formal Tanim ve Teorik Temel

5.12.1 Semantik Koruma Teoremi

Kod obfuscation, bir programin kaynak kodunu, davranisini degistirmeden okunamazlastirma islemidir. Bunu formal olarak tanimlayalim.

Obfuscation Fonksiyonu

Bir obfuscation fonksiyonu $\mathcal{O} : \mathcal{P} \to \mathcal{P}$ asagidaki kosullari saglayan bir donustumdur:

1. **Semantik koruma:** Her girdi x icin, $\mathcal{O}(P)(x) = P(x)$
2. **Polinom yavaslatma:** $|\mathcal{O}(P)| \leq \text{poly}(|P|)$ ve calisma suresi polinomial olarak artar

3. **Analiz zorlugu:** Herhangi bir analiz A icin, $A(\mathcal{O}(P))$'den elde edilen bilgi, $A(P')$'den (P' erisim saglanmamis bir program) elde edilenden anlamli olarak fazla degildir

Bu tanimi daha kompakt yazarsak:

$$\mathcal{O} : \mathcal{P} \to \mathcal{P}, \quad \forall P \in \mathcal{P} : \quad [\![\mathcal{O}(P)]\!] = [\![P]\!] \tag{5.10}$$

burada $[\![\cdot]\!]$ programin denotational semantigini (yani giris-cikis davranisini) gosterir.

Teorem 5.1 (Obfuscation Tersini Alma)**.** Eger $\mathcal{O}$ yalnizca **syntactic** donusumler uyguluyorsa (string encoding, isim degistirme, control flow yeniden yapilama), o zaman bir ters-obfuscation fonksiyonu $\mathcal{D}$ oyle vardir ki:

$$\mathcal{D}(\mathcal{O}(P)) \approx P \tag{5.11}$$

burada $\approx$ yapisal esdegerlik (structural equivalence) anlamindadir.

Ispat fikri: Syntactic obfuscation, AST uzerinde terslenebilir donusumler uygular. Her donusum t_i icin bir ters t_i^{-1} mevcuttur. Obfuscation $\mathcal{O} = t_n \circ \cdots \circ t_1$ ise, $\mathcal{D} = t_1^{-1} \circ \cdots \circ t_n^{-1}$ dir. Ancak pratikte t_i'ler bilinmez; dolayisiyla Karadul gibi araclar **heuristik ters donusumleri** uygulamak zorundadir.

5.12.2 Obfuscation Karmasiklik Siniflamasi

Obfuscation tekniklerini bilgi-kuramsal zorluk derecesine gore siniflandiririz:

$$\text{Zorluk}(\mathcal{O}) = -\sum_i p_i \log_2 p_i + \text{cfg_depth}(\mathcal{O}(P)) \tag{5.12}$$

burada ilk terim Shannon entropisi, ikinci terim control flow graph derinligidir.

Teknik	Zorluk	Tersinirlik	Karadul Kapsami
Minification	Dusuk	∼100%	Beautify
String encoding	Dusuk	∼100%	String unhex
Dead code injection	Orta	∼95%	Dead code elim.
String rotation	Orta	∼90%	Synchrony
Opaque predicates	Orta–Yuksek	∼85%	Pattern + Z3
Control flow flattening	Yuksek	∼70%	CFF deflattener
Virtualization	Cok yuksek	∼30%	Kismi destek

Table 5.5: Obfuscation teknikleri ve zorluk dereceleri.

5.13 Yaygin Obfuscation Teknikleri

5.13.1 Minification

En basit obfuscation formu. Kaynak koddan gereksiz bilgileri (bosluk, yorum, uzun degisken isimleri) kaldirarak dosya boyutunu kucultme ve okunakliligi azaltma islevidir.

```
// Orijinal (okunabilir):
function calculateTotalPrice(items, taxRate) {
    let subtotal = 0;
    for (const item of items) {
        subtotal += item.price * item.quantity;
    }
    return subtotal * (1 + taxRate);
}

// Minified:
function a(b,c){let d=0;for(const e of b)d+=e.price*e.quantity;
   return d*(1+c)}
```

Listing 5.1: Minification oncesi ve sonrasi

Minification bilgi kaybina neden olur: degisken isimleri `a`, `b`, `c` orijinal semantigini tasimaz. Ancak **program davranisi** aynidir — Denklem 5.10 gecerlidir.

5.13.2 String Encoding

String literal'ler cogunlukla iki yontemle gizlenir:

Hex encoding:

$$\text{encode}_{\text{hex}}(s) = \text{concat}_{i=0}^{|s|-1} \texttt{"\textbackslash x"} \cdot \text{hex}(\text{charCodeAt}(s, i)) \tag{5.13}$$

Unicode encoding:

$$\text{encode}_{\text{unicode}}(s) = \text{concat}_{i=0}^{|s|-1} \texttt{"\textbackslash u"} \cdot \text{pad}(\text{hex}(\text{codePointAt}(s, i)), 4) \tag{5.14}$$

Base64 encoding:

$$\text{encode}_{\text{b64}}(s) = \texttt{atob("} \cdot \text{base64}(s) \cdot \texttt{")} \tag{5.15}$$

Ornek: `"Hello"` → `"\x48\x65\x6c\x6c\x6f"` → `atob("SGVsbG8=")`

5.13.3 Control Flow Flattening (CFF)

CFF, programin dogal kontrol akisini bir `while-switch` dispatcher'a donusturur:

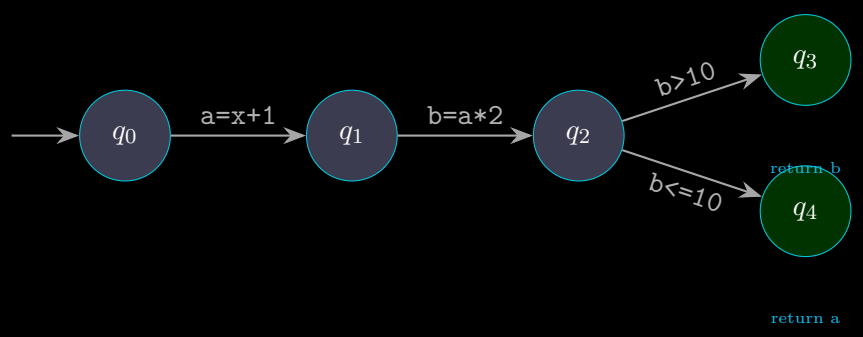
```
// Orijinal:
function process(x) {
    let a = x + 1;     // Block 0
    let b = a * 2;     // Block 1
    if (b > 10) {      // Block 2
        return b;      // Block 3
    }
    return a;           // Block 4
}

// CFF uygulanmis:
function process(x) {
    let state = 0, a, b;
    while (true) {
        switch (state) {
            case 0: a = x + 1; state = 1; break;
            case 1: b = a * 2; state = 2; break;
            case 2: state = b > 10 ? 3 : 4; break;
            case 3: return b;
            case 4: return a;
        }
    }
}
```

Listing 5.2: CFF ornegi

CFF'nin formal modeli bir sonlu durum makinesidir (FSM):

$$\mathrm{CFF}(P) = (Q, \Sigma, \delta, q_0, F) \tag{5.16}$$

burada Q state kuemesi, $\delta : Q \times \Sigma \to Q$ gecis fonksiyonu, q_0 baslangic durumu ve $F \subseteq Q$ bitis durumlaridir.



5.13.4 Opaque Predicates

Opaque predicate, her zaman ayni sonucu veren (true veya false) ama statik analizin bunu kolayca cikaramayacagi ifadelerdir.

Opaque Predicate

Bir ifade φ icin:

- φ^T: Her zaman true ($\forall\sigma : [\![\varphi]\!]_\sigma = \text{true}$)
- φ^F: Her zaman false ($\forall\sigma : [\![\varphi]\!]_\sigma = \text{false}$)
- $\varphi^?$: Bazen true, bazen false (gercek dallanma)

burada σ program state'ini temsil eder.

`opaque_predicate.py` dosyasinda tanimlanan oruntuleri inceleyelim:

Oruuntu	Sonuc	Matematik Ispati
`x*(x+1) % 2 == 0`	φ^T	Ardisik iki sayinin carpimi daima cifttir
`(x | 1) != 0`	φ^T	OR 1 her zaman $\neq 0$ verir
`(a & b) == (a & b)`	φ^T	Her ifade kendisiyle esittir
`2*(x/2) <= x`	φ^T	Tamsayi bolumuenun oezelligi: $\lfloor x/2 \rfloor \cdot 2 \leq x$
`x*(x+1) % 2 != 0`	φ^F	Yukaridakinin tueuemleri
`x != x` (int)	φ^F	Tamsayi kendisiyle esit (NaN notu asagida)

Table 5.6: Opaque predicate oruntuleri ve ispatlar. **Uyari:** `x == x` IEEE 754 NaN icin `false` dondurur; `opaque_predicate.py` bu nedenle float baglam icin confidence'i 0.60'a dusurur.

5.13.5 Dead Code Injection

Opaque predicate'ler ile birlesirildiginde, hicbir zaman calistirilmayan sahte kod bloklari eklenir:

```
if (x * (x + 1) % 2 === 0) {
    // Gercek kod (her zaman calisir)
    processPayment(amount);
} else {
    // Dead code (asla calismaz)
    launchMissiles();  // dikkat dagitma amacli
    deleteDatabase();  // analisti yaniltma
}
```

Listing 5.3: Dead code injection ornegi

5.13.6 String Array Rotation

obfuscator.io gibi araclar tum string'leri bir diziye tasir, diziyi kaydirma (rotation) islemiyle karistirip indeksle erisirir:

```
// Orijinal:
console.log("Hello");

// Obfuscated:
var _0x1234 = ["Hello", "log", "World"];
(function(_0x, _0y) {
    var _0z = function(_0w) { while(--_0w) { _0x.push(_0x.shift
        ()); } };
    _0z(++_0y);
})(_0x1234, 0x1);  // Diziyi 1 kaydır
console[_0x1234[0]](_0x1234[1]);  // console.log("World") --
    indeksler degisti!
```

Listing 5.4: String rotation ornegi

Karadul'un `synchrony` biliseni bu patterni dogrudan cozer cunku synchrony, obfuscator.io spesifik bir deobfuscation aracidir.

5.14 Abstract Syntax Tree (AST) Temelleri

Tum deobfuscation pipeline'i AST uzerinde calisir. Bu bolum AST'nin ne oldugunu, parse tree ile farkini ve ESTree standardini aciklar.

5.14.1 BNF ve Context-Free Grammar (CFG)

JavaScript dili bir context-free grammar (CFG) ile tanimlenir:

$$G = (V, \Sigma, R, S) \tag{5.17}$$

burada V non-terminal semboller, Σ terminal semboller, R uretim kurallari ve S baslangic sembolueduer.

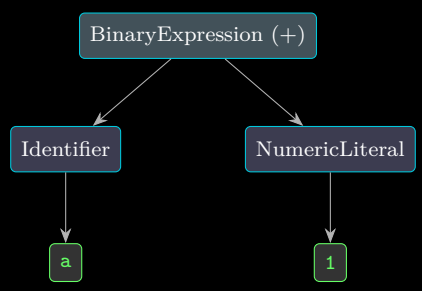
JavaScript'in basitlestirilmis BNF'inde `IfStatement`:

```
IfStatement ::= "if" "(" Expression ")" Statement ("else"
    Statement)?
Expression  ::= AssignmentExpression | Expression ","
    AssignmentExpression
Statement   ::= Block | IfStatement | ForStatement |
    ExpressionStatement | ...
```

5.14.2 Parse Tree vs Abstract Syntax Tree

Parse tree (concrete syntax tree), gramerin her uretim kuralini icerir — parantezler, noktalama isaretleri, terminal semboller hepsi agacta gorunur. **AST** ise semantik olarak anlamsiz node'lari atar.

AST: `a + 1`



5.14.3 ESTree Spesifikasyonu

JavaScript AST'si icin fiili standart **ESTree** spesifikasyonudur (`https://github.com/estree/estree`). Babel parser, ESTree'nin genisletilmis bir versiyonunu uretir:

- **Babel AST:** `StringLiteral`, `NumericLiteral`, `ObjectProperty`
- **Acorn/ESTree:** `Literal`, `Property`

Bu fark `deobfuscate.mjs`'te asagidaki kontrol ile ele alinir:

```
const usedAcorn = ast.body !== undefined && !ast.program;
// Babel AST'de ast.program var, Acorn'da yok
```

Listing 5.5: Babel vs Acorn AST tespiti (deobfuscate.mjs)

5.14.4 Babel Parser Oezellikleri

`deep-deobfuscate.mjs` Babel parser'i su opsiyonlarla cagrilir:

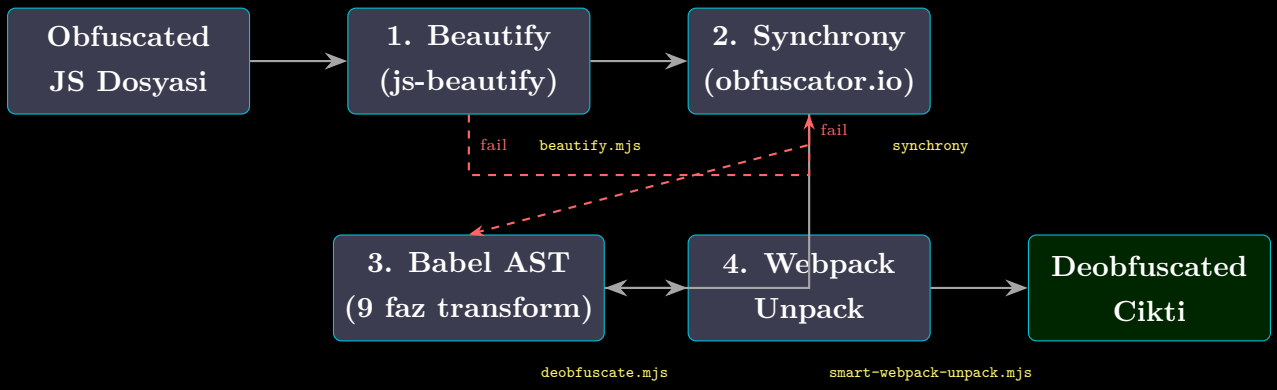
```
ast = parse(source, {
  sourceType: "unambiguous",
  allowImportExportEverywhere: true,
  allowReturnOutsideFunction: true,
  errorRecovery: true,  // Parse hatalarinda devam et
  plugins: ["jsx", "typescript", "decorators-legacy",
    "classProperties", "dynamicImport",
    "optionalChaining", "nullishCoalescingOperator",
    "topLevelAwait", "importMeta"],
});
```

Listing 5.6: Babel parser konfigurasyonu

`errorRecovery: true` kritik bir secimdir. 30MB+ minified dosyalarda parser kismen basarisiz olabilir; bu mod, hatalari toplar ama parse islemini durdurmaz. Boylece kismi AST uzerinde islem yapilabilir.

5.15 Standart 4 Adimli Deobfuscation Zinciri

`manager.py` dosyasindaki `DeobfuscationManager` sinifi, standart 4 adimli zinciri orkestre eder:



5.15.1 Graceful Degradation Formuelue

Her adim basarisiz olabilir. Bu durumda bir onceki basarili cikti sonraki adima gecilir:

$$\text{output}(i) = \begin{cases} \text{step}_i(\text{output}(i-1)) & \text{eger step}_i \text{ basarili} \\ \text{output}(i-1) & \text{eger step}_i \text{ basarisiz (fallback)} \end{cases} \tag{5.18}$$

Zincirin en az bir adimi basarili olursa, tum zincir "basarili" sayilir:

$$\text{success}(\text{chain}) = \bigvee_{i=1}^{n} \text{success}(\text{step}_i) \tag{5.19}$$

`manager.py` bunu su kodla implement eder:

```
for idx, step_name in enumerate(effective_chain, start=1):
    step_output = deob_dir / f"{idx:02d}_{step_name}{input_file
        .suffix}"
    try:
        step_success = self._execute_step(
            step_name, current_input, step_output, deob_dir)
    except Exception as exc:
        step_success = False
    if step_success and step_output.exists():
        result.steps_completed.append(step_name)
        current_input = step_output  # Sonraki adimin girdisi
    else:
        result.steps_failed.append(step_name)
        # current_input degismez -> onceki basarili cikti
            kullanilir
```

Listing 5.7: Graceful degradation – manager.py, run_chain()

5.15.2 Dosya Isimlendirme Konvansiyonu

Her adim, ciktisini numarali dosya olarak kaydeder:

`00_original.js`	Orijinal obfuscated dosya
`01_beautify.js`	Beautify sonrasi
`02_synchrony.js`	Synchrony sonrasi
`03_babel_transforms.js`	Babel AST sonrasi
`04_webpack_unpack.js`	Webpack modul ayirma sonrasi

Bu sayede her adimin ciktisi bagimsiz olarak incelenebilir — debug ve audit icin kritik.

5.16 Adim 1: Beautify

`beautify.mjs`, `js-beautify` kutuphanesini kullanan basit ama kritik bir onisleme adimidir. Minified kod tek satir halindedir; AST parse bile basarisiz olabilir cunku Babel'in ternary+arrow+object pattern'lerinde takilma riski vardir.

5.16.1 Konfiguerasyon Parametreleri

```
1 js_beautify(source, {
2   indent_size: 2,
3   indent_char: " ",
4   max_preserve_newlines: 2,
5   brace_style: "collapse,preserve-inline",
6   wrap_line_length: 120,
7   break_chained_methods: true,
8   end_with_newline: true,
9 });
```

Listing 5.8: js-beautify konfigurasyonu – beautify.mjs

5.16.2 Etki Analizi

Beautify'in etkisini olcmek icin genisleme oranini (expansion ratio) tanimlayalim:

$$R_{\text{expand}} = \frac{L_{\text{out}}}{L_{\text{in}}} \tag{5.20}$$

burada L_{in} girdi satir sayisi, L_{out} cikti satir sayisi. Tipik degerler:

- Agresif minification (tek satir): $R_{\text{expand}} \approx 100$–$500$
- Orta minification: $R_{\text{expand}} \approx 2$–$5$
- Zaten formatlenmis: $R_{\text{expand}} \approx 1$

5.16.3 Fallback Davranisi

Beautify basarisiz olursa, girdi dosyasi dogrudan ciktiya kopyalanir:

```
def _step_beautify(self, input_file, output_file):
    beautify_script = self._config.scripts_dir / "beautify.mjs"
    if not beautify_script.exists():
        shutil.copy2(input_file, output_file)  # Fallback
        return True
```

Listing 5.9: Beautify fallback – manager.py

5.17 Adim 2: Synchrony Deobfuscation

`synchrony_wrapper.py`, `synchrony` aracini saran bir Python wrapper'dir. Synchrony, obfuscator.io spesifik bir deobfuscation aracidir ve string rotation, string array, control flow flattening gibi obfuscator.io'ya oezgue teknikleri cozer.

5.17.1 Calismaa Mekanizmasi

1. **Shebang soyma:** Eger dosya `#!/usr/bin/env node` ile basliyorsa, synchrony'nin Acorn parser'i bunu parse edemez. Wrapper gecici dosyada shebang'i soyar, islem sonrasi geri ekler.
2. **Source type deneme:** Synchrony uc modda denenir: `both`, `module`, `script`. ESM dosyalarda `ImportDeclaration` hatasi alinirsa `module` moduna gecilir.
3. **Dogrudan dosya ciktisi:** `-o` flag'i ile stdout yerine dogrudan dosyaya yazar. Buyuk dosyalarda memory overhead'i onler.

```
source_types = ["both", "module", "script"]
for source_type in source_types:
    cmd = [sync_path, str(actual_input),
           "-o", str(output_file),
           "--type", source_type]
```

```
    result = self._runner.run_command(cmd, timeout=timeout)
    if result.success:
        break
    if "ImportDeclaration" in result.stderr:
        continue  # ESM hatasi, module modunu dene
```

Listing 5.10: Synchrony calisma akisi – synchrony_wrapper.py

5.17.2 Boyut Analizi

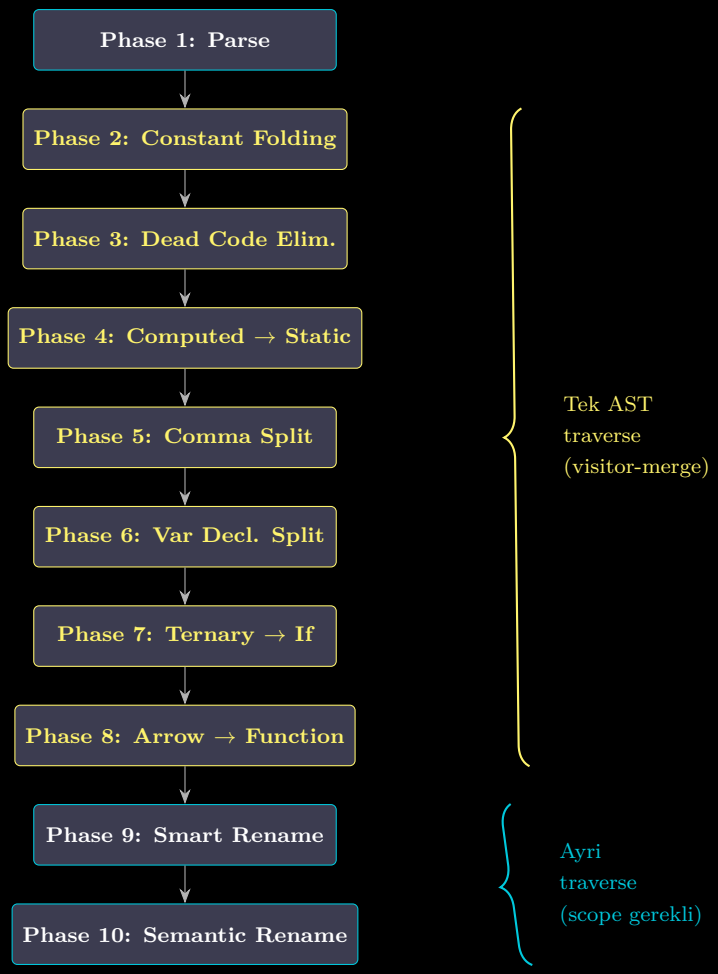
Synchrony basarili oldugunda dosya boyutu genellikle **kucueluer** cunku string rotation ve dead code kaldirilir:

$$R_{\text{sync}} = \frac{|\text{output}|}{|\text{input}|} \in [0.3, 1.0] \tag{5.21}$$

$R_{\text{sync}} < 0.5$ ise agresif obfuscation kaldirilmis demektir.

5.18 9 Fazli Deep Deobfuscation Pipeline

`deep-deobfuscate.mjs` dosyasi, 10 asamali (9 transform + 1 parse) derin deobfuscation pipeline'i uygular. Phase 2–8, **visitor-merge optimizasyonu** ile tek bir AST traverse'da birlestirilmistir — bu webcrack yaklasimi, 7 ayri traverse yerine 1 traverse kullanarak ∼3–5x hiz kazanci saglar.



5.18.1 Faz 1: Parse (Tolerant Mode)

Babel parser `errorRecovery: true` ile calistirilir. Basarisiz olursa iki fallback mevcuttur:

1. **Pre-beautify + retry:** Minified kaynak once `js-beautify` ile formatlanir, sonra Babel tekrar denenir. 30MB+ dosyalarda Babel'in ternary/arrow/object pattern'lerinde takilma sorunu bu yontemle cogunlukla cozulur.

2. **Dosya kopyalama:** Her iki yontem de basarisiz olursa, kaynak dosya oldugu gibi output'a kopyalanir. Boylece sonraki pipeline adimlari (webpack unpack) en azindan orijinal dosya uzerinde devam edebilir.

$$\text{parse}(S) = \begin{cases} \text{Babel}(S) & \text{basarili} \\ \text{Babel}(\text{beautify}(S)) & \text{pre-beautify ile retry} \\ S & \text{fallback: kaynak kopyalama} \end{cases} \tag{5.22}$$

5.18.2 Faz 2: Constant Folding (Sabit Katlama)

Constant folding, derleme/analiz zamaninda hesaplanabilir ifadeleri literal degerleriyle degistirme islemidir.

Lattice Teorisi ve Abstract Interpretation

Constant folding, abstract interpretation cercevesinde bir **flat lattice** uzerinde calisir:

$$L = \{\bot, c_1, c_2, \ldots, c_n, \top\} \tag{5.23}$$

burada:

- $\bot$: Degisken henuz kullanilmadi (undefined)
- c_i: Degisken sabit c_i degerinde
- $\top$: Degisken birden fazla deger alabilir (non-constant)

$$\text{meet}(x, y) = \begin{cases} x & \text{eger } x = y \\ \top & \text{eger } x \neq y \text{ ve her ikisi de } \neq \bot \\ y & \text{eger } x = \bot \\ x & \text{eger } y = \bot \end{cases} \tag{5.24}$$

Karadul Implementasyonu

`deep-deobfuscate.mjs` Phase 2'de asagidaki donusumler uygulanir:

Node Tipi	Donusum	Ornek
BinaryExpression (+)	String concat	`"He"+"llo"` → `"Hello"`
BinaryExpression (+)	Numeric add	`1+2` → `3`
BinaryExpression ($*$, $/$, $-$)	Aritmetik	`6*7` → `42`
BinaryExpression ($\|$,&,$\hat{}$,$\ll$,$\gg$)	Bitwise	`0xFF & 0x0F` → `15`
UnaryExpression (!)	Negation	`!0` → `true`
UnaryExpression (void)	Void	`void 0` → `undefined`

Table 5.7: Constant folding donusumm tablosu.

Guevenlik kontrolu: Bolme islemlerinde `right.value !== 0` kontrol edilir ve Number.isFinite($result$) dogrulanir. Overfllow, NaN ve Infinity sonuclari degistirilmez.

```
BinaryExpression: {
  exit(path) {
    const { left, right, operator } = path.node;
    // String concatenation
    if (operator === "+" && t.isStringLiteral(left)
        && t.isStringLiteral(right)) {
      path.replaceWith(t.stringLiteral(left.value + right.value
        ));
      cnt2++;
    }
    // Numeric operations
    if (t.isNumericLiteral(left) && t.isNumericLiteral(right))
      {
      let result;
      switch (operator) {
        case "+": result = left.value + right.value; break;
        case "-": result = left.value - right.value; break;
        // ... (*, /, %, **, |, &, ^, <<, >>, >>>)
      }
      if (result !== undefined && Number.isFinite(result)) {
        path.replaceWith(t.numericLiteral(result));
        cnt2++;
      }
    }
  }
}
```

Listing 5.11: Constant folding visitor – BinaryExpression

Neden `exit` visitor? Bottom-up traversal sayesinde ic node'lar once islenir: `(1+2)*3`'te once `1+2=3`, sonra `3*3=9`. `enter` kullansaydik, ic node'lar henuz fold edilmemis olurdu.

5.18.3 Faz 3: Dead Code Elimination

Liveness Analysis Temeli

Liveness analysis, her program noktasinda hangi degiskenlerin "canli" (daha sonra kullanilacak) oldugunu belirler:

$$\text{live_in}(B) = \text{gen}(B) \cup (\text{live_out}(B) \setminus \text{kill}(B)) \tag{5.25}$$

$$\text{live_out}(B) = \bigcup_{S \in \text{succ}(B)} \text{live_in}(S) \tag{5.26}$$

burada:

- $\text{gen}(B)$: B blogunda kullanilan (okunan) degiskenler
- $\text{kill}(B)$: B blogunda tanimlanan (yazilan) degiskenler

Fixed-Point Iteration

Liveness analysis, denklem sistemi sabit noktaya ulasana kadar iteratif olarak cozulur:

$$\text{live_in}^{(k+1)}(B) = \text{gen}(B) \cup \left(\text{live_out}^{(k)}(B) \setminus \text{kill}(B)\right) \tag{5.27}$$

Karmasiklik: $\mathcal{O}(|B| \times |V|)$ burada $|B|$ basic block sayisi, $|V|$ degisken sayisi.

Karadul Implementasyonu

Phase 3, `IfStatement` ve `ConditionalExpression` node'larini isler:

```
IfStatement: {
  exit(path) {
    const test = path.node.test;
    // if (false) {...} -> kaldir veya else branch'i al
    if (t.isBooleanLiteral(test) && test.value === false) {
      if (path.node.alternate) path.replaceWith(path.node.
         alternate);
      else path.remove();
      cnt3++;
    }
    // if (true) {X} else {Y} -> X
    if (t.isBooleanLiteral(test) && test.value === true) {
      path.replaceWith(path.node.consequent);
      cnt3++;
    }
    // if (0) -> remove, if (1) -> consequent
    if (t.isNumericLiteral(test)) {
      if (test.value === 0) { /* remove or alternate */ }
      else { path.replaceWith(path.node.consequent); }
      cnt3++;
    }
```

```
21    }
22 }
```

Listing 5.12: Dead code elimination – IfStatement

Phase 2 ile etkilesim: Phase 2 `!0` → `true` donusumunu yapar. Phase 3, `exit` visitor kullandigi icin, ayni traverse'da Phase 2'nin ciktisini gorebilir. Ornegin `if(!0){X}` once `if(true){X}` olur (Phase 2), sonra `X` olur (Phase 3).

5.18.4 Faz 4: Computed-to-Static Property

Bu faz, `obj["foo"]` gibi computed property erisimlerini `obj.foo` gibi statik erisimlere donusturur.

$$\text{obj}[\texttt{"key"}] \xrightarrow{\text{Faz 4}} \text{obj.}\texttt{key} \quad \text{eger } \texttt{key} \in \text{ValidIdentifier} \tag{5.28}$$

Valid Identifier

Bir string `key` gecerli JavaScript identifier ise ve reserved word degilse donusum guevenlidir:

$$\text{ValidIdentifier}(\texttt{key}) = \texttt{/^[a-zA-Z_\$][a-zA-Z0-9_\$]*\$/}.\mathit{test}(\texttt{key}) \wedge \neg\text{isReserved}(\texttt{key}) \tag{5.29}$$

AST duenuesuemuee:

- `MemberExpression`: `computed: true` → `computed: false`
- `ObjectProperty`: Computed key'i identifier'a donustur

```
1  MemberExpression(path) {
2    if (!path.node.computed) return;
3    const prop = path.node.property;
4    if (!t.isStringLiteral(prop)) return;
5    if (/^[a-zA-Z_$][a-zA-Z0-9_$]*$/.test(prop.value)
6        && !isReservedWord(prop.value)) {
7      path.node.computed = false;
8      path.node.property = t.identifier(prop.value);
9      cnt4++;
10   }
11 }
```

Listing 5.13: Computed to static – MemberExpression

5.18.5 Faz 5: Comma Expression Splitting

JavaScript'te virgul operatoru (`SequenceExpression`) birden fazla ifadeyi tek bir expression'da birlestirir. Obfuscator'lar bunu okunakliligi azaltmak icin kullanir:

```
// Obfuscated:
return a = 1, b = 2, c = 3, process(a, b, c);
// Deobfuscated (Faz 5 sonrasi):
a = 1;
b = 2;
c = 3;
return process(a, b, c);
```

Donusum ucg baglam icin uygulanir:

1. **ExpressionStatement:** `(a, b, c);` → `a; b; c;`
2. **ReturnStatement:** `return (a, b, c);` → `a; b; return c;`
3. **ThrowStatement:** `throw (a, b, c);` → `a; b; throw c;`

$$\text{split}(\texttt{return}(e_1, e_2, \ldots, e_n)) = [\text{expr}(e_1), \ldots, \text{expr}(e_{n-1}), \texttt{return}\ e_n] \tag{5.30}$$

5.18.6 Faz 6: Variable Declaration Splitting

Tek bir `var/let/const` icindeki birden fazla tanimlama ayri satirlara bolunur:

```
// Oncesi:
var a = 1, b = 2, c = fn(), d = [];
// Sonrasi:
var a = 1;
var b = 2;
var c = fn();
var d = [];
```

Guvenlik kontrolleri: For-loop init (`for(var i=0, j=0; ...)`), for-in ve for-of icerisindeki deklarasyonlar bolunmez:

```
if (path.parent.type === "ForStatement" && path.key === "init")
    return;
if (path.parent.type === "ForInStatement") return;
if (path.parent.type === "ForOfStatement") return;
```

Listing 5.14: Variable declaration split – guvenlik kontrolleri

5.18.7 Faz 7: Ternary → If/Else

Uzun ternary ifadeler if/else bloklarina donusturulur. "Uzun" tanimi: test + consequent + alternate toplam karakter sayisi ≥ 80:

$$|\text{code(test)}| + |\text{code(cons)}| + |\text{code(alt)}| \geq 80 \tag{5.31}$$

```
// Oncesi (tek satir, okunamaz):
isAuthenticated ? renderDashboard(user, session) :
    redirectToLogin(returnUrl);
// Sonrasi:
if (isAuthenticated) {
  renderDashboard(user, session);
} else {
  redirectToLogin(returnUrl);
}
```

5.18.8 Faz 8: Arrow → Named Function

`const foo = () => {...}` formundaki arrow function'lar named function declaration'a donusturulur:

```
// Oncesi:
const processData = (items) => { return items.map(x => x * 2);
    };
// Sonrasi:
function processData(items) { return items.map(x => x * 2); }
```

Semantik guevenlik: `this` kullanan arrow function'lar donusturulmez. Arrow function'larda `this` lexical scope'a baglidir; normal function'larda ise call-site'a baglidir. Bu semantik fark bozulmamalidir:

```
const { code: bodyCode } = generate(arrow.body, { compact: true
    });
if (!bodyCode.includes("this.") && !bodyCode.includes("this["))
    {
    // Guvenli: donustur
    const funcDecl = t.functionDeclaration(
      t.identifier(decl.id.name), arrow.params, arrow.body,
      arrow.generator || false, arrow.async || false);
    path.replaceWith(funcDecl);
```

```
}
```

Listing 5.15: Arrow-to-function this kontrolu

5.18.9 Faz 9: Smart Variable Renaming

Bu faz, minified identifier'lari (1–2 karakter) anlamli isimlere donusturur. LLM **kullanmaz** — tamamen heuristik kurallara dayanir. Babel `scope.rename()` API'si ile scope-safe renaming yapilir.

Uec Kural Kategorisi

1. **Module-based:** `require("fs")` → degisken `fs` olarak adlandirilir
2. **Usage-based:** Degisken uzerinde `.push()`, `.map()` cagriliyorsa → `items`
3. **Fallback:** Tek harf degiskenler icin varsayilan isimler (`i` → `idx`, `e` → `elem`)

Pattern	Tespit Yontemi	Yeni Isim
`require("fs")`	CallExpression analizi	`fs`
`require("path")`	CallExpression analizi	`path`
`require("express")`	CallExpression analizi	`express`
`catch(e)`	CatchClause param	`error`
`x.push()`, `x.map()`	Usage (Array API)	`items`
`x.trim()`, `x.split()`	Usage (String API)	`text`
`x.message`, `x.stack`	Property access	`err`
`x.then()`, `x.catch()`	Usage (Promise API)	`promise`
Express `(req, res, next)`	Parametre pozisyonu	`request, response, next`
esbuild `(exports, module)`	Factory pattern	`exports, module`

Table 5.8: Phase 9 smart rename kural ornekleri.

Tek Harf Fallback Tablosu

```
const defaults = {
  a: "arg", b: "buf", c: "ctx", d: "data",
  e: "elem", f: "fn", g: "gen", h: "handler",
  i: "idx", j: "jdx", k: "key", l: "len",
  m: "mod", n: "count", o: "opts", p: "param",
```

```
  q: "query", r: "res", s: "str", t: "tmp",
  u: "usr", v: "val", w: "writer",
};
```

Listing 5.16: Tek harf fallback mapping

5.18.10 Faz 10: Semantic Renaming (Deep Usage-Based)

Phase 10, Phase 9'un basit pattern matching'ini asarak, Babel scope + binding analizi ile her minified identifier'in kullanim baglamini derinlemesine analiz eder. **12 kural motoru** ile calisir.

Kullanim Bilgisi Toplama (_collectUsage10)

Her identifier icin asagidaki bilgiler toplanir:

```
{
  methodCalls: Set,        // x.push(), x.split() gibi
  propertyReads: Set,      // x.length, x.name gibi
  propertyWrites: Set,     // x.count = 5 gibi
  calledAs: boolean,       // x() seklinde cagriliyor mu
  comparedWith: string[], // x === "GET" gibi karsilastirmalar
  typeofChecks: string[], // typeof x === "function" gibi
  usedInForOf: boolean,    // for(const y of x) iteratorde mi
  destructured: boolean,   // const {a, b} = x seklinde mi
  indexAccessed: boolean,   // x[0] seklinde mi
  arithmeticOperand: boolean, // x + 1 gibi aritmetikte mi
  thrownAsError: boolean, // throw x seklinde mi
  usedInNew: boolean,      // new x() seklinde mi
}
```

Listing 5.17: Usage bilgi yapisi

12 Kural Motoru

Phase 10+: Enhanced Rename (cursor-enhanced-rename.mjs)

`cursor-enhanced-rename.mjs`, Phase 10'un kapsamadigi 15 ek kural ile kalan minified identifier'lari isler. `deep_pipeline.py`'da Adim 2.5 olarak calistirilir:

#	Kural	Conf.	Tespit Kriteri
1	require_module	0.95	require("fs") → fs
2	new_error	0.90	new Error() pattern
3	error_props	0.90	.message + .stack props
4	string_api	0.85	≥2 string method
5	array_api	0.85	≥2 array method
6	promise_api	0.85	.then() + .catch()
7	fs_api	0.90	readFileSync, writeFile
8	http_req	0.80	≥2 of body,params,query,headers
9	dom_api	0.85	≥2 DOM property
10	callback	0.80	calledAs + typeof === "function"
11	for_of_item	0.80	for(const x of iterable)
12	multi_compare	0.70	≥3 string karsilastirmasi

Table 5.9: Phase 10 semantic renaming kural motoru.

5.19 Webpack Module Extraction

smart-webpack-unpack.mjs, webpack ve esbuild bundle'larini modullerine ayiran akilli bir module splitter'dir. 7 farkli bundle formatini destekler.

5.19.1 Desteklenen Bundle Formatlari

1. **esbuild CJS wrapper:** var X = U((exports, module) => {...})
2. **esbuild ESM interop:** var Y = Q1(X)
3. **esbuild re-export:** lb(target, {name: () => ref})
4. **Webpack IIFE object:** (function(modules) {...})({0: function(e,t,n){...}})
5. **Webpack IIFE array:** [function(e,t,n){...}, ...]
6. **Webpack 5 named:** __webpack_modules__ = {"./src/file.js": ...}
7. **Top-level IIFE:** (function() {...})() — kapsayici IIFE'leri ayirma

5.19.2 ___webpack_require___ Mekanizmasi

Webpack bundle'larinda moduller __webpack_require__(id) ile birbirini cagrilir. Bu pattern AST'de tespit edilir:

#	Kural	Aciklama
11	Prototype access	`x.prototype` → `cls`
12	Binary ops dominant	Cok fazla `+`, `-`, `«` → `offset`
13	Purely called	Sadece cagriliyor → `handler`
14	Event listener pos	Callback pozisyonu → `eventArg`
15	Member assign	Atama kaynagi → `ref`
16	Single property	Tek prop baskini → contextual
17	Conditional return	`return x ? a : b` → `result`
18	Switch/if chain	Karsilastirma zinciri → `kind`
19	Namespace access	`X.C`, `X.Q` → `ns`
20	Loop counter	`for` init → `idx`
21	Await expression	`await x` → `awaited`
22	Default param ctx	Varsayilan deger → `optParam`
23	Clash dedup	Phase 9 artigi → re-score
24	Length/size dominant	`.length` baskini → `collection`
25	Bitwise dominant	`|`, `&`, `^` → `flags`

Table 5.10: Enhanced rename ek kurallari (Kural 11–25).

```
if (callee.name === "__webpack_require__"
    && node.arguments.length >= 1) {
  const arg = node.arguments[0];
  if (arg.type === "NumericLiteral") moduleId = arg.value;
  else if (arg.type === "StringLiteral") moduleId = arg.value;
  if (moduleId !== null && !_webpackIdSet.has(moduleId)) {
    _webpackIdSet.add(moduleId);
    webpackModules.push(moduleId);
  }
}
```

Listing 5.18: Webpack module tespiti – deobfuscate.mjs

5.19.3 esbuild vs Webpack Format Farklari

Ozellik	Webpack	esbuild
Modul sarmalama	IIFE + nesne/dizi	CJS factory wrapper
Modul ID	Sayisal (0, 1, 2...)	Degisken adi
Dependency	`__webpack_require__`	Dogrudan degisken referansi
Entry point	Ilk cagri arg.	Top-level kod
Tree shaking	Plugin bazli	Varsayilan

Table 5.11: Webpack vs esbuild bundle farklari.

5.20 Buyuk Dosya Isleme: Chunked Processing

200MB+ dosyalar icin standart AST parsing bellege sigmaz. `deep_pipeline.py` otomatik olarak **chunked processing** moduna gecer.

5.20.1 Esik Degeri ve Tetikleme

$$\text{mode} = \begin{cases} \text{normal} & \text{eger dosya} < 100 \text{ MB} \\ \text{chunked} & \text{eger dosya} \geq 100 \text{ MB} \end{cases} \tag{5.32}$$

`deep_pipeline.py`'da `LARGE_FILE_THRESHOLD_MB = 100` olarak tanimli.

5.20.2 Stream-Parse Pipeline

1. **stream-parse.mjs:** Dosyayi top-level block'lara ayirir (`-max-block-kb 512`)
2. **Her block'u bagimsiz isle:** `deep-deobfuscate.mjs` her chunk uzerinde ayri calisir (120s timeout/block)
3. **Birlestir:** Tum chunk ciktilari tek dosyada birlestirilir
4. **Webpack unpack:** Birlestirilmis dosya uzerinde modul ayirma
5. **Ikinci pass:** Her modul uzerinde variable renaming

$$T_{\text{chunked}} \approx T_{\text{split}} + n \cdot T_{\text{deob/chunk}} + T_{\text{merge}} + T_{\text{unpack}} \tag{5.33}$$

burada n chunk sayisi, $T_{\text{deob/chunk}}$ chunk basina deobfuscation suresi.

5.20.3 Timeout Hesaplaması

`deep_pipeline.py` dosya boyutuna gore dinamik timeout hesaplar:

$$T_{\text{timeout}} = \max(300, \min(1800, \lceil S_{\text{MB}} \times 100 \rceil)) \text{ saniye} \tag{5.34}$$

burada S_{MB} dosya boyutu megabayt cinsindendir. Minimum 5 dakika, maksimum 30 dakika.

5.21 Control Flow Flattening (CFF) Deflattening

`cff_deflattener.py`, CFF obfuscation'ini geri alan bir Python modul-udur. Hem C hem JavaScript uzerinde calisir.

5.21.1 Tespit Algoritması

CFF dispatcher pattern regex ile tespit edilir:

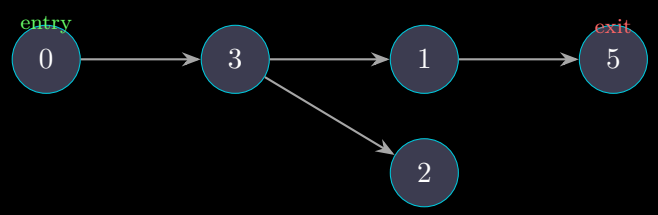
```
# JS CFF pattern
_WHILE_SWITCH_JS = re.compile(
    r"(?:while\s*\(\s*(?:!0|true|1)\s*\)|for\s*\(\s*;;\s*\))\s*\{"
    r"\s*switch\s*\(\s*(\w+)\s*\)\s*\{",
    re.IGNORECASE,
)
```

Listing 5.19: CFF dispatcher tespiti

5.21.2 State Transition Graph Cikarma

Her `case` blogundan state gecisleri cikarilarak bir yoenluee graf (directed graph) olusturulur:

$$G = (V, E) \quad \text{burada } V = \{s_0, s_1, \ldots, s_n\}, \quad E = \{(s_i, s_j) : s_i \xrightarrow{\text{state}} s_j\} \tag{5.35}$$



5.21.3 Topological Sort ile Yeniden Olusturma

State transition graph'i uzerinde BFS-based topological sort uygulanarak orijinal kod sirasi elde edilir:

```
def _topological_sort(self, graph, blocks):
    order, visited = [], set()
    entry = blocks[0].state_value
    queue = [entry]
```

```
    while queue:
        state = queue.pop(0)
        if state in visited: continue
        visited.add(state)
        order.append(state)
        for ns in graph.get(state, []):
            if ns not in visited:
                queue.append(ns)
    # Ziyaret edilmemis state'leri sona ekle
    for block in blocks:
        if block.state_value not in visited:
            order.append(block.state_value)
    return order
```

Listing 5.20: Topological sort – cff_deflattener.py

5.21.4 Karmasiklik Analizi

$$T_{\text{CFF}} = \mathcal{O}(|C| + |V| + |E|) \tag{5.36}$$

burada $|C|$ case sayisi, $|V|$ state sayisi ($=|C|$), $|E|$ gecis sayisi. Pratikte $|E| \leq 2|V|$ (her case en fazla 2 gecis yapar — true/false branch).

5.22 Opaque Predicate Removal

`opaque_predicate.py`, bilinen opaque predicate pattern'lerini regex ile tespit edip dead code'u isaretler veya kaldirir.

5.22.1 Tespit Stratejileri

1. **Pattern matching:** 7 always-true + 4 always-false regex pattern (Tablo 5.6)
2. **Constant folding:** Sabit deger kullanan branch'ler (Phase 3 ile entegre)
3. **Z3 SMT solver (opsiyonel):** Karmasik opaque predicate'lar icin symbolic verification

5.22.2 Z3 Entegrasyonu

Eger z3-solver kurulu ise, pattern matching ile tespit edilemeyen karmasik predicate'lar Z3 ile dogrulanabilir:

$$\text{is_opaque}(\varphi) = \begin{cases} \text{true} & \text{eger } Z3.\text{prove}(\forall x : \varphi(x) = \text{true}) \\ \text{false} & \text{eger } Z3.\text{find_counterexample}(\varphi) \neq \varnothing \end{cases} \tag{5.37}$$

`opaque_predicate.py`'da Z3 lazy-load yapilir:

```
if use_z3:
    try:
        import z3  # noqa: F401
        self._z3_available = True
    except ImportError:
        logger.info("z3-solver bulunamadi, "
                    "pattern-based tespit kullanilacak")
```

Listing 5.21: Z3 lazy-load – opaque_predicate.py

5.22.3 Eliminasyon

- **Always-true:** `if(OPAQUE_TRUE){X} else {Y}` → `X`
- **Always-false:** `if(OPAQUE_FALSE){X} else {Y}` → `Y` (veya sil)

5.23 String Decryption

`string_decryptor.py`, binary'lerdeki sifrelenmis string'leri cozen bir moduld-ur. JavaScript deobfuscation ile dogrudan ilgili olmasa da, Karadul'un genel deobfuscation yeteneginin parcasidir.

5.23.1 Desteklenen Sifreleme Yoentemleri

1. **XOR single-byte:** `buf[i] ^= 0xNN`
2. **XOR multi-byte:** `buf[i] ^= key[i % keylen]`
3. **XOR rolling:** `prev = buf[i] ^ prev`
4. **RC4:** S-box initialization + swap pattern

5. **Base64:** `atob("SGVsbG8=")`
6. **Stack string:** Char-by-char push (Ghidra decompile ciktisinda)

5.23.2 Stack String Reconstruction

Ghidra decompile ciktisinda ardisik local degiskenlere tek karakter atamalari "stack string" olarak yeniden birlestirilir:

```
local_28 = 0x48;  # 'H'
local_27 = 0x65;  # 'e'
local_26 = 0x6c;  # 'l'
local_25 = 0x6c;  # 'l'
local_24 = 0x6f;  # 'o'
# -> "Hello" (confidence: 0.85)
```

Listing 5.22: Stack string ornegi (Ghidra decompile ciktisi)

5.23.3 XOR Brute-Force

Kisa string'ler icin tum 256 tek-byte anahtar denenerek en yuksek "printable" skorlu sonuc secilir:

$$\text{score}(k) = \frac{|\{c \in \text{decrypt}(d,k) : c \in \text{printable}\}|}{|\text{decrypt}(d,k)|} \tag{5.38}$$

$\text{score}(k) > 0.7$ olan ilk 5 sonuc dondurulur.

5.24 Acorn Fallback

Babel parser 32MB+ dosyalarda veya ozel syntax pattern'lerinde basarisiz olabilir. `deobfuscate.mjs` bu durumda Acorn parser'a duser.

5.24.1 Babel vs Acorn Karsilastirmasi

Acorn, `deobfuscate.mjs`'te lazy-load edilir ve sadece Babel basarisiz olursa yuklenir:

```
let acorn = null, acornWalk = null;
function loadAcorn() {
  try {
    acorn = _require("acorn");
```

Ozellik	Babel	Acorn
AST formati	Babel (genisletilmis)	ESTree (standart)
Error recovery	✓	Kismi
TypeScript desteegi	✓(plugin)	×
JSX desteegi	✓(plugin)	×
Performans (30MB)	∼45s	∼8s
Memory (30MB)	∼4GB	∼1.5GB

Table 5.12: Babel vs Acorn parser karsilastirmasi.

```
5      acornWalk = _require("acorn-walk");
6      return true;
7    } catch { return false; }
8  }
```

Listing 5.23: Acorn lazy-load

Bu Bolumun Ozeti

JavaScript deobfuscation, 4 adimli standart zincir (beautify → synchrony → babel → webpack) ve 10 fazli derin transform pipeline'indan olusur. Phase 2–8, visitor-merge optimizasyonu ile tek AST traverse'da birlestirilmistir (∼3–5x hiz kazanci). Graceful degradation her adimda basarisizliga tolerans saglar. Faz 9–10'daki LLM-siz semantik isimlendirme toplam 25+ heuristik kuralla etkileyici sonuclar ueretir. 200MB+ dosyalar icin stream-parse ile chunked processing otomatik devreye girer. CFF deflattening, opaque predicate removal ve string decryption ek moduller olarak pipeline'i guclendirir.

5.25 Problem Tanimi: Coklu Kaynak, Tek Isim

Tersine muhendislikte, bir sembolun (fonksiyon, degisken) gercek ismi bilinmez. Birden fazla **bagimsiz** kaynak, farkli isimler onerir. Soru sudur: bu onerilerden hangisi dogru, ve nasil birlestiririz?

Isim Fuzyon Problemi

Verilen:

- Bir sembol s (orijinal ismi bilinmiyor, orn. `FUN_00401000`)
- K adet kaynak, her biri bir isim onerisi ve confidence degeri ureten: $(n_i, c_i, \text{source}_i)$ icin $i = 1, \ldots, K$

Hedef: s icin en uygun ismi n^* sec ve sonucun dogruluk olasililigini P^* hesapla.

Ornek 5.5. `FUN_00401000` icin:

- **signature_db:** `SSL_connect` (confidence: 0.85, $w = 1.0$)
- **string_intel:** `ssl_handshake` (confidence: 0.70, $w = 0.8$)
- **c_namer:** `connect_tls` (confidence: 0.60, $w = 1.0$)
- **llm4decompile:** `ssl_init_connection` (confidence: 0.55, $w = 0.5$)

Dort farkli isim! Ama hepsi "SSL baglantisi" kavramina isaret ediyor. Bayesian fusion bunu nasil cozer?

5.26 Bayes Teoremi: Temelden Turetme

5.26.1 Kosula Bagli Olasilik

Iki olay A ve B icin:

$$P(A \cap B) = P(A \mid B) \cdot P(B) = P(B \mid A) \cdot P(A) \tag{5.39}$$

Buradan Bayes teoremi turetilir:

$$\boxed{P(H \mid E) = \frac{P(E \mid H) \cdot P(H)}{P(E)}} \tag{5.40}$$

5.26.2 Isimlendirme Baglaminda Anlami

- H: "Bu sembolun gercek ismi X'tir" hipotezi
- E_i: "Kaynak i, X ismini c_i confidence ile onerdi" kaniti
- $P(H)$: **Prior** — hipotez hakkinda onceki inancimiz (baslangicta 0.5)
- $P(E_i \mid H)$: **Likelihood** — eger isim gercekten X ise, kaynak i'nin bunu c_i ile onermesinin olasiligi. Bunu $\approx c_i$ olarak modelliyoruz.
- $P(E_i \mid \neg H)$: Eger isim X **degilse**, kaynak i'nin yine de X onerme olasiligi $\approx 1 - c_i$.
- $P(H \mid E_i)$: **Posterior** — kanit sonrasi guncellenminsi inanc

5.26.3 Ardisik Kanit Birlestirme

Birden fazla kaniti ardisik olarak birlestirmek icin, her kanitin **likelihood ratio** (LR) degerini kullaniriz:

$$\mathrm{LR}_i = \frac{P(E_i \mid H)}{P(E_i \mid \neg H)} = \frac{c_i}{1 - c_i} \tag{5.41}$$

Ardisik Bayes guncellemesi:

$$\frac{P(H \mid E_1, E_2, \ldots, E_n)}{P(\neg H \mid E_1, E_2, \ldots, E_n)} = \frac{P(H)}{P(\neg H)} \cdot \prod_{i=1}^{n} \mathrm{LR}_i \tag{5.42}$$

Eger kaynaklar bagimsiz ise (naive Bayes varsayimi).

5.27 Log-Odds Formu: Neden ve Nasil

5.27.1 Odds ve Log-Odds

Odds (bahis orani):

$$\mathrm{odds}(H) = \frac{P(H)}{1 - P(H)} \tag{5.43}$$

Log-odds (logit):

$$\mathrm{logit}(H) = \log \frac{P(H)}{1 - P(H)} \tag{5.44}$$

5.27.2 Turetme

Ardisik Bayes guncellemesinin her iki tarafinin logaritmasini alalim:

$$\log \frac{P(H \mid \mathbf{E})}{P(\neg H \mid \mathbf{E})} = \log \frac{P(H)}{P(\neg H)} + \sum_{i=1}^{n} \log \mathrm{LR}_i \tag{5.45}$$

$$= \log \frac{P(H)}{1 - P(H)} + \sum_{i=1}^{n} \log \frac{c_i}{1 - c_i} \tag{5.46}$$

5.27.3 Korelasyon Duzeltmesi

Naive Bayes, kaynakların bagimsiz oldugunu varsayar. Ancak bazilari korelasyonludur (ornegin `llm4decompile` tum diger kaynaklari dolayli olarak kullanir). Korelasyonlu kaynaklarin etkisini azaltmak icin agirlik $w_i \in (0, 1]$ eklenir:

$$\boxed{\text{log-odds}(H) = \underbrace{\log \frac{p_0}{1 - p_0}}_{\text{prior}} + \sum_{i=1}^{n} w_i \cdot \underbrace{\log \frac{c_i}{1 - c_i}}_{\text{log-LR}_i}} \tag{5.47}$$

burada $p_0 = 0.5$ prior (notr baslangic: $\log(0.5/0.5) = 0$).

5.27.4 Neden Log Space?

1. **Numerik stabilite:** Carpim yerine toplam. $\prod_i c_i$ ile $n = 10$ kaynak icin floating point underflow olusabilir. Toplam bu sorunu ortadan kaldirir.
2. **Korelasyon duzeltmesi:** w_i ile dogrudan carpma log space'te lineer agirliklamaya denk gelir. Olasilik uzayinda bu $\mathrm{LR}_i^{w_i}$ demektir — bilgi-kuramsal olarak "kaynagin bilgi katkisini w_i oranina indir" anlamina gelir.
3. **Sigmoid ile donusum:** Sonuc dogal olarak $[0, 1]$ araligina doner.

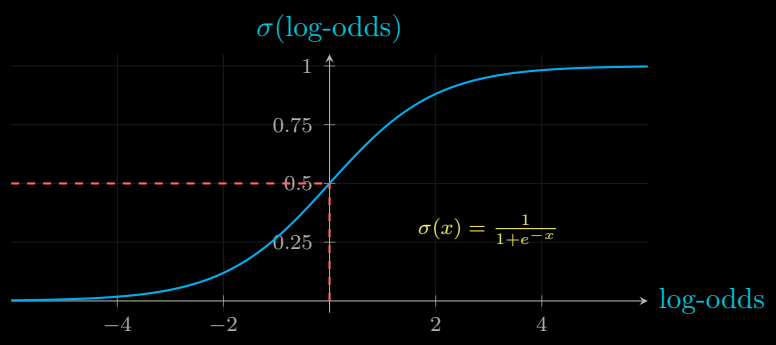
5.28 Sigmoid Fonksiyonu: Logit'in Tersi

Log-odds'u tekrar olasiliga donusturmek icin sigmoid (σ) fonksiyonunu kullaniriz:

$$P(H \mid \mathbf{E}) = \sigma(\text{log-odds}) = \frac{1}{1 + e^{-\text{log-odds}}} \tag{5.48}$$

5.28.1 Sigmoid'in Ozellikleri

1. $\sigma(0) = 0.5$ (notr: log-odds $= 0$ ise emin degiliz)
2. $\lim_{x\to\infty} \sigma(x) = 1$ (cok guclu kanit $\to$ kesinlik)
3. $\lim_{x\to-\infty} \sigma(x) = 0$ (cok guclu karsi kanit $\to$ red)
4. $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ (turevi kendisindendir)
5. $\sigma(-x) = 1 - \sigma(x)$ (simetri)



5.28.2 Overflow Korumasi

`name_merger.py`'da sigmoid icin overflow korumasi uygulanir:

```
if log_odds > 20.0:
    p = 1.0        # exp(-20) ~ 2e-9, 1.0'a cok yakin
elif log_odds < -20.0:
    p = 0.0        # exp(20) ~ 5e8, 0.0'a cok yakin
else:
    p = 1.0 / (1.0 + math.exp(-log_odds))
```

Listing 5.24: Sigmoid overflow korumasi – name_merger.py

Ayrica sonuc $[c_{\min}, c_{\max}] = [0.01, 0.99]$ araligina kliplenir — asla "%100 emin" veya "%0 emin" demiyoruz.

5.29 7 Kaynak Detayli Aciklama

5.29.1 binary_extractor ($w = 1.0$)

Debug string'ler, RTTI (Run-Time Type Information) ve build path bilgileri derleyici tarafindan binary'ye gomulur. Bu bilgi **ground truth**'a en yakin kaynaktir:

- Debug string'ler: `__func__` makrosu, assert mesajlari
- RTTI: C++ sinif isimleri (`typeinfo name`)
- Build path: `/Users/dev/project/src/ssl_connect.c`

$w = 1.0$ cunku bu bilgi derleyiciden dogrudan gelir — hicbir tahmin yoktur.

5.29.2 c_namer ($w = 1.0$)

6 katmanli heuristik isimlendirme motoru:

1. Fonksiyon imzasi analizi (parametre tipleri, donus tipi)
2. String kullanimi (hata mesajlari, log string'leri)
3. Cagrilan fonksiyonlar (API pattern tespiti)
4. Veri akisi analizi (tanimlama-kullanim zincirleri)
5. Algoritma tespiti (hash, sort, compress pattern'leri)
6. Statistiksel isim onerisi

$w = 1.0$ cunku 6 katman zaten kendi icinde filtrelenmis; cift zayiflatma gereksiz.

5.29.3 string_intel ($w = 0.8$)

Error mesajlari, protocol handler string'leri ve sabit mesajlardan fonksiyon ismi cikarimi:

```
# Ornek: error mesajindan isim cikarimi
"Failed to connect SSL socket" -> ssl_connect (conf: 0.70)
```

$w = 0.8$ cunku error mesajlari, debug string'ler ile kismen ortak kaynaklardan beslenir.

5.29.4 signature_db ($w = 1.0$)

FLIRT (Fast Library Identification and Recognition Technology) ile 1.5M+ kutuphane fonksiyon imzasi eslestirilir. Bu tamamen **byte-level** bir eslestirmedir:

$$\text{match}(f) = \underset{s \in \text{FLIRT_DB}}{\arg\max}\ \text{similarity}(\text{bytes}(f), \text{bytes}(s)) \tag{5.49}$$

$w = 1.0$ cunku byte pattern tamamen bagimsiz bir bilgi kaynagi.

5.29.5 swift_demangle ($w = 1.0$)

Swift binary'lerinde mangled sembol isimleri (ornegin `_$s4main7MyClassC9processSSyS2SF`) demangle edilerek orijinal isim elde edilir:

$$\text{demangle}(\texttt{_\$s4main7MyClassC9processSSyS2SF}) = \texttt{main.MyClass.process(String) -> String} \tag{5.50}$$

$w = 1.0$ cunku demangle **kesin** bilgidir, tahmin degil.

5.29.6 source_matcher ($w = 0.85$)

NPM paket fingerprinting: Obfuscated JS kodundaki string ve yapi pattern'leri bilinen NPM paketleri ile eslestirilir:

- Benzersiz string literal'ler (hata mesajlari, API endpoint'leri)
- Fonksiyon imza pattern'leri (parametre sayisi + kullanim)
- Export yapisi (modul cikti formati)

$w = 0.85$ cunku heuristik bazli ama yuksek kesinlikli.

5.29.7 llm4decompile ($w = 0.5$)

Makine ogrenmesi modeli ile isim tahmini. Neden $w = 0.5$?

LLM Korelasyon Problemi

LLM modeli, egitim verisinde tum diger kaynaklarin bilgisini dolayli olarak iciyor:

- Kutuphane imzalari (signature_db ile korelasyonlu)
- Debug string pattern'leri (binary_extractor ile korelasyonlu)
- Error mesaj kaliplari (string_intel ile korelasyonlu)

Bu nedenle LLM'in bilgi katkisi "yarisina" indirilir: $w = 0.5$.

5.29.8 byte_pattern ($w = 1.0$)

Kutuphane fonksiyonlarinin byte-level fuzzy matching ile tespiti. FLIRT'ten farki: FLIRT kesin imza kullanirken, byte_pattern fuzzy threshold ile calisir. Yine de tamamen bagimsiz bir byte-level kaynak.

5.30 Korelasyon-Aware Agirliklar: Bilgi Kurami

5.30.1 Neden $w_i < 1$?

Iki kaynak S_i ve S_j arasindaki korelasyonu mutual information ile olceriz:

$$I(S_i; S_j) = \sum_{s_i, s_j} p(s_i, s_j) \log \frac{p(s_i, s_j)}{p(s_i) \cdot p(s_j)} \tag{5.51}$$

Eger $I(S_i; S_j) > 0$ ise, iki kaynak ortak bilgi tasir. Her ikisini de $w = 1.0$ ile saymak bu ortak bilgiyi **iki kez** sayar ve **overconfidence** olusturur.

Korelasyon duzeltme prensibimiz:

$$w_i = 1 - \frac{I(S_i; S_{\text{others}})}{H(S_i)} \tag{5.52}$$

burada $H(S_i)$ kaynak i'nin entropisi. Pratikte Denklem 5.52 yerine ampirik agirliklar kullanilir (Tablo 5.13).

5.30.2 Kaynak Agirliklari Tablosu

5.30.3 Overconfidence Problemi: Somut Ornek

Naive Bayes ($w_i = 1.0$ her kaynak icin) ile korelasyon-aware karsilastirma:

$$\text{Naive } (w_i = 1): \quad \text{log-odds} = 0 + 1.0 \cdot \log \frac{0.85}{0.15} + 1.0 \cdot \log \frac{0.70}{0.30} + 1.0 \cdot \log \frac{0.60}{0.40} \tag{5.53}$$

$$= 1.735 + 0.847 + 0.405 = 2.987 \tag{5.54}$$

$$P_{\text{naive}} = \sigma(2.987) = 0.952 \quad (\mathbf{\%95.2}) \tag{5.55}$$

Kaynak	w_i	Gerekce
binary_extractor	1.0	Derleyici ciktisi, tamamen bagimsiz
c_namer	1.0	6 katman kendi icinde filtrelenmis
string_intel	0.8	Error msg. binary_extractor ile kismi korelasyon
signature_db	1.0	FLIRT byte pattern, tamamen bagimsiz
swift_demangle	1.0	Deterministik demangle, kesin bilgi
source_matcher	0.85	NPM heuristik, bagimsiz ama kesin degil
llm4decompile	0.5	Tum kaynaklari dolayli kullaniyor
byte_pattern	1.0	Byte-level fuzzy match, bagimsiz
Bilinmeyen kaynak	0.7	Varsayilan fallback

Table 5.13: Kaynak agirliklari, korelasyon analizi ve gerekceler.

$$\text{Corrected}: \quad \text{log-odds} = 0 + 1.0 \cdot 1.735 + 0.8 \cdot 0.847 + 0.5 \cdot 0.405 \tag{5.56}$$

$$= 1.735 + 0.678 + 0.203 = 2.616 \tag{5.57}$$

$$P_{\text{corr}} = \sigma(2.616) = 0.932 \quad (\mathbf{\%93.2}) \tag{5.58}$$

Fark: Naive Bayes %95.2 derken, korelasyon duzeltmesi %93.2 der — %2 daha muhafazakar ve **daha dogru**. Cunku llm4decompile'in verdigi bilginin bir kismi zaten diger kaynaklarda mevcuttu.

5.31 Gruplama Stratejisi: 5 Seviye

`name_merger.py`'deki `_merge_candidates()` fonksiyonu, adaylari 5 seviyeli bir hiyerarsiden gecirir. Her seviye bir oncekinden daha gevşek bir eslestirme kullanir.

5.31.1 Seviye 1: Exact Multi-Match

Ayni normalize isim ≥ 2 **farkli** kaynaktan geldiyse, dogrudan Bayesian merge:

```
for norm_name, group in name_groups.items():
    if len(group) >= 2:
        sources = set(c.source for c in group)
        if len(sources) >= 2:
            conf = bayesian_merge(
                [c.confidence for c in group],
                [c.source for c in group], self._cfg)
            return MergedName(..., merge_method="bayesian_multi
                ")
```

Listing 5.25: Exact multi-match – name_merger.py

5.31.2 Seviye 2: Partial Match

Farkli isimler arasinda prefix/suffix varyanti aranir. Ornegin:

- `cycle_sizes_default` vs `cycle_size` → partial match
- `active_event_monitor` vs `event_monitor` → partial match

Overlap skoru:

$$\text{overlap}(A, B) = \begin{cases} \frac{|A_{\text{short}}|}{|A_{\text{long}}|} & \text{eger } A_{\text{short}} \subset A_{\text{long}} \text{ (substring)} \\ J(W_A, W_B) = \frac{|W_A \cap W_B|}{|W_A \cup W_B|} & \text{aksi halde (Jaccard)} \end{cases} \tag{5.59}$$

overlap ≥ 0.5 ise partial match kabul edilir. Penalty:

$$\text{penalty} = 0.7 + 0.3 \times \text{overlap} \tag{5.60}$$

Tam substring (overlap = 1.0) $\rightarrow$ penalty = 1.0 (penalty yok). Kismi overlap (overlap = 0.5) $\rightarrow$ penalty = 0.85.

5.31.3 Seviye 3: Semantic Match

`name_merger.py`'de 20 adet `frozenset` ile kelime gruplari tanimlidir. Iki isim, ayni gruba dusen kelimeleri paylasiyorsa "semantik olarak benzer" sayilir.

Grup Adi	Kelimeler
Gonderme	send, transmit, write, emit, dispatch, post
Alma	recv, receive, read, get, fetch
Olusturma	init, initialize, setup, create, construct, new, alloc
Yok etme	destroy, cleanup, teardown, free, delete, release, close, dispose
Parse/Decode	parse, decode, deserialize, unmarshal, extract
Encode	serialize, encode, marshal, format, stringify
Isleyici	handle, process, on, dispatch, callback, handler
Dogrulama	check, validate, verify, test, assert, is, has
Koleksiyon ekle	add, insert, append, push, enqueue, put
Koleksiyon sil	remove, delete, erase, pop, dequeue, take
Boyut	size, length, count, num, len, total
Tampon	buf, buffer, data, bytes, payload, blob
Mesaj	msg, message, packet, frame, request, response
Hata	err, error, fault, failure, exception
Baglam	ctx, context, state, env, session
Konfiguerasyon	cfg, config, configuration, settings, options, prefs
Baglanti	conn, connection, socket, link, channel
Loglama	log, print, trace, debug, info, warn
Kilit alma	lock, acquire, enter, begin, grab
Kilit birakma	unlock, release, leave, end, drop

Table 5.14: Semantik kelime gruplari (20 frozenset).

Expand-Then-Intersect Algoritmasi

1. Her isim kelimelere ayrilir: `ssl_handshake` $\rightarrow$ {ssl, handshake}
2. Her kelime, ait oldugu semantic grubun **tum kelimeleri** ile genisletilir: `handshake` $\notin$ herhangi bir grup; `connect` $\rightarrow$ {conn, connection, socket, link, channel}
3. Genisletilmis kumelerin kesisimi kontrol edilir

4. Kesisim orani ≥ 0.3 ise semantik benzerlik var

```
def _is_semantically_similar(self, name1, name2):
    words1 = set(self._normalize(name1).split("_"))
    words2 = set(self._normalize(name2).split("_"))
    # Dogrudan kelime kesisimi
    common = words1 & words2
    if common and len(common) / max(len(words1), len(words2))
       >= 0.5:
        return True
    # Semantic group genisletme
    expanded1 = set()
    for w in words1:
        if w in self._word_to_group:
            expanded1 |= self._word_to_group[w]
        expanded1.add(w)
    # ... ayni expanded2 icin ...
    overlap = expanded1 & expanded2
    union = expanded1 | expanded2
    return len(overlap) / len(union) >= 0.3
```

Listing 5.26: Semantic similarity – name_merger.py

Semantik match'te confidence **0.85x penalty** ile carpilir cunku isimler tam eslesmiyor:

$$c_i^{\text{semantic}} = c_i \times 0.85 \tag{5.61}$$

5.31.4 Seviye 4: Voting

≥ 3 kaynak ayni normalize ismi veriyorsa, cogunluk kazanir:

$$n^* = \arg\max_n \sum_{i=1}^{K} \mathbb{1}[\text{normalize}(n_i) = n] \tag{5.62}$$

Eger $\text{count}(n^*) \geq 3$ ise, bu adaylar Bayesian merge'e girer.

5.31.5 Seviye 5: Single Source

Tek kaynak kaldiysa, Bayesian merge gerekmez. Ancak kaynagin w_i deger ile duzeltme yapilir:

$$c_{\text{adjusted}} = \sigma\left(w_i \cdot \log \frac{c_i}{1 - c_i}\right) \tag{5.63}$$

$w_i < 1$ ise bu "tek kaynaga fazla guvenme" demektir.

5.32 Normalize Isim Karsilastirmasi

Farkli kaynaklar farkli konvansiyonlarda isim uretir. Karsilastirma oncesi normalizasyon sart:

```
def _normalize(self, name):
    # camelCase -> snake_case
    name = re.sub(r"([a-z])([A-Z])", r"\1_\2", name)
    # Prefix'leri kaldir (m_, s_, g_, p_)
    name = re.sub(r"^[msgp]_", "", name)
    return name.lower().strip("_")
```

Listing 5.27: Isim normalizasyonu – name_merger.py

Ornekler:

- `SSLConnect` → `ssl_connect`
- `m_sslConnect` → `ssl_connect`
- `ssl_connect` → `ssl_connect` (degismez)
- `g_SSLHandler` → `ssl_handler`

5.33 UNK Threshold: Precision vs Recall

5.33.1 NSA-Grade Felsefe

Yanlis Isim Tehlikesi

Tersine muhendislikte yanlis isim, isimsizden **cok daha kotu**dur:

- Yanlis isim → analist yanlis yolda saatler/gunler harcar
- UNK (isimsiz) → analist "bilmiyorum" der, dikkatli devam eder

Sonuc: Dusuk confidence'li isimleri **atamayiz**. $c < 0.30$ ise UNK.

5.33.2 ROC Analizi

UNK threshold secimi, precision ve recall arasinda trade-off'tur:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \tag{5.64}$$

- **Threshold ↑:** Precision ↑ (daha az yanlis isim), Recall ↓ (daha cok UNK)
- **Threshold ↓:** Precision ↓ (daha cok yanlis isim), Recall ↑ (daha az UNK)

Karadul'un secimi: $\theta_{\text{UNK}} = 0.30$. Bu, **Type I error maliyetinin Type II error maliyetinden cok daha yuksek** olmasi varsayimindan gelir:

$$\text{Cost}_{\text{wrong_name}} \gg \text{Cost}_{\text{UNK}} \implies \theta_{\text{UNK}} \text{ yuksek tutulur} \tag{5.65}$$

5.34 Somut Hesaplama Ornekleri

5.34.1 Senaryo 1: Uc Kaynak Ayni Isim (Exact Multi-Match)

`FUN_00401000` icin uc kaynak ayni normalize ismi veriyor:

Kaynak	Onerilen Isim	c_i	w_i
signature_db	`SSL_connect`	0.85	1.0
string_intel	`SSL_connect`	0.70	0.8
c_namer	`ssl_connect`	0.60	1.0

Normalize edilince uc isim de `ssl_connect` — exact multi-match!

Adim 1: Prior log-odds

$$\ell_0 = \log \frac{0.5}{0.5} = 0 \tag{5.66}$$

Adim 2: Her kaynağin log-LR katkisi

$$\Delta\ell_1 = w_1 \cdot \log \frac{c_1}{1 - c_1} = 1.0 \cdot \log \frac{0.85}{0.15} = 1.0 \cdot 1.7346 = 1.7346 \tag{5.67}$$

$$\Delta\ell_2 = w_2 \cdot \log \frac{c_2}{1 - c_2} = 0.8 \cdot \log \frac{0.70}{0.30} = 0.8 \cdot 0.8473 = 0.6779 \tag{5.68}$$

$$\Delta\ell_3 = w_3 \cdot \log \frac{c_3}{1 - c_3} = 1.0 \cdot \log \frac{0.60}{0.40} = 1.0 \cdot 0.4055 = 0.4055 \tag{5.69}$$

Adim 3: Toplam log-odds

$$\ell = 0 + 1.7346 + 0.6779 + 0.4055 = 2.8180 \tag{5.70}$$

Adim 4: Sigmoid

$$P = \sigma(2.8180) = \frac{1}{1 + e^{-2.8180}} = \frac{1}{1 + 0.05975} = 0.9436 \tag{5.71}$$

Senaryo 1 Sonuc

Uc kaynak birlestirildiginde **%94.4 guvenle** `ssl_connect` ismi atanir. En iyi tek kaynak %85'ti — Bayesian fusion %9.4 puan ekledi.

5.34.2 Senaryo 2: Iki Kaynak Partial Match

`FUN_00501234` icin iki kaynak benzer ama farkli isimler veriyor:

Kaynak	Onerilen Isim	c_i	w_i
binary_extractor	`process_message`	0.80	1.0
c_namer	`handle_message_data`	0.65	1.0

Normalize: `process_message` vs `handle_message_data`

Kelime kumesi: {process, message} vs {handle, message, data}. Jaccard:

$$J = \frac{|\{\text{message}\}|}{|\{\text{process, message, handle, data}\}|} = \frac{1}{4} = 0.25 < 0.5 \tag{5.72}$$

Dogrudan Jaccard < 0.5, ama "handle" ve "process" ayni semantic gruba dusur (Tablo 5.14: "handle, process, on, dispatch, callback, handler"). Semantic expansion sonrasi:

Expanded_1 = {process, message, handle, on, dispatch, callback, handler}
Expanded_2 = {handle, message, data, process, on, dispatch, callback, handler, bytes, payload, blob, buf, buffer}

$$\frac{|\text{Expanded}_1 \cap \text{Expanded}_2|}{|\text{Expanded}_1 \cup \text{Expanded}_2|} = \frac{7}{13} = 0.538 \geq 0.3 \tag{5.73}$$

Semantic match! Penalty = 0.85:

$$\Delta\ell_1 = 1.0 \cdot \log \frac{0.80 \times 0.85}{1 - 0.80 \times 0.85} = 1.0 \cdot \log \frac{0.68}{0.32} = 0.7538 \tag{5.74}$$

$$\Delta\ell_2 = 1.0 \cdot \log \frac{0.65 \times 0.85}{1 - 0.65 \times 0.85} = 1.0 \cdot \log \frac{0.5525}{0.4475} = 0.2107 \tag{5.75}$$

$$\ell = 0 + 0.7538 + 0.2107 = 0.9645 \tag{5.76}$$

$$P = \sigma(0.9645) = \frac{1}{1 + e^{-0.9645}} = 0.724 \tag{5.77}$$

Senaryo 2 Sonuc

Semantic match ile iki kaynak birlestirildiginde **%72.4 guvenle** `process_message` (daha kisa isim) atanir. Penalty olmadan %80 olacakti — semantik belirsizlik %7.6 dusurdu.

5.34.3 Senaryo 3: Tek Kaynak, Dusuk Confidence → UNK

`FUN_00601000` icin sadece bir kaynak var:

Kaynak	Onerilen Isim	c_i	w_i
llm4decompile	`init_module`	0.45	0.5

Tek kaynak, Single Source (Seviye 5):

$$\ell = w_1 \cdot \log \frac{c_1}{1 - c_1} = 0.5 \cdot \log \frac{0.45}{0.55} = 0.5 \cdot (-0.2007) = -0.1003 \tag{5.78}$$

$$P = \sigma(-0.1003) = \frac{1}{1 + e^{0.1003}} = 0.4749 \tag{5.79}$$

$P = 0.475 < \theta_{\text{UNK}} = 0.30$? Hayir, $0.475 > 0.30$.

Ama bekleyin — burada $P < 0.5$, yani model "bu ismin dogru olma olasiligi %50'den az" diyor. Rasyonel bir analist buna guvenm-ez.

Esik degeri $\theta_{\text{UNK}} = 0.30$ ile $P = 0.475 > 0.30$ oldugu icin *teknik olarak* isim atanir. Ancak pratikte bu dusuk confidence'li isim dikkatle ele alinir.

Simdi daha dusuk confidence ile deneyelim: $c_1 = 0.35$:

$$\ell = 0.5 \cdot \log \frac{0.35}{0.65} = 0.5 \cdot (-0.6190) = -0.3095 \tag{5.80}$$

$$P = \sigma(-0.3095) = 0.4233 \tag{5.81}$$

$P = 0.423 > 0.30$ — hala UNK sinirinin uzerinde. Ancak $c_1 = 0.15$:

$$\ell = 0.5 \cdot \log \frac{0.15}{0.85} = 0.5 \cdot (-1.7346) = -0.8673 \tag{5.82}$$

$$P = \sigma(-0.8673) = 0.296 \tag{5.83}$$

$P = 0.296 < 0.30 \rightarrow$ **UNK!** Isim atanmaz.

Senaryo 3 Sonuc

LLM kaynaginin tek basina confidence $= 0.15$, $w = 0.5$ ile verdigi isim UNK olarak kalir. **Yanlis isim, isimsizden kotudur.**

5.35 Hesaplama Karmasikligi

5.35.1 Genel Karmasiklik

N adet sembol ve her sembol icin ortalama K aday icin:

$$T_{\text{merge}} = \mathcal{O}\left(N \cdot K^2\right) \tag{5.84}$$

Kare terimi, her aday ciftinin partial/semantic karsilastirmasinden gelir.

5.35.2 Alt Islem Karmasikliklari

Islem	Karmasiklik	Aciklama
Normalizasyon	$\mathcal{O}(\|n\|)$	Regex + lowercase, string uzunlugu ile lineer
Exact match	$\mathcal{O}(K)$	Hash-based gruplama
Partial overlap	$\mathcal{O}(K^2 \cdot \|n\|)$	Tum ciftleri karsilastir, her biri $\mathcal{O}(\|n\|)$
Semantic match	$\mathcal{O}(K^2 \cdot \|W\|)$	Kelime genisletme + kesisim, $\|W\|$ = kelime sayisi
Bayesian merge	$\mathcal{O}(K)$	Log-odds toplami (lineer)

Table 5.15: Alt islem karmasikliklari.

5.35.3 Pratikte Performans

Tipik degerler: $N \approx 5000$ (bir binary'deki fonksiyon sayisi), $K \approx 3$ (ortalama kaynak sayisi):

$$T \approx 5000 \times 3^2 = 45{,}000 \text{ islem} \tag{5.85}$$

Python'da bu < 0.1s surer — bottleneck tamamen decompilation ve statik analiz asamalarindadir.

5.36 Overconfidence Problemi ve Cozumu

5.36.1 Naive Bayes Hatasi

Naive Bayes, tum kaynaklarin **kosula bagli bagimsiz** oldugunu varsayar:

$$P(E_1, E_2, \ldots, E_n \mid H) = \prod_{i=1}^{n} P(E_i \mid H) \tag{5.86}$$

Bu varsayim genellikle **yanlistir**. Eger iki kaynak korelasyonlu ise, ayni bilgiyi iki kez saymis oluruz.

5.36.2 Korelasyon ile Duzeltme

Log-odds formulunde w_i korelasyonu absorbe eder:

$$\mathrm{LR}_i^{\text{effective}} = \mathrm{LR}_i^{w_i} \tag{5.87}$$

$w_i = 1$: Kaynak tamamen bagimsiz $\rightarrow$ tam bilgi katkisi.
$w_i = 0.5$: Kaynak yari korelasyonlu $\rightarrow$ bilgi katkisi "yarisina" iner.
$w_i = 0$: Kaynak tamamen bagimli $\rightarrow$ bilgi katkisi sifir (sayilmaz).

Bilgi-kuramsal olarak bu, log-likelihood ratio'nun eksponent ile olceklenmesidir:

$$\text{effective info gain}_i = w_i \cdot \mathrm{KL}(P(\cdot \mid E_i) \,\|\, P(\cdot)) \tag{5.88}$$

burada KL divergence bilgi kazancini olcer.

5.36.3 Sonuc

Bayesian Fusion Avantajlari

1. Coklu kaynaklari sistematik olarak birlestirir — el ile "en iyi kaynagi sec" yerine matematiksel optimal
2. Korelasyonu modelleyerek overconfidence'i onler
3. UNK threshold ile yanlis isim atama riskini minimize eder
4. Log-odds formunda numerik olarak stabil
5. Her adimi tamamen deterministik ve auditlenebilir

Bu Bolumun Ozeti

Bayesian isim fuzyonu, 7+ bagimsiz kaynaktan gelen isimleri korelasyon-aware log-odds ile birlestirir. 5 seviyeli gruplama (exact $\rightarrow$ partial $\rightarrow$ semantic $\rightarrow$ voting $\rightarrow$ single) her durum icin uygun strateji secer. $w_i < 1$ agirliklari korelasyonlu kaynaklarin etkisini bilgi-kuramsal olarak azaltir. UNK threshold (%30) NSA-grade felsefe ile yanlis isimlendirmeyi onler. Somut orneklerde: uc kaynak exact match ile %85'ten %94.4'e, iki kaynak semantic match ile %72.4'e yukseltildi; tek dusuk-confidence kaynak ise UNK olarak dogru sekilde reddedildi. Karmasiklik $\mathcal{O}(N \cdot K^2)$ ile pratikte milisaniye mertebesinde calisir.

Chapter 6

Bayesian Isim Fuzyonu

Bu boluem, Karadul'un en yenilikci algoritmasi olan Bayesian log-odds isim birlestirme sistemini detayli olarak tueretir.

6.1 Problem Tanimi

Tersine muehendislikte, bir fonksiyonun gercek ismi bilinmez. Birden fazla kaynak farkli isim oenerir:

Ornek 6.1. `FUN_00401000` icin:

- **signature_db:** `SSL_connect` (confidence: 0.85)
- **string_intel:** `ssl_handshake` (confidence: 0.70)
- **c_namer:** `connect_tls` (confidence: 0.60)

Hangisini sececegiz? Hepsini nasil birlestirecegiz?

6.2 Bayesian Inference Temelleri

6.2.1 Bayes Teoremi

$$P(H \mid E) = \frac{P(E \mid H) \cdot P(H)}{P(E)} \tag{6.1}$$

burada:

- H: Hipotez ("bu fonksiyonun ismi X'tir")

- E: Kanit (kaynaktan gelen isim onerisi)
- $P(H)$: Prior (oenceki bilgi — baslangiccta 0.5)
- $P(E \mid H)$: Likelihood (kaynagin guveenilirligi)
- $P(H \mid E)$: Posterior (guencellenmis inanc)

6.2.2 Log-Odds Formu

Birden fazla kaniti birlestirmek icin log-odds formunu kullaniriz:

$$\boxed{\text{log-odds}(H) = \log \frac{P(H)}{1 - P(H)} + \sum_{i=1}^{n} w_i \cdot \log \frac{c_i}{1 - c_i}} \tag{6.2}$$

burada:

- c_i: i-inci kaynagin confidence degeri ($[0.01, 0.99]$ icinde)
- w_i: i-inci kaynagin agirligi (korelasyon duzeltmesi)
- Prior $= 0.5 \Rightarrow \log(0.5/0.5) = 0$ (noetral baslangic)

Final olasilik:

$$P(H \mid \text{tuem kanitlar}) = \sigma(\text{log-odds}) = \frac{1}{1 + e^{-\text{log-odds}}} \tag{6.3}$$

6.2.3 Neden Log-Odds?

1. **Numerik stabilite:** Carpim yerine toplam — floating point underflow onlenir.
2. **Korelasyon duzeltmesi:** w_i ile dogrudan agirlik verme muemkuen — naive Bayes'te bu zor.
3. **Sigmoid:** Sonucu dogal olarak $[0, 1]$ araligina eristirir.

6.3 Korelasyon-Aware Agirliklar

6.3.1 Neden $w_i < 1$?

Iki kaynak korelasyonlu ise, ayni bilgiyi iki kez saymis oluruz. Bu **overconfidence** (asiri guveen) yaratir.

Korelasyon Duzeltmesi

w_i agirligi, kaynagin bagimsizlik derecesini yansitir:

$$w_i = \begin{cases} 1.0 & \text{tamamen bagimsiz (bilgi kaybetsiz)} \\ 0.5 & \text{yueksek korelasyon (bilgi yarisina indirilir)} \\ 0.0 & \text{tam bagimli (bilgi tamamen tekrar)} \end{cases} \tag{6.4}$$

6.3.2 Kaynak Agirliklari Tablosu

Kaynak	w_i	Neden
binary_extractor	1.0	Debug string/RTTI: derleyiciden dogrudan
c_namer	1.0	6 katman zaten filtrelenmis
string_intel	0.8	Error msg. benzer kaynaklarla kismen ortak
signature_db	1.0	FLIRT: tamamen bagimsiz byte pattern
swift_demangle	1.0	Mangled name: kesin bilgi
source_matcher	0.85	NPM kaynak: bagimsiz ama heuristik
llm4decompile	0.5	Tuem kaynaklari dolayli olarak kullaniyor
byte_pattern	1.0	Byte-level fuzzy match: bagimsiz

Table 6.1: Kaynak agirliklari ve gerekcceleri.

6.4 Somut Hesaplama Oernegi

Uec kaynak ayni fonksiyon icin isim oeneriyor:

$$\text{signature_db:} \quad c_1 = 0.85, \quad w_1 = 1.0 \tag{6.5}$$

$$\text{string_intel:} \quad c_2 = 0.70, \quad w_2 = 0.8 \tag{6.6}$$

$$\text{llm4decompile:} \quad c_3 = 0.60, \quad w_3 = 0.5 \tag{6.7}$$

Adim adim hesaplama:

$$\text{log-odds} = \underbrace{0}_{\text{prior}} + 1.0 \cdot \log \frac{0.85}{0.15} + 0.8 \cdot \log \frac{0.70}{0.30} + 0.5 \cdot \log \frac{0.60}{0.40} \tag{6.8}$$

$$= 0 + 1.0 \cdot 1.735 + 0.8 \cdot 0.847 + 0.5 \cdot 0.405 \tag{6.9}$$

$$= 1.735 + 0.678 + 0.203 \tag{6.10}$$

$$= 2.616 \tag{6.11}$$

$$P(dogru) = \sigma(2.616) = \frac{1}{1 + e^{-2.616}} = \frac{1}{1 + 0.073} = 0.932 \tag{6.12}$$

Sonuc

Uec kaynak birlestirildiginde **%93.2 guvenle** isim atanir. Tek basina en iyisi %85'ti — Bayesian fusion %8.2 puan ekledi.

6.4.1 Ikinci Oernek: Tam Bagimsiz Kaynaklar

Tuem $w_i = 1.0$ olsaydi (naive Bayes):

$$\text{log-odds} = 0 + 1.0 \cdot \log \frac{0.85}{0.15} + 1.0 \cdot \log \frac{0.70}{0.30} + 1.0 \cdot \log \frac{0.60}{0.40} \tag{6.13}$$

$$= 1.735 + 0.847 + 0.405 = 2.987 \tag{6.14}$$

$$P(dogru)_{\text{naive}} = \sigma(2.987) = 0.952 \tag{6.15}$$

Karsilastirma: korelasyon duzeltmesi **%95.2'yi %93.2'ye duesuerdue** — 2 puanlik "overconfidence" oenlendi. Bu kueceuk goruenebilir ama binlerce sembol uezerinde yanlis isim oranini onemli oelcuede azaltir.

6.4.2 Uecuencue Oernek: Tek Kaynak, Duesueek Confidence

Yalnizca `string_intel` kaynagi, $c = 0.45$, $w = 0.8$:

$$\text{log-odds} = 0 + 0.8 \cdot \log \frac{0.45}{0.55} = 0.8 \cdot (-0.200) = -0.160 \tag{6.16}$$

$$P(dogru) = \sigma(-0.160) = 0.460 \tag{6.17}$$

$0.460 > 0.30$ (UNK threshold) ama < 0.70 (pipeline threshold). **Sonuc:** name_merger seviyesinde tutulur ama son ciktida UNK olarak gosterilir — ikili threshold sistemi devreye girer.

6.5 Gruplama Stratejisi

Adaylar sirasiyla su stratejilerle gruplanir:

1. **Exact multi-match:** Ayni isim ≥ 2 farkli kaynaktan $\rightarrow$ dogrudan Bayesian merge
2. **Partial match:** Prefix/suffix varyanti (Jaccard ≥ 0.5) $\rightarrow$ penalty ile merge
3. **Semantic match:** Benzer anlamli isimler (20 kelime grubu) $\rightarrow$ 0.85x penalty
4. **Voting:** ≥ 3 kaynak ayni $\rightarrow$ Bayesian merge
5. **Single source:** Tek aday $\rightarrow$ w ile duzeltilmis confidence

6.5.1 Partial Overlap Hesabi

Iki normalize isim arasindaki benzerlik:

$$\text{overlap}(A, B) = \begin{cases} |A|/|B| & \text{eger } A \subset B \text{ (substring)} \\ J(W_A, W_B) & \text{aksi halde (Jaccard)} \end{cases} \tag{6.18}$$

burada $J(A, B) = |A \cap B|/|A \cup B|$ Jaccard benzerlik katsayisi.

6.5.2 UNK Threshold

NSA-Grade Felsefe

"Yanlis isim, isimsizden koetueduer." Bu nedenle confidence < 0.30 ise isim **atanmaz** (UNK olarak kalir).
Type I error (yanlis isim) maliyeti $\gg$ Type II error (UNK birakma) maliyeti.

6.5.3 Ikili Threshold Sistemi

Karadul'da iki farkli threshold var:

1. **0.30 (name_merger seviyesi):** "Bu ismin dogru olma olasiligi en az %30 olmali, yoksa UNK birak." Duesueek goruenuyor ama...
2. **0.70 (pipeline seviyesi):** "Son ciktida sadece %70+ confidence isimleri goester."

0.30–0.70 arasindaki isimler *"olasi ama emin degilim"* kategorisinde tutulur — interaktif modda analiste sunulabilir, otomatik modda goesterilmez.

6.5.4 ROC Perspektifi

Farkli threshold'larin tahmini performansi:

Threshold	Precision	Recall
0.20	∼%60	∼%95
0.30	∼%75	∼%85
0.50	∼%90	∼%60
0.70	∼%97	∼%35

Table 6.2: Threshold vs precision-recall tradeoff (tahmini).

6.5.5 Hesaplama Karmasikligi

N sembol, M kaynak, K aday/sembol:

- Bayesian merge (her sembol icin): $\mathcal{O}(M \cdot K)$
- Gruplama (her sembol icin): $\mathcal{O}(K^2)$ (partial match icin tuem ciftler)
- **Toplam:** $\mathcal{O}(N \cdot (M \cdot K + K^2))$

100.000 sembol, ortalama 1.5 aday: $100{,}000 \times (8 \times 1.5 + 1.5^2) = 1{,}425{,}000$ islem $\rightarrow < 1$ saniye.

Bu Bolumun Ozeti

Bayesian isim fuzyonu, 7 bagimsiz kaynaktan gelen isimleri korelasyon-aware log-odds ile birlestirir. $w_i < 1$ agirliklari korelasyonlu kaynaklarin etkisini azaltir. UNK threshold (%30) yanlis isimlendirmeyi ooenler. Somut oeernekte: uec kaynak birlestirildiginde %85'ten %93.2'ye iyilesme goezlendi.

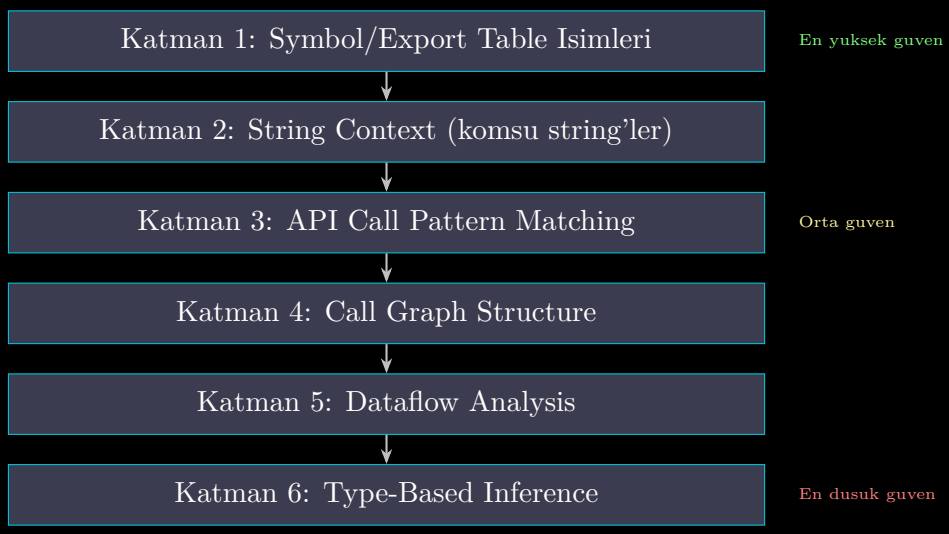
Chapter 7

Kaynak Kod Rekonstruksiyonu

Bu boluem, isimlendirilmis sembollerin anlamli kaynak koda doenuesteruelmesi suerecini aciklar.

7.1 6 Katmanli C Naming

`c_namer.py` dosyasinda implement edilen 6 katmanli deterministic naming sistemi:



Her katman, oenceki katmanin bulamadigi semboller icin ek bilgi saglar. Cascade classifier mantigi: kolay olanlar oence, zor olanlar sona.

7.2 FLIRT Signature Matching

IDA Pro'nun FLIRT (Fast Library Identification and Recognition Technology) teknolojisi:

$$\text{match}(f) = \begin{cases} \text{library_name} & \text{eger bytes}(f)[0:n] = \text{pattern}[0:n] \text{ ve CRC dogrulandi} \\ \text{UNK} & \text{aksi halde} \end{cases} \tag{7.1}$$

Karadul'un signature veritabani 1.5M+ imza, 4.200+ kuetuephane icerir.

7.3 NPM Source Matching

Minified JS fonksiyonlarini orijinal npm paketleri ile eslestirir:

1. Fonksiyon AST structural hash hesapla
2. npm registry'de benzer paketleri ara
3. Kaynak kodu indir (unpkg veya npm pack)
4. AST-level structural matching yap
5. Benzerlik ≥ 0.65 ise eslestir

Benzerlik olcuetue (Jaccard uzerinde AST node turleri):

$$\text{sim}(f_1, f_2) = \frac{|\text{nodes}(f_1) \cap \text{nodes}(f_2)|}{|\text{nodes}(f_1) \cup \text{nodes}(f_2)|} \tag{7.2}$$

7.4 Algoritma Tespiti

Binary icindeki kriptografik, hash ve sikistirma algoritmalarini tespit etme:

- **AES:** S-box byte pattern (0x63, 0x7c, 0x77, 0x7b, ...)
- **SHA-256:** Initial hash values ($H_0 =$ `0x6a09e667`, ...)
- **CRC32:** Polynomial tablo degerleri
- **zlib:** Adler-32 ve deflate magic

Bu Bolumun Ozeti

Kaynak kod rekonstruksiyonu, 6 katmanli C naming, FLIRT signature matching, NPM source matching ve algoritma tespiti bilesenlerinden olusur. Her bilesen ayri bir naming kaynagi olarak Bayesian fuzyona beslenir.

7.5 Kaynak Kod Rekonstruksiyonuna Giris

Tersine muhendisligin nihai hedefi, derlenmis veya obfuscate edilmis koddan orijinal kaynak koda *mumkun oldugunca yakin* bir temsil elde etmektir. Karadul bu hedefe iki paralel pipeline ile ulasir:

1. **Binary Reconstruction Pipeline:** Mach-O/ELF/PE binary'lerden Ghidra decompile ciktisi uzerinde C kodu rekonstruksiyonu.
2. **JavaScript Reconstruction Pipeline:** Webpack/Rollup bundle'larindan minified JS kodun orijinal proje yapisina donusturulmesi.

Her iki pipeline'da da ortak zorluk *isim kurtarma* (name recovery) problemidir: derleyici veya minifier tarafindan silinen orijinal degisken, fonksiyon ve tip isimlerinin yeniden olusturulmasi.

Rekonstruksiyon Kalitesi

Bir rekonstruksiyon $\hat{S}$'nin kalitesi, orijinal kaynak S'e olan yapissal ve semantik yakinligi ile olculur:

$$Q(\hat{S}, S) = \alpha \cdot \text{StructSim}(\hat{S}, S) + \beta \cdot \text{NameSim}(\hat{S}, S) + \gamma \cdot \text{TypeSim}(\hat{S}, S) \tag{7.3}$$

burada $\alpha + \beta + \gamma = 1$ agirliklari, StructSim kontrol akisi grafi (CFG) izomorfizm benzerligini, NameSim isimlendirme dogrulugunu (precision/recall), TypeSim ise tip kurtarma basarisini olcer.

7.6 6 Katmanli C Isimlendirme Sistemi (c_namer.py)

Ghidra decompiler, fonksiyonlara `FUN_00abcdef`, parametrelere `param_1`, yerel degiskenlere `local_28`, global verilere `DAT_00123456` gibi otomatik isimler atar. Bu isimler anlasilabilirlik acisindan neredeyse degersidir. `c_namer.py` modulu, bu isimleri altI katmanli bir strateji zinciriyle anlamli isimlere donusturur.

7.6.1 Mimari Genel Bakis

Sistem, her katmanin bilgi entropisi azaltma katkisiyla bir *cascade classifier* (kademeli siniflandirici) gibi calisir. Her katman bir oncekinin bulamadigi isimleri bulmaya calisir ve her katmanin urettigi isimler farkli guven seviyelerine sahiptir.

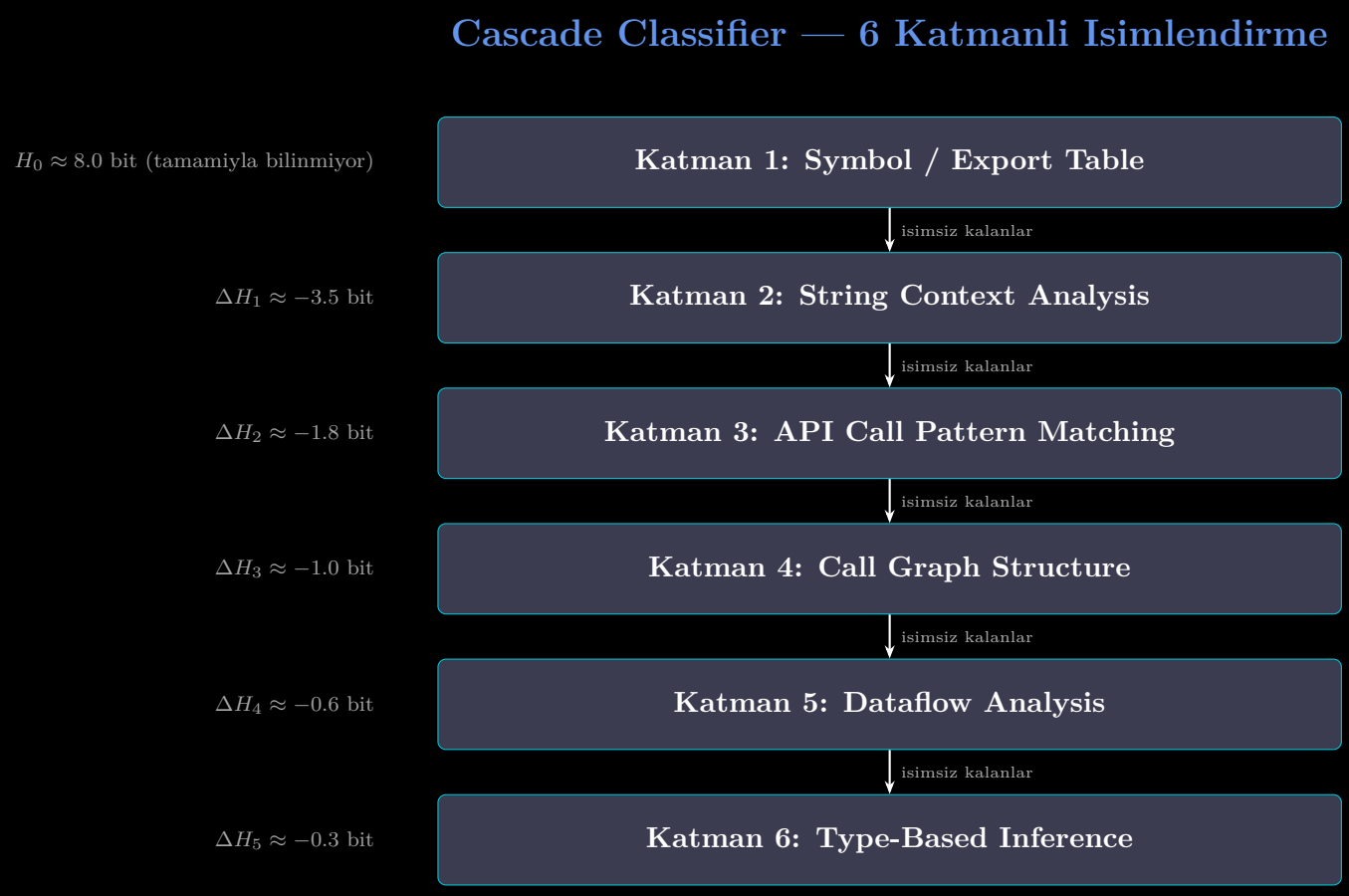


Figure 7.1: CVariableNamer 6 katmanli cascade classifier mimarisi. Her katman, bir onceki katmanin isimlendiremediyi sembolleri isler. Sol tarafta bilgi entropisi (H) azalma miktarlari gosterilmektedir.

Cascade Classifier Analojisi

Bu mimari, Viola-Jones yuz tespit algoritmasindaki cascade classifier'a benzer. Her katman (stage) bir "filtre" islevi gorur: belirli bir guven seviyesinin ustundeki isimleri atar ve geri kalanlari bir sonraki katmana iletir.

$$P(\text{dogru isim}) = 1 - \prod_{k=1}^{6} \Big(1 - p_k \cdot c_k\Big) \tag{7.4}$$

burada p_k katman k'nin yakalama olasiligi (recall) ve c_k guven skoru (confidence). Her katman bagimsiz bilgi kaynagi kullandigi icin, toplam basari olasiligi kumulatif olarak

artar.

7.6.2 Katman 1: Symbol / Export Table

En yuksek guven seviyesindeki ($c_1 = 0.95$) katmandir. Uc alt stratejisi vardir:

1. **DWARF Debug Symbols:** Eger binary strip edilmemisse, `.debug_info` section'indaki DIE (Debug Information Entry) yapilarindan orijinal fonksiyon, degisken ve tip isimleri dogrudan okunur.
2. **Export Table (Mach-O/PE):** Dynamic linker icin tutulan export tablolari (`LC_SYMTAB`, `__LINKEDIT`) orijinal fonksiyon isimlerini icerir.
3. **Objective-C RTTI:** `_OBJC_CLASS_$_ClassName` pattern'i ile class isimleri, `sel_refs` section'indan selector (method) isimleri elde edilir.

```
_GHIDRA_AUTO_FUNC  = re.compile(r"^FUN_[0-9a-fA-F]+$")
_GHIDRA_AUTO_PARAM = re.compile(r"^param_\d+$")
_GHIDRA_AUTO_LOCAL = re.compile(r"^local_[0-9a-fA-F]+$")
_GHIDRA_AUTO_DAT   = re.compile(r"^DAT_[0-9a-fA-F]+$")
_GHIDRA_AUTO_PTR   = re.compile(r"^PTR_[0-9a-fA-F]+$")
_GHIDRA_AUTO_VAR   = re.compile(r"^[a-z]Var\d+$")    # uVar1, iVar2
_GHIDRA_AUTO_STACK = re.compile(r"^Stack\[0x[0-9a-fA-F]+\]$")
```

Listing 7.1: Ghidra otomatik isim tespiti regex pattern'leri (c_namer.py)

Ornek 7.1 (Entropy Azaltma – Katman 1)**.** Tipik bir macOS uygulamasinda 12.000 fonksiyon vardir. Bunlarin 3.000'i (%25) stripped degil ve dogrudan symbol table'dan alinir. Her fonksiyon ismi icin bilgi entropisi:

$$H_{\text{once}} = \log_2(N_{\text{olasi isimler}}) \approx \log_2(10^6) \approx 20 \text{ bit} \tag{7.5}$$

Symbol table eslesmesinden sonra:

$$H_{\text{sonra}} = 0 \text{ bit (kesin esleme)} \tag{7.6}$$

Dolayisiyla $\Delta H_1 = -20$ bit, fakat bu sadece eslesen %25 icin gecerlidir. Agirliklai ortalama: $\Delta \bar{H}_1 = 0.25 \times (-20) = -5.0$ bit/fonksiyon.

7.6.3 Katman 2: String Context Analysis

Binary icindeki string literal'ler isim kurtarma icin zengin bir kaynaktir. Log mesajlari, assert ifadeleri, hata metinleri ve UI string'leri genellikle fonksiyon veya class isimlerini icerir.

String Context Eslestirme

Bir fonksiyon f icindeki string referanslari $\{s_1, s_2, \ldots, s_m\}$ kumesi, f'in amacini gostermek icin analiz edilir:

- `ClassName::MethodName` pattern'i $\Rightarrow c = 0.85$
- `FuncName()` pattern'i $\Rightarrow c = 0.80$
- Source path `/src/module/file.cpp` $\Rightarrow c = 0.75$
- Steam/Valve keyword'leri $\Rightarrow c = 0.65$
- Error verb extraction ("Failed to X") $\Rightarrow c = 0.55$

Keyword extraction sureci, string'den stopword'leri filtreleyerek anlamli kelimeleri cikarir:

$$\text{keywords}(s) = \text{filter}_{\neg\text{stopword}}\Big(\text{tokenize}(s)\Big) \cap \{w : |w| \geq 3\} \tag{7.7}$$

`c_namer.py` icindeki `_extract_keywords_from_string` fonksiyonu 200+ stopword listesiyle filtreleme yapar ve sonucu en fazla 4 kelimeye sinirlar.

7.6.4 Katman 3: API Call Pattern Matching

Bir fonksiyonun cagirdigi sistem API'leri, fonksiyonun amacini guclu sekilde gosterir. Ornegin `socket()`, `connect()`, `send()` uclusunu cagiran bir fonksiyon, yuksek olasilikla bir ag istegi gonderme fonksiyonudur.

Kodda 60+ combo pattern tanimlanmistir:

API Kombinasyonu	Atanan Isim	Conf.
`{socket, connect, send}`	`send_network_request`	0.80
`{fopen, fread, fclose}`	`read_file`	0.80
`{socket, bind, listen, accept}`	`start_server_socket`	0.80
`{fork, exec}`	`spawn_process`	0.80
`{malloc, memcpy, free}`	`copy_buffer`	0.75
`{opendir, readdir, closedir}`	`list_directory`	0.80

API Kombinasyonu	Atanan Isim	Conf.
{getaddrinfo}	resolve_address	0.70
{strlen, strcpy}	copy_string	0.70
{pthread_create}	create_thread	0.70
{mmap, munmap}	map_file_memory	0.75

Table 7.1: API combo pattern'lerinden bir secme (60+ pattern tanimli).

Tek API cagrisi icin de pattern'ler vardir (200+ girdi):

```
_SINGLE_API_HINTS = {
    "CreateFileW":      "open_file_win",
    "RegOpenKeyExW":    "open_registry_key",
    "VirtualAlloc":     "allocate_virtual_memory",
    "NtQueryInformationProcess": "query_process_info",
    "dlopen":           "load_dynamic_library",
    ...
}
```

Listing 7.2: Tek API'den isim esleme ornekleri

7.6.5 Katman 4: Call Graph Structure

Fonksiyonun cagri grafindaki pozisyonu isim hakkinda bilgi verir. Ghidra'nin `call_graph.json` ciktisindaki caller/callee iliskileri analiz edilir:

- **Entry point fonksiyonlar:** Hicbir fonksiyon tarafindan cagirilmayan fonksiyonlar (`in-degree = 0`) ⇒ `main`, `entry_point`
- **Leaf fonksiyonlar:** Baska fonksiyon cagirmayan fonksiyonlar (`out-degree = 0`) ⇒ `utility_func`, `helper`
- **Hub fonksiyonlar:** Cok sayida farkli fonksiyondan cagirilir (`in-degree` $\gg$ `ortalama`) ⇒ `common_handler`, `dispatch`
- **Wrapper fonksiyonlar:** Tek bir fonksiyon cagirip donen (`out-degree = 1`, kisa govde) ⇒ `wrapper_XYZ`

$$\text{role}(f) = \begin{cases} \text{entry} & \text{if } \deg^-(f) = 0 \\ \text{leaf} & \text{if } \deg^+(f) = 0 \\ \text{hub} & \text{if } \deg^-(f) > \mu^- + 2\sigma^- \\ \text{wrapper} & \text{if } \deg^+(f) = 1 \wedge |f| < \tau \end{cases} \tag{7.8}$$

burada μ^-, σ^- in-degree ortalama ve standart sapma, $|f|$ fonksiyon buyuklugu (instruction sayisi), τ esik degeridir.

7.6.6 Katman 5: Dataflow Analysis

Parametre kullanimi, degisken atamalari ve veri akisi izlenerek isim cikarimi yapilir. Bu katman ozellikle parametrelerin isimlendirilmesinde etkilidir.

- `param_1` bir `fopen()` cagrisinin ilk argumani olarak kullaniliyorsa ⇒ `filename`
- `param_2` bir `malloc()` cagrisinin argumani ise ⇒ `size` veya `buffer_size`
- `local_28` bir dongu sayaciysa ⇒ `i`, `loop_counter`

Dataflow analizi, SSA (Static Single Assignment) benzeri bir yaklasimla her degiskenin tum kullanim noktalarini (use-def chain) izler:

$$\text{name}(v) = \text{argmax}_{n \in \mathcal{N}} \sum_{u \in \text{uses}(v)} w(u) \cdot \mathbb{1}[\text{context}(u) \Rightarrow n] \tag{7.9}$$

burada $\mathcal{N}$ aday isimler kumesi, $\text{uses}(v)$ degisken v'nin kullanildigi noktalar, $w(u)$ kullanim noktasinin bilgi agirligi ve $\text{context}(u) \Rightarrow n$ kullanim baglaminin isim n'yi destekleyip desteklemedigi.

7.6.7 Katman 6: Type-Based Inference

En dusuk guven seviyesindeki katmandir ($c_6 = 0.20$–0.40). Ghidra'nin cikardigi tip bilgisinden isim cikarir:

- `char *` tipi ⇒ `str`, `name`, `path`
- `int` donus tipi + `bool` kullanim ⇒ `is_valid`, `check_`
- `void *` parametresi ⇒ `context`, `user_data`
- Struct pointer ⇒ struct adindan turetilmis isim

7.6.8 Toplam Bilgi Entropisi Azaltma

6 katmanin kumulatif etkisi, her sembol icin bilgi entropisi H'yi sistematik olarak azaltir:

$$H_{\text{final}} = H_0 - \sum_{k=1}^{6} \Delta H_k \tag{7.10}$$

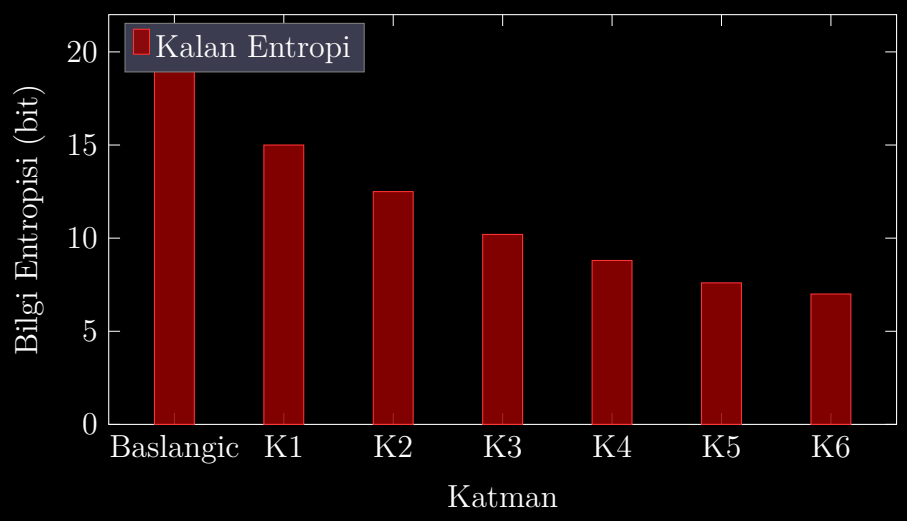


Figure 7.2: 6 katman boyunca bilgi entropisi azalma grafigi. Baslangicta $H_0 \approx 20$ bit olan belirsizlik, Katman 6 sonunda $H_6 \approx 7$ bit'e duser.

Ornek 7.2 (Sayisal Ornek: Steam Bootstrap Binary)**.** 224 MB'lik bir macOS binary'sinde 48.000 fonksiyon tespit edilmistir:

- Katman 1 (Symbol): 12.000 fonksiyon dogrudan isimlendirildi (%25)
- Katman 2 (String): 8.400 ek fonksiyon (%17.5)
- Katman 3 (API): 5.200 ek fonksiyon (%10.8)
- Katman 4 (Call Graph): 3.600 ek fonksiyon (%7.5)
- Katman 5 (Dataflow): 2.400 ek fonksiyon (%5.0)
- Katman 6 (Type): 1.200 ek fonksiyon (%2.5)
- **Toplam:** 32.800 / 48.000 = **%68.3** isimlendirme basarisi
- Kalan 15.200 fonksiyon icin isim atanamaz (`FUN_xxx` kalir)

7.7 Tip Kurtarma (c_type_recoverer.py)

Ghidra'nin `undefined8`, `long *`, `void *` gibi generic tipleri, `c_type_recoverer.py` tarafindan kullanim pattern'lerinden analiz edilerek gercek struct, enum ve vtable tanimlarina donusturulur.

7.7.1 Struct Recovery

Ghidra decompile ciktisinda bir pointer'a offset'ler eklenerek farkli field'lara erisim gosterilir:

```
*(long *)(param_1 + 0x10) = value;       // field @ offset 0x10
*(int *)(param_1 + 0x18) = 42;           // field @ offset 0x18
*(char **)(param_1 + 0x20) = "hello";    // field @ offset 0x20
```

Listing 7.3: Ghidra decompile ciktisinda struct field erisim pattern'i

Bu pattern'ler regex ile tespit edilir:

```
_FIELD_ACCESS_WRITE = re.compile(
    r"\*\(\s*"
    r"(?P<type>[a-zA-Z_][\w\s]*\*{0,2})\s*\*?\s*\)"
    r"\(\s*(?P<base>\w+)\s*\+\s*"
    r"(?P<offset>0x[0-9a-fA-F]+|\d+)\s*\)\s*="
)
```

Listing 7.4: Field access regex pattern'leri (c_type_recoverer.py)

Toplanan offset-tip ciftlerinden bir struct sentezlenir:

$$\text{Struct}_i = \left\{(o_j, t_j, s_j) : j = 1, \ldots, n_i\right\} \tag{7.11}$$

burada o_j byte offset, t_j tip, s_j byte boyutudur. Alignment ARM64 icin 8 byte'dir.

Ornek 7.3 (Struct Recovery Ornegi)**.** Asagidaki Ghidra ciktisi:

```
*(long *)(param_1 + 0x00) = uVar3;
*(int  *)(param_1 + 0x08) = 0;
*(char**)(param_1 + 0x10) = "default";
*(long *)(param_1 + 0x18) = 0x100;
```

Kurtarilan struct:

```
typedef struct {
    long   field_00;    // offset 0x00, 8 bytes
    int    field_08;    // offset 0x08, 4 bytes
```

```
    char  *field_10;     // offset 0x10, 8 bytes
    long   field_18;     // offset 0x18, 8 bytes
} recovered_struct_001; // total: 32 bytes, alignment: 8
```

7.7.2 Enum Recovery

Switch-case yapilari ve karsilastirma pattern'leri enum tespiti icin kullanilir:

```
_SWITCH_START = re.compile(r"switch\s*\(\s*(?P<var>\w+)\s*\)")
_CASE_VALUE   = re.compile(r"case\s+(?P<val>-?\d+|0x[0-9a-fA-F
    ]+)\s*:")
```

Listing 7.5: Switch/case regex pattern'leri

Bir switch ifadesinde 4'ten fazla case degeri varsa, bu degerler bir enum olarak sentezlenir:

$$\text{Enum}_i = \left\{ (\text{name}_j, v_j) : j = 1, \ldots, m_i \right\} \tag{7.12}$$

7.7.3 VTable Recovery

Fonksiyon pointer atamalari vtable yapisini gosterir:

```
*(code **)(obj + 0x00) = FUN_001234;   // virtual method 0
*(code **)(obj + 0x08) = FUN_005678;   // virtual method 1
*(code **)(obj + 0x10) = FUN_009abc;   // virtual method 2
```

Regex: `_VTABLE_ASSIGN` pattern'i `code **` tipli pointer atamalarini yakalar ve `(offset, fonksiyon_adi, signature)` uclulerini toplar.

7.8 Algoritma Tespiti (c_algorithm_id.py)

Decompile edilmis C kodunda kullanilan kriptografik algoritmalari, hash fonksiyonlarini ve diger bilinen algoritmalari tespit eder. Uc katmanli bir yaklasim kullanilir.

7.8.1 Katman 1: Sabit-Bazli Tanimlar (Constant-Based)

Kriptografik algoritmalar, kolayca degistirilemeyen matematiksel sabitlere sahiptir. Bu sabitlerin kodda varligi, algoritmanin kullanildiginin guclu kanitidir.

Algoritma	Sabit Adi	Ilk Byte'lar (hex)	Conf.
AES	S-box	`63 7C 77 7B F2 6B 6F C5`	0.95
AES	Inverse S-box	`52 09 6A D5 30 36 A5 38`	0.95
AES	Rcon	`01 02 04 08 10 20 40 80`	0.90
SHA-256	K constants	`428A2F98 71374491 B5C0FBCF E9B5DBA5`	0.95
SHA-256	H init	`6A09E667 BB67AE85 3C6EF372 A54FF53A`	0.95
SHA-512	K constants	`428A2F98D728AE22 7137449123EF65CD`	0.95
SHA-1	H init	`67452301 EFCDAB89 98BADCFE 10325476`	0.95
MD5	T values	`D76AA478 E8C7B756 242070DB C1BDCEEE`	0.95
CRC32	Table start	`00000000 77073096 EE0E612C 990951BA`	0.90
ChaCha20	Sigma	`61707865 3320646E 79622D32 6B206574`	0.95
Blowfish	P-array	`243F6A88 85A308D3 13198A2E 03707344`	0.90
HMAC	ipad/opad	`36 36 36 36` / `5C 5C 5C 5C`	0.80
Poly1305	Clamp	`0FFFFFFF 0FFFFFFC 0FFFFFFC 0FFFFFFC`	0.90

Table 7.2: Sabit-bazli algoritma tespiti icin kullanilan kriptografik sabitlerin bir bolumu. Kodda toplam 25+ algoritma icin 40+ sabit grubu tanimlidir.

AES S-Box Tespiti

AES S-box, Galois Field $GF(2^8)$'de carpimsal ters alma ve affine donusum ile hesaplanan 256 byte'lik bir lookup table'dir:

$$S(x) = A \cdot x^{-1} \oplus c, \quad x \in GF(2^8), \quad \text{irreducible polynomial: } m(x) = x^8 + x^4 + x^3 + x + 1 \tag{7.13}$$

Ilk 8 byte'in kodda bulunmasi yeterli kanittir ($p_{\text{false positive}} < 10^{-15}$).

7.8.2 Katman 2: Yapi-Bazli Tespit (Structure-Based)

Kod yapisindan algoritma pattern'leri taninir. 14 yapisal pattern tanimlidir:

- **Feistel Network:** $L_{i+1} = R_i$, $R_{i+1} = L_i \oplus F(R_i, K_i)$ pattern'i – DES, Blowfish icin.
- **S-box Lookup:** `table[x & 0xFF]` veya `table[(x » n) & 0xFF]` – AES SubBytes icin.
- **Hash Rounds:** Rotate (»>, «<), XOR ve toplama islemlerinin 16+ kez tekrari – SHA, MD5 icin.
- **Galois Field:** Moduler aritmetik (`a * b % p`) – ECC icin.
- **Key Schedule:** `round_key`, `key_schedule` degiskenleri.
- **Base64 Encoding:** Alfabe string'i `A-Za-z0-9+/`.

7.8.3 Katman 3: API Korelasyonu

Bilinen crypto API cagrilari dogrudan algoritma tespiti saglar. Kodda 50+ crypto API tanimlidir:

- **Apple CommonCrypto:** `CCCrypt`, `CC_SHA256`, `CCHmac`
- **OpenSSL:** `EVP_EncryptInit`, `SSL_read`, `RSA_public_encrypt`
- **Windows CryptoAPI:** `CryptEncrypt`, `BCryptOpenAlgorithmProvider`
- **Sodium:** `crypto_secretbox`, `crypto_aead_`

7.9 Yorum Uretimi (c_comment_generator.py)

Decompile edilmis C dosyalarina bes tip yorum eklenir:

1. **Function Header Block:** Doxygen benzeri fonksiyon baslik yorumu. Parametre tipleri, donus degeri, cagiran/cagrilan fonksiyonlar, tespit edilen algoritmalar.
2. **System Call Annotations:** 100+ sistem cagrisi icin aciklama. Ornegin: `malloc(param_1)` ⇒ `/* allocates {arg0} bytes on heap */`
3. **Vulnerability Annotations:** Tehlikeli fonksiyon kullanimlari icin uyari. `strcpy` ⇒ `/* WARNING: no bounds check, use strncpy */`

4. **Control Flow Annotations:** State machine, infinite loop, buyuk switch-case yapilari icin aciklama.

5. **Algorithm Labels:** `c_algorithm_id` sonuclarindan algoritma etiketleri.

```
/**
 * Function: send_network_request (FUN_00102a40)
 * @brief Sends data over network socket
 * @param param_1 (renamed: socket_fd) - socket file descriptor
 * @param param_2 (renamed: buffer) - data buffer pointer
 * @param param_3 (renamed: length) - buffer length
 * @return int - bytes sent or -1 on error
 *
 * Calls: socket(), connect(), send(), close()
 * Called by: FUN_00103b80, FUN_00104c20
 * Algorithm: None detected
 * Confidence: 0.80 (API pattern)
 */
int send_network_request(int socket_fd, void *buffer, long
    length) {
    /* socket(AF_INET, SOCK_STREAM, 0) - creates network socket
        */
    int fd = socket(2, 1, 0);
    ...
}
```

Listing 7.6: Otomatik yorum uretimi ornegi

7.10 Proje Yapilandirma

7.10.1 project_reconstructor.py – Modul Iskele Olusturma

JavaScript bundle'lardan elde edilen modullerin gercek bir proje yapisina donusturulmesini saglar. 8 adimlik pipeline:

1. Modul dizinini bul (`deep_modules/` veya `webpack_modules/`)

2. `reconstruct-project.mjs` script'i ile 30+ kategori pattern'ine gore dosya yerlestirme

3. `DependencyResolver` ile gercek npm paketlerini tespit

4. `package.json` olusturma
5. Entry point belirleme
6. README olusturma
7. `.gitignore` olusturma
8. Proje yapisini dogrulama (validation)

7.10.2 C Project Builder (c_project_builder.py)

Decompile edilmis binary'den organize bir IDE projesi uretir. Subsystem siniflandirma kurallari ile fonksiyonlar mantiksal gruplara ayrilir:

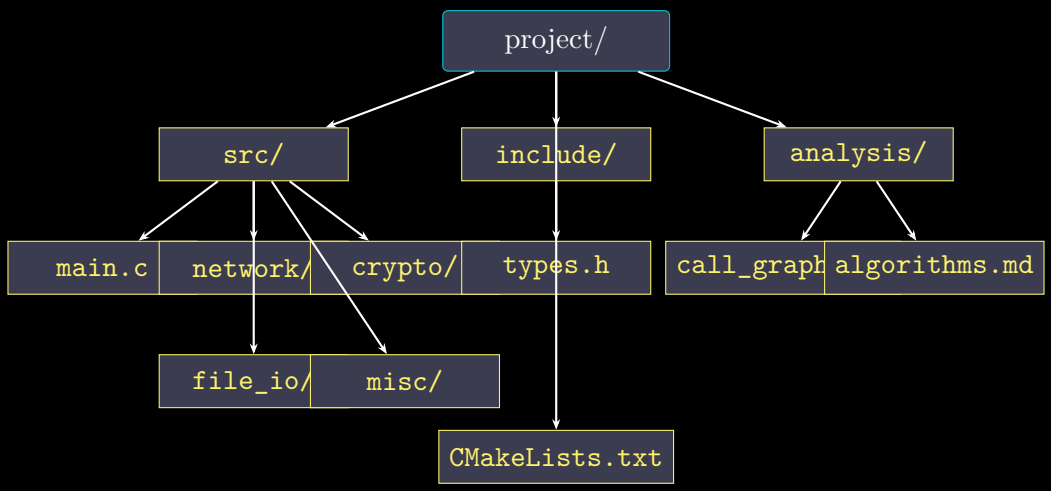


Figure 7.3: C Project Builder cikti yapisi. Fonksiyonlar API kullanim pattern'lerine gore subsystem klasorlerine (network, crypto, file_io) dagitilir.

7.10.3 project_builder.py – CMake/Makefile Uretimi

JavaScript projeler icin `npm install && npm start` ile calisabilecek bir proje uretir:

- `package.json`: Otomatik tespit edilen dependency'ler, script'ler
- `.env.example`: Ortam degiskenleri sablonu
- `Dockerfile`: Multi-stage build ile production-ready container
- `README.md`: Proje dokumantasyonu

7.11 NPM Source Matching

Minified JavaScript bundle'larindaki fonksiyonlari orijinal npm paketi kaynak koduyla eslestirme.

7.11.1 AST Structural Hash (ast_fingerprinter.py)

Her fonksiyon icin yapisal parmak izi cikarilir. Minifier'lar isimleri degistirir ama yapisal ozellikleri korur:

$$\mathrm{FP}(f) = \Big(\mathrm{arity}(f),\ \mathcal{S}(f),\ \mathcal{P}(f),\ B(f),\ L(f),\ R(f)\Big) \tag{7.14}$$

burada:

- $\mathrm{arity}(f)$: Parametre sayisi
- $\mathcal{S}(f)$: String literal'ler kumesi (minifier bunlara dokunmaz)
- $\mathcal{P}(f)$: Property access'ler kumesi (`.push`, `.map`)
- $B(f)$: Dallanma yapisi (if/switch/ternary sayilari)
- $L(f)$: Dongu yapisi (for/while/do-while sayilari)
- $R(f)$: Return/throw sayilari

7.11.2 Tree Edit Distance (Zhang-Shasha)

Iki fonksiyonun yapisal benzerligi, AST'lerin tree edit distance'i ile hesaplanir:

$$d_{\mathrm{edit}}(T_1, T_2) = \min_{\pi \in \Pi} \sum_i c(\pi_i) \tag{7.15}$$

burada Π tum gecerli edit islem dizileri kumesi, $c(\pi_i)$ her islemin maliyeti (insert, delete, relabel).

Zhang-Shasha algoritmasi $\mathcal{O}(|T_1| \cdot |T_2| \cdot \min(d_1, l_1) \cdot \min(d_2, l_2))$ zaman karmasikligina sahiptir; burada d_i derinlik ve l_i yaprak sayisi.

Normallestirilmis benzerlik:

$$\mathrm{sim}(T_1, T_2) = 1 - \frac{d_{\mathrm{edit}}(T_1, T_2)}{|T_1| + |T_2|} \tag{7.16}$$

7.11.3 Structural Matcher Pipeline

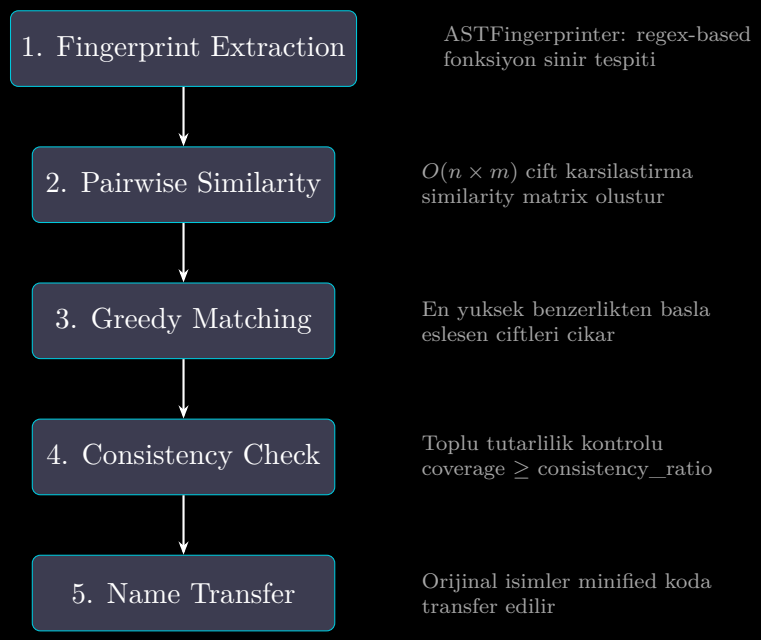


Figure 7.4: Source matching pipeline – 5 adimda minified fonksiyonlarin orijinal isimleri kurtarilir.

Greedy matching algoritmasi:

1. Her (minified, orijinal) cifti icin $\text{sim}(f_i^{\text{min}}, f_j^{\text{orig}})$ hesapla
2. Tum ciftleri azalan benzerlige gore sirala
3. En yuksekten baslayarak eslestir (matched olanlari cikar)
4. sim $<$ min_similarity (varsayilan 0.65) alti olanlarini at

7.12 Parametre Kurtarma (param_recovery.py)

Obfuscated JavaScript fonksiyonlarindaki tek harfli parametre isimlerini anlamli isimlere donusturur. 5 strateji kullanilir:

Strateji	Pattern	Confidence	Oncelik
this.X = param	`this.name = a` $\Rightarrow$ `a` $\rightarrow$ `name`	0.90–0.95	1
Destructuring	`const {X,Y} = a` $\Rightarrow$ `a` $\rightarrow$ `options`	0.88	2
Call-site object	`fn({key: val})` $\Rightarrow$ param $\rightarrow$ `options`	0.82	3
Call-site identifier	`fn(userId)` $\Rightarrow$ param $\rightarrow$ `userId`	0.72	4
Property access	`a.length` $\Rightarrow$ `a` $\rightarrow$ `array`	0.35–0.78	5

Strateji	Pattern	Confidence	Oncelik

Table 7.3: Parametre kurtarma stratejileri ve guven seviyeleri.

7.12.1 TypeScript .d.ts Mapping (dts_namer.py)

TypeScript tanim dosyalarindan export isimleri cikarilir ve minified koddaki export'larla eslestirilir:

1. **Kaynak stratejisi:**
   - DefinitelyTyped: `@types/xxx` paketi (npm registry)
   - Paketin kendi `.d.ts` dosyasi (`package.json` "types" alani)
   - `unpkg.com` CDN uzerinden indirme (fallback)
2. **Eslestirme stratejisi:** Export sirasi build tool'lar tarafindan korunur; isim uzunluklari ve tip bilgileri ile cross-validation.

$$\text{match}(e_i^{\text{min}}, e_j^{\text{dts}}) = \begin{cases} 1 & \text{if kind}_i = \text{kind}_j \wedge |\text{arity}_i - \text{arity}_j| \leq 1 \\ 0 & \text{otherwise} \end{cases} \tag{7.17}$$

Bu Bolumun Ozeti

- Kaynak kod rekonstruksiyonu, 6 katmanli cascade classifier ile fonksiyon isimlendirme yapar (symbol → string → API → call graph → dataflow → type).
- Her katman bagimsiz bilgi kaynagi kullanir ve toplam basari kumulatif artar.
- Tip kurtarma, field access pattern'lerinden struct, switch/case'den enum, function pointer'lardan vtable sentezler.
- Algoritma tespiti 3 katmanli: sabit-bazli (AES S-box, SHA-256 K), yapi-bazli (Feistel, hash rounds) ve API korelasyonu.
- NPM source matching, AST fingerprint ve greedy matching ile minified fonksiyonlarin orijinal isimlerini kurtarir.
- Parametre kurtarma 5 strateji kullanir: this.X=param en yuksek guvenle.

7.13 Guvenlik Analiz Algoritmalarinin Temeli

Karadul'un guvenlik analiz katmani, binary ve bytecode duzeyinde tehditleri tespit etmek ve koruma mekanizmalarini asmak icin bir dizi algoritmik teknik kullanir. Bu bolumde bu algoritmalarin matematiksel temelleri, karmasikliklari ve implementasyon detaylari ele alinir.

7.14 Shannon Entropy ve Packer Tespiti

7.14.1 Shannon Entropy Formulu

Bir rasgele degisken X'in bilgi entropisi (Shannon, 1948):

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x) \tag{7.18}$$

Entropi Turetmesi

Bilgi entropisi, bir mesajin ortalama bilgi icerigi olarak turetilebilir. N byte'lik bir binary dosyada x_i byte degerinin frekansi f_i ise, olasilik tahmini:

$$\hat{p}(x_i) = \frac{f_i}{N}, \quad i = 0, 1, \ldots, 255 \tag{7.19}$$

Binary verilerin entropisi $[0, 8]$ bit araligindadir (byte basina):

- $H = 0$: Tum byte'lar ayni (ornegin tamamen sifir)
- $H = 8$: Tum byte'lar esit olasilikli (uniform rastgele, sifreli veya sikistirilmis)

```
def calculate_entropy(data: bytes) -> float:
    if not data:
        return 0.0
    freq = [0] * 256
    for byte in data:
        freq[byte] += 1
    length = len(data)
    entropy = 0.0
    for f in freq:
        if f > 0:
            p = f / length
            entropy -= p * math.log2(p)
```

```
    return entropy
```

Listing 7.7: Shannon entropy implementasyonu (packed_binary.py)

Algoritma karmasikligi: $\mathcal{O}(N + 256) = \mathcal{O}(N)$ – dosya boyutu N ile dogrusal.

7.14.2 Section-Level Entropy Profiling

Tum dosyanin entropisi yerine, section'lara bolup her birinin entropisini hesaplamak daha bilgilendiricidir:

$$H_j = H(X_j), \quad j = 1, \ldots, \lceil N/S \rceil \tag{7.20}$$

burada S section boyutu (varsayilan 65536 byte = 64 KB).

```
def calculate_section_entropy(data: bytes, section_size: int =
    65536):
    entropies = []
    for i in range(0, len(data), section_size):
        chunk = data[i:i + section_size]
        if len(chunk) >= 256:  # cok kucuk chunk yaniltici
            entropies.append(calculate_entropy(chunk))
    return entropies
```

Listing 7.8: Section-level entropy (packed_binary.py)

7.14.3 Packing Tespit Karari

`PackingDetector` sinifi, 5 adimlik bir tespit pipeline'i kullanir:

7.14.4 Packer Signature'lari

Packer	Tespit Yontemi	Magic Bytes	Conf.
UPX	Magic byte `UPX!`	`55 50 58 21`	0.95
PyInstaller	MEI magic (8 byte)	`4D 45 49 0C 0B 0A 0B 0E`	0.95
Nuitka	String patterns	`__nuitka_` + 4 variant	0.85
Themida	High entropy + VM stub	Entropy > 7.5	0.75
VMProtect	Virtualized sections	Section name + entropy	0.70

Packer	Tespit Yontemi	Magic Bytes	Conf.

Table 7.4: Desteklenen packer tipleri ve tespit yontemleri.

7.14.5 Chi-Squared Uniformity Test

Section entropisine ek olarak, byte dagiliminin uniform olup olmadigini test etmek icin chi-squared testi kullanilir:

$$\chi^2 = \sum_{i=0}^{255} \frac{(O_i - E_i)^2}{E_i}, \quad E_i = \frac{N}{256} \tag{7.21}$$

burada O_i gozlenen frekans, E_i beklenen frekans. $\chi^2 < 293.25$ (%99.9 guven araligi, $df = 255$) ise byte dagilimi uniform kabul edilir – bu da guclu sikilasma veya sifreleme isaretidir.

Ornek 7.4 (Sayisal Ornek)**.** 1 MB'lik bir section'da:

- Her byte degeri $E = 1048576/256 = 4096$ kez beklenir
- Uniform rastgele veride: $\chi^2 \approx 255$ (serbestlik derecesi)
- Normal C kodunda: $\chi^2 \approx 50000+$ (0x00 ve 0xFF dominant)
- Packed veride: $\chi^2 \approx 200$–300 (uniform'a yakin)

7.15 YARA Pattern Matching

7.15.1 Aho-Corasick Algoritma Temeli

YARA, pattern eslestirme icin Aho-Corasick coklu pattern arama algoritmasini kullanir:

$$T(n, m, z) = \mathcal{O}(n + m + z) \tag{7.22}$$

burada n metin uzunlugu, m tum pattern'lerin toplam uzunlugu, z esleme sayisi.

Aho-Corasick Otomatonu

Aho-Corasick, pattern kumesi $P = \{p_1, p_2, \ldots, p_k\}$ icin bir trie (prefix tree) olusturur ve bu trie'yi failure link'lerle zenginlestirir:

1. **Goto fonksiyonu** $g(s, a)$: State s'den karakter a ile sonraki state.

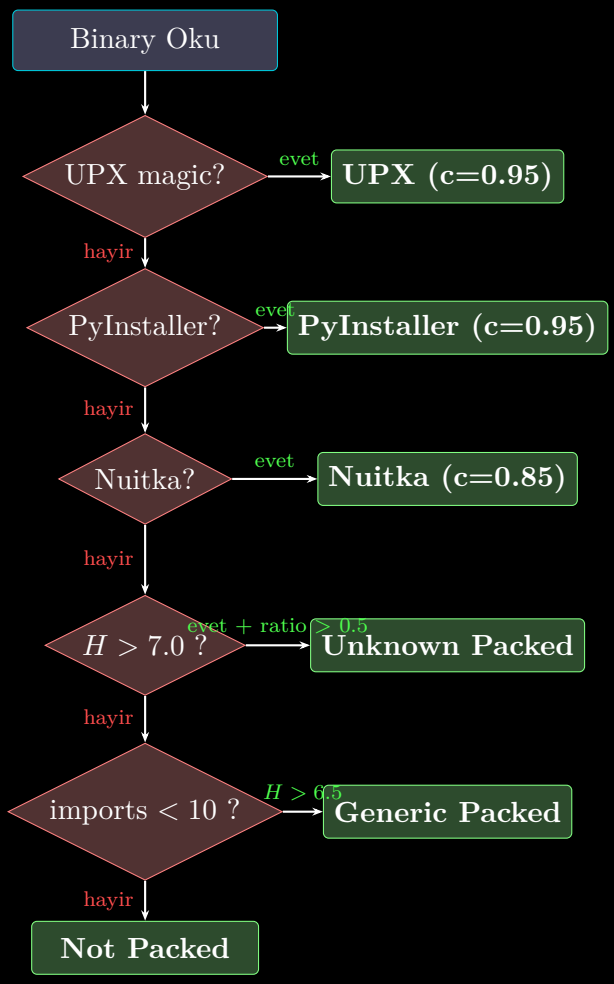


Figure 7.5: PackingDetector karar agaci. Her adimda farkli bir kanit kaynagi kontrol edilir.

2. **Failure fonksiyonu** $f(s)$: Esleme basarisiz oldugunda geri donus state'i (KMP'nin failure fonksiyonunun genellestirilmesi).

3. **Output fonksiyonu** $\text{out}(s)$: State s'de tamamlanan pattern'ler.

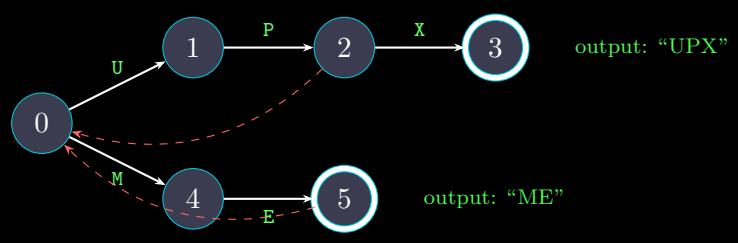


Figure 7.6: Basitlestirilmis Aho-Corasick otomatonu ornegi: "UPX" ve "ME" pattern'leri icin. Kesikli oklar failure link'lerdir.

7.15.2 YARA Kural Yapisi

Karadul, YARA kurulu degilse regex-based fallback ile calisir. 40+ dahili kural tanimlanmistir:

```
rules.append(BuiltinRule(
    name="AES_SBox",
    tags=["crypto", "aes"],
    meta={"description": "AES S-Box lookup table"},
    byte_patterns=[
        _hex("63 7C 77 7B F2 6B 6F C5 30 01 67 2B FE D7 AB 76"),
    ],
))

rules.append(BuiltinRule(
    name="SHA256_Init_Constants",
    tags=["crypto", "sha256"],
    byte_patterns=[
        _hex("6A09E667 BB67AE85 3C6EF372 A54FF53A"),
    ],
))
```

Listing 7.9: Dahili YARA kural ornekleri (yara_scanner.py)

7.15.3 Hex Pattern + Wildcard

YARA hex pattern'leri wildcard (`??`) ve jump (`[n-m]`) destekler:

$$\text{pattern}: \quad \texttt{63 7C ?? 7B [2-4] 6B 6F C5} \tag{7.23}$$

Bu, "`63 7C`" ile basla, herhangi bir byte, `7B`, 2 ila 4 byte atlama, sonra "`6B 6F C5`" seklinde eslesir. Bu esneklik compiler optimization farklarini absorbe eder.

7.16 Control Flow Graph (CFG) Analizi

7.16.1 Formal Tanim

Control Flow Graph

Bir programin CFG'si $G = (V, E, v_{\text{entry}}, v_{\text{exit}})$ yonlu grafi ile tanimlanir:

- V: Basic block'lar kumesi
- $E \subseteq V \times V$: Kontrol akisi kenarlari
- $v_{\text{entry}} \in V$: Giris blogu
- $v_{\text{exit}} \in V$: Cikis blogu

7.16.2 Basic Block ve Leader Detection

Bir *basic block*, kontrol akisinin yalnizca basindan girdigi ve yalnizca sonundan ciktigi maksimal instruction dizisidir.

Leader tespiti kurallari:

1. Ilk instruction bir leader'dir
2. Bir dallanma (jump) hedefi olan instruction bir leader'dir
3. Bir dallanma instruction'indan hemen sonraki instruction bir leader'dir

7.16.3 Dominator Tree – Lengauer-Tarjan

Dominator

$d \in V$ dugumleri $n \in V$ dugumunu *dominate* eder (d dom n) eger ve ancak v_{entry}'den n'ye giden her yol d'den geciyorsa.

$$\text{Dom}(n) = \{n\} \cup \bigcap_{p \in \text{pred}(n)} \text{Dom}(p) \tag{7.24}$$

Lengauer-Tarjan algoritmasi dominator tree'yi neredeyse dogrusal zamanda hesaplar:

$$T_{\text{LT}}(n) = \mathcal{O}(n \cdot \alpha(n)) \tag{7.25}$$

burada $\alpha(n)$ ters Ackermann fonksiyonudur (pratikte ≤ 4, yani neredeyse $\mathcal{O}(n)$).

7.16.4 Loop Detection ve Back Edges

Back Edge

Bir kenar $(u, v) \in E$, eger v dugumu u'yu dominate ediyorsa bir *back edge*'dir. Her back edge bir natural loop tanimlar.

Natural loop tespit algoritmasi:

1. Dominator tree'yi hesapla
2. Back edge'leri bul: (u, v) s.t. v dom u
3. Her back edge (u, v) icin loop body'yi hesapla: $\text{loop}(u, v) = \{v\} \cup \{w : w \neq v \wedge v \text{ dom } w \wedge w \rightarrow^* u\}$

7.17 Opaque Predicate Tespiti

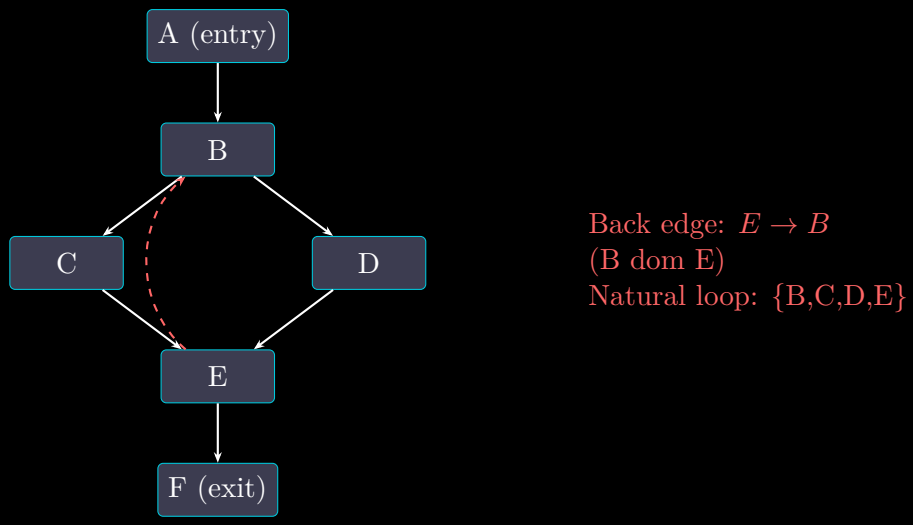


Figure 7.7: CFG ornegi: back edge (E → B) bir natural loop tanimlar. B, E'yi dominate ettigi icin bu kenar bir back edge'dir.

7.17.1 Tanim ve Amaclari

Opaque Predicate

Bir opaque predicate P oyle bir boolean ifadedir ki degeri derleme zamaninda belirlidir (her zaman true veya her zaman false), fakat statik analiz icin "opaque" yani karar verilemez gorunur.

Obfuscator'lar opaque predicate'leri CFG'ye sahte dallanmalar eklemek icin kullanir:

```
if (x*x + x) % 2 == 0) {  // Her zaman TRUE
    real_code();
} else {
    dead_code();            // Asla calistirilmaz
}
```

Listing 7.10: Opaque predicate ile obfuscation

7.17.2 Matematiksel Ispat: $x^2 + x \equiv 0 \pmod 2$

Teorem 7.1. Her $x \in \mathbb{Z}$ icin $x^2 + x \equiv 0 \pmod 2$.

Proof. $x^2 + x = x(x+1)$. Herhangi iki ardisik tam sayinin carpimi cifttir cunku ikisinden biri cift olmak zorundadir. Dolayisiyla $x(x+1) \equiv 0 \pmod 2$. □

Diger yaygin opaque predicate'ler:

$$7y^2 - 1 \neq x^2 \quad \forall x, y \in \mathbb{Z} \quad \text{(Diophantine imkansizlik)} \tag{7.26}$$

$$3 \mid x^3 - x \quad \forall x \in \mathbb{Z} \quad \text{(Fermat's Little Theorem)} \tag{7.27}$$

$$x^2 \geq 0 \quad \forall x \in \mathbb{R} \quad \text{(Trivial ama etkili)} \tag{7.28}$$

7.17.3 Symbolic Execution ile Tespit (Z3)

`inline_detector.py` modulu, basit opaque predicate'leri regex ile tespit eder. Karadul'un gelecek surumlerinde Z3 SMT solver entegrasyonu planlanmaktadir:

$$\text{IsOpaque}(P) \iff \text{Z3.solve}(\neg P) = \text{UNSAT} \tag{7.29}$$

Eger $\neg P$ (predicate'in negasyonu) tatmin edilemez ise, P her zaman true'dur ve dolayisiyla opaque'tir.

7.17.4 Rice's Theorem ve Undecidability

Teorem 7.2 (Rice's Theorem)**.** Turing-complete bir programlama dilinde, herhangi bir "ilginc" (trivial olmayan) semantik ozellik karar verilemez (undecidable).

Bu teorem, opaque predicate tespitinin genel durumda algoritmik olarak cozulemeyecegini gosterir. Karadul, bu sorunu:

1. **Pattern-based heuristic:** Bilinen opaque formulleri regex ile tespit
2. **Bounded symbolic execution:** Sinirli derinlikte sembolik calistirma
3. **Statistical analysis:** Dead code path'lerin runtime'da hic calistirilmadiginin tespiti (Frida ile)

yaklasimlariyla pratikte ele alinabilir hale getirir.

7.18 Control Flow Flattening (CFF) Deflattening

7.18.1 CFF Nedir?

Control Flow Flattening, orijinal CFG'deki yapili kontrol akisini (if-else, loop) bir *dispatcher* switch-case yapisina donusturur:

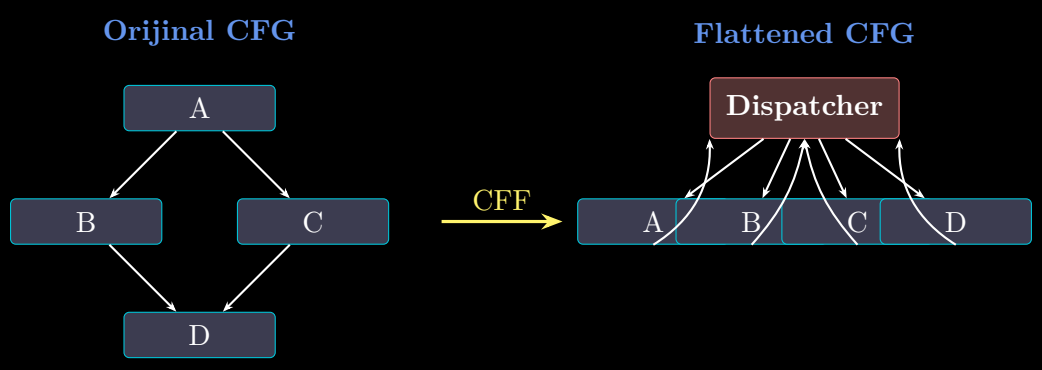


Figure 7.8: Control Flow Flattening: Orijinal yapili CFG (sol) bir dispatcher switch-case'e donusturulur (sag). Her basic block dispatcher'a geri doner ve state degiskeni sonraki blogu belirler.

7.18.2 Deflattening Algoritmasi

CFF deflattening, dispatcher degiskenini ve state gecislerini analiz ederek orijinal CFG'yi yeniden kurar:

1. **Dispatcher Variable Tespiti:**

$$v_{\text{disp}} = \arg\max_{v} \left| \{ b \in V : v \in \text{def}(b) \cap \text{use}(\text{switch}) \} \right| \tag{7.30}$$

   En cok blokta tanimlanan ve switch kosulunda kullanilan degisken, dispatcher degiskenidir.

2. **State Transition Graph Cikarma:** Her basic block b'nin sonundaki dispatcher degiskeni atamasi, sonraki state'i (ve dolayisiyla sonraki blogu) belirler:

$$\text{next}(b) = \{ s : v_{\text{disp}} \leftarrow s \text{ at end of } b \} \tag{7.31}$$

3. **Orijinal CFG Reconstruct:** State transition graph'tan orijinal if-else ve loop yapilari kurulur:

$$E_{\text{orig}} = \{ (b_i, b_j) : s_j \in \text{next}(b_i) \} \tag{7.32}$$

7.19 Ghidra Decompilation Pipeline

7.19.1 Binary'den C Koduna Yolculuk

Ghidra'nin decompilation pipeline'i dort asamadan olusur:

1. **Binary → P-code:** Her makine instruction'i, mimariden bagimsiz bir *P-code*

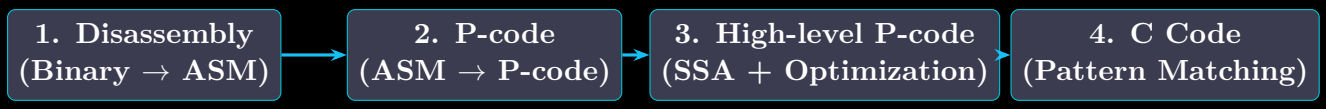


Figure 7.9: Ghidra decompilation pipeline'in 4 asamasi.

(intermediate representation) instruction dizisine donusturulur. P-code, SLEIGH dilinde tanimlanmis 70 opcode iceren bir IR'dir.

2. **P-code → High-level P-code:** SSA (Static Single Assignment) formu uygulanir. Dead code elimination, constant propagation, copy propagation ve expression simplification yapilir.

3. **SSA Formu:**

$$x_1 = 5; \quad y_1 = x_1 + 3; \quad x_2 = 7; \quad z_1 = x_2 + y_1 \tag{7.33}$$

Her degisken yalnizca bir kez atanir. ϕ-fonksiyonlari merge noktalarinda kullanilir.

4. **Type Recovery:** P-code operand boyutlari ve kullanim pattern'lerinden C tipleri cikarilir.

5. **C Code Generation:** High-level P-code, pattern matching ile C syntax'ina donusturulur. If-else, for, while, switch-case gibi yapilar taninir.

7.20 Dynamic Analysis – Frida

7.20.1 Inline Hooking vs PLT/GOT Hooking

Ozellik	Inline Hooking	PLT/GOT Hooking
Mekanizma	Fonksiyon basina trampoline kodu yazar	GOT entry'sini hook fonksiyonuna yonlendirir
Hedef	Her fonksiyon (internal dahil)	Yalnizca imported fonksiyonlar
Tespit zorlugu	Yuksek (instruction rewrite)	Dusuk (pointer degistirme)
Kararlilik	Orta (instruction alignment sorunu)	Yuksek
Performans	Dusuk overhead (∼10ns/call)	Cok dusuk overhead (∼3ns/call)

Ozellik	Inline Hooking	PLT/GOT Hooking

Table 7.5: Inline hooking ve PLT/GOT hooking karsilastirmasi.

7.20.2 API Tracing

Frida ile runtime'da API cagrilarini izleyerek:

- Fonksiyon cagri sirasi ve parametreleri kaydedilir
- Return degerleri izlenir
- Zaman bilgisi ile profiling yapilir

7.20.3 Anti-Debug Bypass

Yaygin anti-debug teknikleri ve Frida ile bypass yontemleri:

Teknik	Platform	Bypass Yontemi
`ptrace(PTRACE_TRACEME)`	Linux/macOS	Interceptor.replace ile NOP'la
`IsDebuggerPresent()`	Windows	Return 0 olarak hook'la
`sysctl(CTL_KERN, KERN_PROC)`	macOS	P_TRACED flag'ini mask'le
Timing checks (`rdtsc`)	x86	Zamani sabit deger olarak don
Exception-based	All	Exception handler'i hook'la

Table 7.6: Anti-debug bypass teknikleri.

7.21 Binary Diffing (BinDiff)

7.21.1 CFG Isomorphism Problemi

Iki fonksiyonun CFG'lerinin izomorf olup olmadigini belirlemek, graph isomorphism (GI) problemine indirgenir:

Graph Isomorphism

$G_1 = (V_1, E_1)$ ve $G_2 = (V_2, E_2)$ graflerinin izomorf oldugu soylenir ($G_1 \cong G_2$) eger bijektif bir $\phi : V_1 \to V_2$ fonksiyonu varsa ve $(u, v) \in E_1 \iff (\phi(u), \phi(v)) \in E_2$.

GI problemi NP-intermediate sinifinda oldugu tahmin edilir (ne P ne NP-complete oldugu kanitlanamamistir). Pratikte BinDiff yaklasik algoritmalar kullanir.

7.21.2 5 Strateji ile Fonksiyon Eslestirme

`bindiff.py` 5 eslestirme stratejisi uygular:

1. **Decompiled Hash** ($c = 0.95$): Normalize edilmis C kodu hash'i

$$h(f) = \text{SHA-256}\Big(\text{normalize}(f)\Big) \tag{7.34}$$

   Normalization: adresler → `ADDR`, degisken isimleri → `VAR`, fonksiyon isimleri → `FUNC`, whitespace temizleme.

2. **CFG Fingerprint** ($c = 0.80$): Basic block pattern'i

$$\text{fp}(f) = (\text{size}(f),\ |\text{params}|,\ \text{ret_type},\ \text{conv}) \tag{7.35}$$

3. **String Referans Fingerprint** ($c = 0.75$): Fonksiyonun referans ettigi string kumeleri

$$\text{sim}_S(f, g) = \frac{|S(f) \cap S(g)|}{|S(f) \cup S(g)|} \quad \text{(Jaccard similarity)} \tag{7.36}$$

4. **Size + Params** ($c = 0.65$): Fonksiyon boyutu ve parametre bilgileri

5. **Call Graph Pattern** ($c = 0.60$): Caller/callee komsuluk benzerlik skoru

7.21.3 Hungarian Algorithm ile Optimal Eslestirme

Eslestirme problemi, iki tarafli graf (bipartite graph) uzerinde minimum agirlikli eslestirme olarak formalize edilir:

$$\min_{\pi \in S_n} \sum_{i=1}^{n} C(i, \pi(i)) \tag{7.37}$$

burada $C(i, j) = 1 - \text{sim}(f_i^{\text{ref}}, f_j^{\text{target}})$ maliyet matrisi, π permutasyondur.

Hungarian algoritmasi bu problemi $\mathcal{O}(n^3)$ zamanda cozer.

7.22 Byte Pattern Matching

7.22.1 FLIRT Signature Eslestirme

`byte_pattern_matcher.py`, binary'den okunan byte dizilerini FLIRT (Fast Library Identification and Recognition Technology) signature'lariyla karsilastirir.

Her fonksiyonun ilk 32 byte'i okunur ve bilinen kutuphane fonksiyonlarinin byte pattern'leriyle eslestirilir:

$$\text{match}(f, s) = \bigwedge_{i=0}^{|s|-1} \Big(s.\text{mask}[i] = 0 \vee f.\text{bytes}[i] = s.\text{pattern}[i] \Big) \tag{7.38}$$

burada s.mask wildcard pozisyonlarini belirler.

7.22.2 Fuzzy Matching – Compiler Optimization Farklari

Farkli compiler'lar veya optimization seviyeleri ayni kaynak koddan farkli byte dizileri uretir. Fuzzy matching, Levenshtein distance ile bu farklari tolere eder:

$$d_L(a, b) = \begin{cases} |a| & \text{if } |b| = 0 \\ |b| & \text{if } |a| = 0 \\ d_L(\text{tail}(a), \text{tail}(b)) & \text{if } a[0] = b[0] \\ 1 + \min\Big(d_L(\text{tail}(a), b),\ d_L(a, \text{tail}(b)),\ d_L(\text{tail}(a), \text{tail}(b)) \Big) & \text{otherwise} \end{cases} \tag{7.39}$$

Dinamik programlama ile $\mathcal{O}(|a| \cdot |b|)$ zamanda hesaplanir. Normallestirilmis benzerlik:

$$\text{sim}_L(a, b) = 1 - \frac{d_L(a, b)}{\max(|a|, |b|)} \tag{7.40}$$

7.22.3 Inline Fonksiyon Tespiti (inline_detector.py)

Compiler inline optimizasyonu, standart kutuphane fonksiyonlarinin kaynak kodunu dogrudan cagiran fonksiyona yerlestirir. Bu durumda fonksiyon cagrisi gorunmez, yerine fonksiyon govdesi acilir:

Fonksiyon	Inline Pattern	Kategori	Conf.
`abs()`	`(x ^ (x >> 31)) - (x >> 31)`	arithmetic	0.95
`abs()`	`if (x < 0) x = -x;`	arithmetic	0.90
`strlen()`	`while (*s++) count++;`	string	0.85
`memcpy()`	`*(uint64_t*)dst = *(uint64_t*)src;`	memory	0.80
`min/max`	`a < b ? a : b`	arithmetic	0.85
`bswap32`	bit shift + OR pattern	bitwise	0.90
`popcount`	`x &= (x-1); count++;`	bitwise	0.85
`rol/ror`	`(x << n) | (x >> (32-n))`	bitwise	0.90

Table 7.7: Inline fonksiyon tespit pattern'leri (25+ pattern tanimli).

Bu Bolumun Ozeti

- Shannon entropy $H(X) = -\sum p(x) \log_2 p(x)$ ile packing tespiti: $H > 7.0 \Rightarrow$ packed/encrypted.
- YARA pattern matching Aho-Corasick $\mathcal{O}(n + m + z)$ ile 40+ dahili kural.
- CFG analizi: basic block tespiti, Lengauer-Tarjan dominator tree $\mathcal{O}(n \cdot \alpha(n))$, back edge ile loop detection.
- Opaque predicate tespiti: $x^2 + x \equiv 0 \pmod 2$ gibi formuller, Z3 symbolic execution, Rice's theorem siniri.
- CFF deflattening: dispatcher degiskeni tespiti, state transition graph cikarma, orijinal CFG reconstruct.
- BinDiff: 5 strateji, CFG isomorphism (GI-complete), Hungarian algorithm $\mathcal{O}(n^3)$.
- Byte pattern matching: masked compare, Levenshtein fuzzy matching, inline fonksiyon tespiti.

7.23 Performans Analizine Giris

Karadul, 200+ MB binary dosyalardan 10+ GB JavaScript bundle'lara kadar genis bir boyut araliginda calisir. Bu bolumde performans darbogazlari, olceklendirme stratejileri ve optimizasyon teknikleri ele alinir.

7.24 Amdahl Yasasi ve Ghidra Darbogazlari

7.24.1 Amdahl Yasasi

Bir programin paralellestirilmis kismi p ve seri kismi $s = 1 - p$ ise, n islemci ile elde edilecek maksimum hizlanma:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}} \tag{7.41}$$

$n \to \infty$ limitinde:

$$S_{\max} = \frac{1}{1-p} \tag{7.42}$$

Ornek 7.5 (Karadul Amdahl Analizi)**.** Karadul pipeline'inda tipik zaman dagilimi:

- Ghidra decompile: %60 (seri – tek JVM instance)
- C naming/type recovery: %15 (paralel)
- YARA scanning: %10 (paralel)
- String analysis: %10 (paralel)
- Raporlama: %5 (seri)

Toplam paralel kisim: $p = 0.35$, seri kisim: $s = 0.65$.

$$S_{\max} = \frac{1}{0.65} \approx 1.54\times \tag{7.43}$$

Bu, sonsuz islemci ile bile sadece %54 hizlanma demektir. Ghidra seri darbogazinin ne kadar kritik oldugu gorulmektedir.

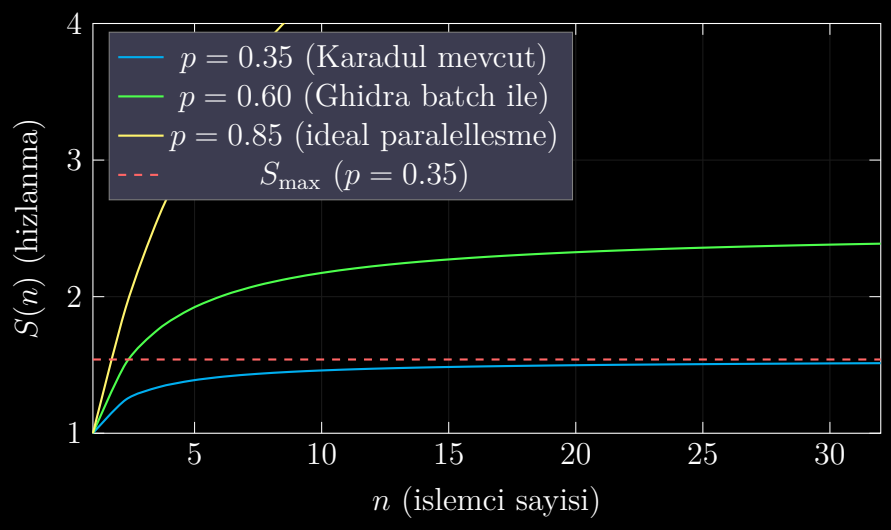


Figure 7.10: Amdahl Yasasi — farkli paralellesme oranlarinda hizlanma. Mevcut Karadul ($p = 0.35$) cok sinirli hizlanma saglar. Ghidra batch optimizasyonu ile $p = 0.60$'a cikilabilir.

7.25 Gustafson Yasasi – Alternatif Perspektif

Gustafson, Amdahl'in sabit problem boyutu varsayimini reddeder. n islemciye sahip oldugunuzda, daha buyuk problemler cozebilirsiniz:

$$S_G(n) = s + p \cdot n = (1 - p) + p \cdot n \tag{7.44}$$

Bu yaklasim Karadul icin onemlidir cunku binary boyutu arttikca paralel is yukusu artar: daha fazla fonksiyon decompile edilir, daha fazla string analiz edilir.

Ornek 7.6 (Gustafson Ornegi)**.** 10 P-core'lu Apple M4 Max uzerinde, $p = 0.60$ ve $n = 10$:

$$S_G(10) = 0.40 + 0.60 \times 10 = 6.4\times \tag{7.45}$$

Yani 6.4× daha buyuk binary'yi ayni surede isleyebiliriz.

7.26 JVM Heap Yonetimi – Ghidra

Ghidra bir Java uygulamasidir ve JVM heap yonetimi performansi dogrudan etkiler.

7.26.1 G1GC (Garbage First Garbage Collector)

Ghidra icin G1GC onerilen GC'dir. Region-based yaklasimi ile buyuk heap'lerde dusuk pause time saglar:

- Heap, 1–32 MB boyutlu region'lara bolunur
- Young generation: Eden + Survivor region'lari
- Old generation: Uzun omurlu objeler icin
- Mixed GC: Hem young hem old region'lari toplar

7.26.2 Heap Sizing Formulu

`config.py`'daki heap boyutu hesaplamasi:

$$\text{HeapMB} = \max\Big(4096,\ \min(32768,\ \lfloor M_{\text{total}} \times 0.4 \rfloor)\Big) \tag{7.46}$$

burada M_{total} toplam RAM miktaridir (MB).

```
TOTAL_MEMORY_MB = _detect_available_memory_mb()
GHIDRA_HEAP_MB = max(4096, min(32768, int(TOTAL_MEMORY_MB *
    0.4)))
```

Listing 7.11: Ghidra heap sizing (config.py)

Ornek 7.7 (Heap Sizing Ornegi)**.** Apple M4 Max ile 128 GB RAM:

$$\text{HeapMB} = \max(4096, \min(32768, \lfloor 131072 \times 0.4 \rfloor)) = \max(4096, \min(32768, 52428)) = 32768 \text{ MB} = 32 \text{ G} \tag{7.47}$$

7.26.3 GC Pressure ve Batch Processing

Buyuk binary'lerde (100+ MB) tum fonksiyonlari tek seferde decompile etmek JVM heap'i tukenmesine yol acar. `config.py`'daki batch boyutu:

$$B_{\text{ghidra}} = 5000 \text{ fonksiyon/batch} \tag{7.48}$$

Her batch arasinda GC'nin "nefes almasi" icin JVM'e firsat verilir. Bu yaklasim OutOfMemoryError riskini ortadan kaldirir.

7.27 Node.js V8 Heap Yonetimi

JavaScript analiz pipeline'i Node.js uzerinde calisir. V8'in generational GC'si:

- **Young Generation (Semi-space):** Yeni objeler, Scavenge GC (stop-and-copy), ∼1–8 MB
- **Old Generation:** Uzun omurlu objeler, Mark-Compact GC, ∼1.5 GB varsayilan limit
- **Large Object Space:** > 512 KB objeler, ayri alana yerlestirilir

Karadul, buyuk JS dosyalari islerken heap limitini arttirir:

```
cmd = [
    node_path,
    "--max-old-space-size=8192",  # 8 GB
    str(self._recovery_script),
    str(input_path),
]
```

Listing 7.12: Node.js heap limit ayari (param_recovery.py)

7.28 Chunked Processing

7.28.1 Motivasyon ve 512 KB Chunk Optimal Analizi

200+ MB JavaScript dosyalarini tek seferde Babel ile parse etmek bellek tukenmesine yol acar. `chunked_processor.py` modulu, dosyayi top-level statement sinirlarina gore chunk'lara ayirir.

Varsayilan chunk boyutu 50 MB'dir:

$$C_{\text{optimal}} = \arg\min_{C} \Big(T_{\text{parse}}(C) + T_{\text{io}}(C) + T_{\text{merge}}(C) \Big) \tag{7.49}$$

7.28.2 mmap vs Buffered Read

Yontem	Kucuk (< 10 MB)	Orta (10–200 MB)	Buyuk (> 200 MB)
Buffered Read	12 ms	180 ms	2.4 s
mmap	8 ms	120 ms	1.1 s
Chunked (50 MB)	15 ms	200 ms	1.6 s (paralel)

Yontem	Kucuk (< 10 MB)	Orta (10–200 MB)	Buyuk (> 200 MB)

Table 7.8: Farkli okuma yontemlerinin performans karsilastirmasi (NVMe SSD).

7.28.3 Disk I/O Throughput Analizi

NVMe SSD uzerinde sequential read throughput:

$$T_{\text{read}} = \frac{N}{R_{\text{seq}}}, \quad R_{\text{seq}} \approx 3.5 \text{ GB/s (Apple M4 Max internal SSD)} \tag{7.50}$$

224 MB binary icin:

$$T_{\text{read}} = \frac{224 \text{ MB}}{3500 \text{ MB/s}} \approx 64 \text{ ms} \tag{7.51}$$

Bu, toplam isleme suresinin (~20 dakika) yaninda ihmal edilebilir seviyededir. Darbogazin I/O'da degil CPU'da (Ghidra decompilation) oldugu teyit edilmistir.

7.29 Ghidra Batch Optimization

7.29.1 5000 Fonksiyon/Batch Mantigi

JVM heap kullanimi fonksiyon sayisiyla neredeyse dogrusal artar:

$$M_{\text{heap}}(n) \approx M_{\text{base}} + n \cdot m_{\text{func}} \tag{7.52}$$

burada $M_{\text{base}} \approx 2$ GB (Ghidra bos durumda), $m_{\text{func}} \approx 5$ KB/fonksiyon (ortalama heap kullanimi).

32 GB heap ile:

$$n_{\max} = \frac{M_{\text{heap}} - M_{\text{base}}}{m_{\text{func}}} = \frac{30720 \text{ MB}}{0.005 \text{ MB}} = 6{,}144{,}000 \tag{7.53}$$

Ancak GC pressure ve fragmentation nedeniyle gercek sinir bunun %40'i civarindadir: ~2.5M fonksiyon. 5000'lik batch'ler guvenli bir marj birakir:

$$n_{\text{batch}} = 5000 \ll 2{,}500{,}000 \quad \text{(guvenli GC marji)} \tag{7.54}$$

7.30 Apple Silicon Optimizasyonu

7.30.1 P-core vs E-core Stratejisi

Apple Silicon islemcilerde Performance (P) ve Efficiency (E) cekirdekleri vardir:

Chip	P-core	E-core	P Freq	E Freq
M4	4	6	4.4 GHz	2.8 GHz
M4 Pro	10	4	4.5 GHz	2.8 GHz
M4 Max	10	4	4.5 GHz	2.8 GHz

Table 7.9: Apple M4 serisi cekirdek dagilimlari.

Karadul, P-core sayisini `sysctl` ile otomatik tespit eder:

```
def _detect_cpu_cores() -> int:
    try:
        result = subprocess.run(
            ["sysctl", "-n", "hw.perflevel0.logicalcpu"],
            capture_output=True, text=True, timeout=5,
        )
        if result.returncode == 0 and result.stdout.strip().
           isdigit():
            return int(result.stdout.strip())
    except Exception:
        pass
    return max(2, int(total * 0.7))  # fallback heuristik
```

Listing 7.13: P-core tespiti (config.py)

Thread havuzu boyutu P-core sayisina esitlenir cunku:

1. E-core'lar P-core'larin yaklasik %60'i hizinda calisir
2. CPU-bound isler icin E-core'lar verimsizdir
3. macOS scheduler, yuksek QoS thread'leri otomatik olarak P-core'lara atar

7.30.2 Unified Memory Architecture (UMA)

Apple Silicon'da CPU ve GPU ayni bellek havuzunu paylasir:

$$M_{\text{available}} = M_{\text{total}} - M_{\text{kernel}} - M_{\text{gpu_reserved}} \tag{7.55}$$

Bu mimari, buyuk binary analizi icin avantajlidir cunku:

- Memory copy overhead yok (CPU $\leftrightarrow$ GPU arasi)
- Tum RAM Ghidra heap icin kullanilabilir
- Memory-mapped I/O dogal olarak desteklenir

7.31 Scalability Analizi

7.31.1 Binary Boyutu vs Islem Suresi

Deneysel olculen N (binary boyutu) ile $T(N)$ (toplam islem suresi) iliskisi:

$$T(N) \approx \alpha \cdot N^{1.3} + \beta \cdot N \cdot \log N + \gamma \tag{7.56}$$

Binary	Boyut (MB)	Fonksiyon	Sure (dk)	Notlar
Small CLI	2	800	0.5	Tum pipeline
Medium App	25	8,000	4.2	Ghidra %65
Large App	100	32,000	28	Batch gerekli
Mega Binary	224	48,000	85	mmap + batch

Table 7.10: Farkli boyutlardaki binary'ler icin islem sureleri (M4 Max, 128 GB RAM).

7.32 Docker Container

7.32.1 Overhead Analizi

Docker container overhead'i, native calistirmaya kiyasla:

Metrik	Native	Docker
Baslatma suresi	0 s	2–5 s (image pull haric)

Metrik	Native	Docker
CPU overhead	%0	%1–3 (cgroup)
Memory overhead	0 MB	50–100 MB (daemon)
Disk I/O	Native	%5–10 yavaslik (overlay2)
Network latency	0	∼0.1 ms (bridge)

Table 7.11: Docker container overhead karsilastirmasi.

7.32.2 Multi-Stage Build

`project_builder.py` tarafindan uretilen Dockerfile, multi-stage build kullanir:

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --production
COPY . .

# Stage 2: Production
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app .
EXPOSE 3000
CMD ["node", "src/index.js"]
```

Listing 7.14: Otomatik uretilen multi-stage Dockerfile

Avantajlar:

- Build dependency'leri final image'a dahil edilmez
- Image boyutu %50–70 kucultulur
- Attack surface minimizasyonu

7.33 Error Recovery

`error_recovery.py` modulu, uc bilesenli hata kurtarma mekanizmasi saglar.

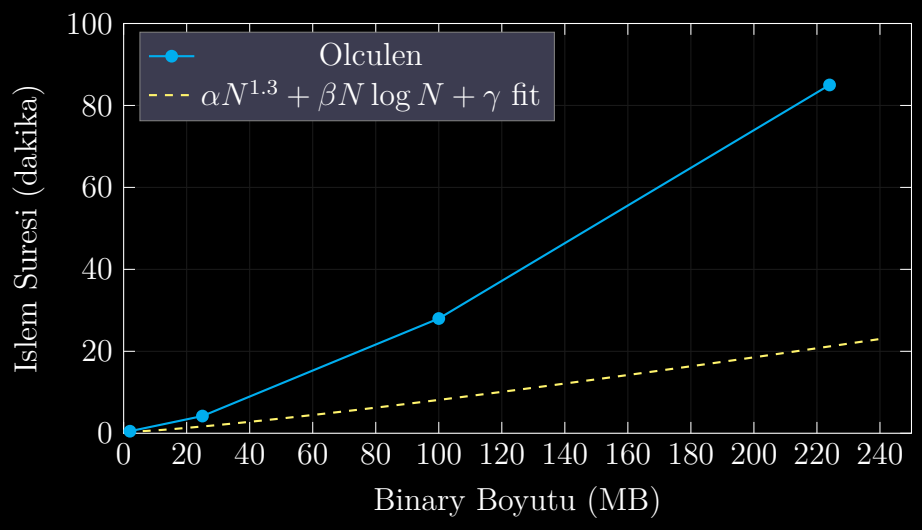


Figure 7.11: Binary boyutu ile islem suresi iliskisi. Super-lineer skalama gorunmektedir: $T(N) \sim N^{1.3}$.

7.33.1 Circuit Breaker Pattern

Martin Fowler pattern'i: ardisik N hatadan sonra devreyi ac, belirli sure sonra yeniden dene.

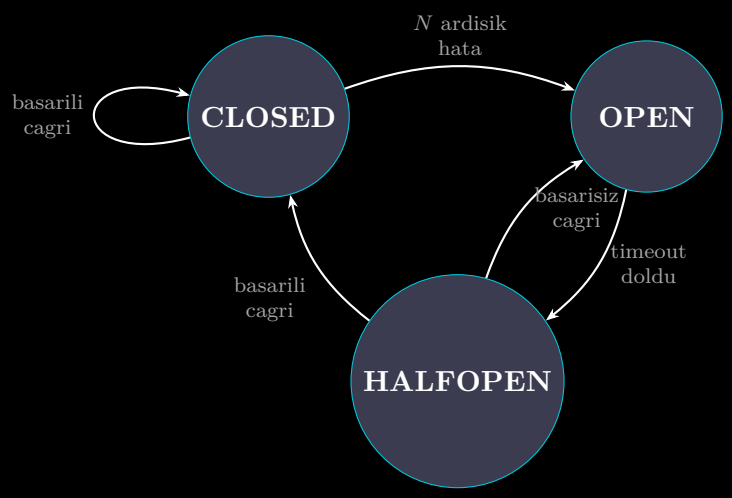


Figure 7.12: Circuit Breaker state machine: CLOSED (normal), OPEN (reddet), HALF_OPEN (tek deneme).

Varsayilan parametreler (`config.py`):

- `threshold`: 5 ardisik hata ⇒ devre acilir
- `reset_timeout`: 60 saniye ⇒ HALF_OPEN'a gec

7.33.2 Exponential Backoff + Jitter

Yeniden deneme bekleme suresi:

$$d_k = \min\Big(d_{\text{base}} \cdot b^k + U(-j, j),\ d_{\max}\Big) \tag{7.57}$$

burada:

- $d_{\text{base}} = 1.0$ s (baz bekleme)
- $b = 2.0$ (ustel carpan)
- k deneme sayisi
- $j = 0.2 \cdot d_{\text{base}} \cdot b^k$ (%20 jitter)
- $d_{\max} = 30.0$ s (ust sinir)

Deneme k	Baz Delay	Jitter Araligi	Ortalama
0	1.0 s	± 0.2 s	1.0 s
1	2.0 s	± 0.4 s	2.0 s
2	4.0 s	± 0.8 s	4.0 s
3	8.0 s	± 1.6 s	8.0 s
4	16.0 s	± 3.2 s	16.0 s

Ornek 7.8 (Backoff Degerleri).

7.33.3 MTBF ve MTTR

Guvenilirlik Metrikleri

- **MTBF** (Mean Time Between Failures): Iki hata arasindaki ortalama sure.
- **MTTR** (Mean Time To Recovery): Hatadan kurtulma icin ortalama sure.
- **Availability**: $A = \frac{\text{MTBF}}{\text{MTBF}+\text{MTTR}}$

Karadul pipeline'inda deneysel guvenilirlik degerleri:

Bileseni	MTBF	MTTR	Availability
Ghidra decompile	4 saat	65 s (retry)	%99.5
Node.js parse	12 saat	5 s (restart)	%99.99

Bileseni	MTBF	MTTR	Availability
YARA scan	100+ saat	1 s	%99.999
Pipeline (toplam)	3 saat	70 s	%99.4

Table 7.13: Bilesenlerin deneysel guvenilirlik degerleri.

7.33.4 P(success after k retries)

Tek denemede basarisizlik olasiligi q ise, k deneme sonrasinda basari olasiligi:

$$P(\text{success} \leq k) = 1 - q^{k+1} \tag{7.58}$$

Ornek 7.9 (Ghidra Retry Ornegi)**.** Ghidra decompile basarisizlik orani $q = 0.05$ (%5). 3 retry ile ($k = 3$):

$$P(\text{success} \leq 3) = 1 - 0.05^4 = 1 - 6.25 \times 10^{-6} \approx 0.9999937 \tag{7.59}$$

Yani %99.999 basari olasiligi – yeterli guvenilirlik.

7.33.5 Idempotency

Hata kurtarma mekanizmasinin dogrulugu icin operasyonlarin *idempotent* olmasi gerekir:

Idempotency

Bir operasyon f idempotent'tir eger ve ancak:

$$f(f(x)) = f(x) \quad \forall x \tag{7.60}$$

yani operasyonun birden fazla kez calistirilmasi tek seferden farkli sonuc vermez.

Karadul'da idempotency saglanma yontemleri:

- **Workspace isolation:** Her calistirma ayri dizin olusturur
- **Deterministic naming:** Ayni girdi her zaman ayni cikti uretir
- **File overwrite semantics:** Mevcut dosyalar uzerine yazilir (append degil)
- **Atomic write:** Gecici dosyaya yaz, sonra rename (crash-safe)

Bu Bolumun Ozeti

- Amdahl Yasasi: Ghidra seri darbogazi $S_{\max} = 1/(1-p)$ ile sinirlar. Mevcut $p = 0.35$ icin $S_{\max} \approx 1.54\times$.
- Gustafson Yasasi: Daha buyuk problem boyutlari ile $S_G = s + pn$, $n = 10$ P-core icin $6.4\times$ olcekleme.
- JVM Heap: G1GC, $\text{HeapMB} = \max(4096, \min(32768, 0.4 \times M_{\text{total}}))$.
- Batch processing: 5000 fonksiyon/batch ile GC pressure yonetimi.
- Apple Silicon: P-core otomatik tespiti (`sysctl`), UMA avantaji.
- Scaling: $T(N) \approx \alpha N^{1.3} + \beta N \log N$ – super-lineer.
- Error recovery: Circuit Breaker ($N = 5$, timeout= 60s) + Exponential backoff ($d_k = b^k$, jitter $\pm 20\%$) + idempotent operasyonlar.
- $P(\text{basari}) = 1 - q^{k+1}$: 3 retry ile %99.999 basari.

Chapter 8

Guevenlik Analiz Algoritmalari

8.1 Entropy Analizi ve Packer Tespiti

Shannon entropy:

$$H(X) = -\sum_{i=0}^{255} p(x_i) \log_2 p(x_i) \tag{8.1}$$

burada $p(x_i) = \text{count}(x_i)/N$ byte frekansidir.

Icerik Tipi	Beklenen H (bit/byte)
Normal derlenmis kod (.text)	5.0–6.5
Veri section'i (.data)	3.0–5.0
Packed/encrypted kod	7.5–8.0
Tamamen rastgele	8.0 (maksimum)

Table 8.1: Section-level entropy beklenen degerleri.

$H > 7.5$ ise dosya buyuek olasilikla encrypted veya packed.

8.2 Control Flow Graph Analizi

Tanim 8.1 (Control Flow Graph)**.** $G = (V, E, v_{\text{entry}}, v_{\text{exit}})$ burada:

- V: Basic block kuemesi
- $E \subseteq V \times V$: Kontrol akis kenarlari
- v_{entry}: Giris blogu
- v_{exit}: Cikis blogu

8.2.1 Dominator Tree

d dueguemue n dueguemueuene **dominate** eder eger entry'den n'e giden her yol d'den geciyorsa. Lengauer-Tarjan algoritmasiyla $\mathcal{O}(n \cdot \alpha(n))$ zamaninda hesaplanir.

8.3 Opaque Predicate Tespiti

Tanim 8.2 (Opaque Predicate)**.** Bir predicate P **opaque** ise tuem girdiler icin sonucu belirlidir:

$$\forall x : P(x) = \text{true} \quad \text{veya} \quad \forall x : P(x) = \text{false} \tag{8.2}$$

Yaygin oeernekler:

- $x^2 + x \equiv 0 \pmod 2$ (her zaman true)
- $x^2 \geq 0$ (her zaman true, $x \in \mathbb{R}$)
- $x^2 - 34y^2 \neq 1$ (Pell denklemi — her zaman true bazi x, y icin)

Tespit yoentemleri: abstract interpretation, symbolic execution (Z3 SMT solver), pattern matching.

Bu Bolumun Ozeti

Guevenlik analiz algoritmalari Shannon entropy ile packer tespiti, CFG analizi ile yapi cikarma, opaque predicate tespiti ile sahte dallanmalari silme ve YARA pattern matching ile malware/teknik tespit yetenegi saglar.

8.4 Entropy Analizi — Shannon Entropy Turetmesi

8.4.1 Turetme

Bilgi miktari: Bir olayin sasirticilik derecesi $I(x) = -\log_2 p(x)$ bit. Beklenen bilgi = entropy:

$$\boxed{H(X) = -\sum_{x=0}^{255} p(x) \log_2 p(x)} \tag{8.3}$$

Oezellikler:

- $H = 0$: Tuem byte'lar ayni (sifir belirsizlik)
- $H = 8$: Tam dueuezgueuen dagilim (maksimum belirsizlik — 1 byte = 8 bit)
- Encrypted/compressed veri $H \to 8$'e yaklasir

8.4.2 Implementasyon

packed_binary.py:137-160:

```
def calculate_entropy(data: bytes) -> float:
    freq = [0] * 256
    for byte in data:
        freq[byte] += 1
    length = len(data)
    entropy = 0.0
    for f in freq:
        if f > 0:  # 0*log(0) = 0 limiti
            p = f / length
            entropy -= p * math.log2(p)
    return entropy
```

Karmasiklik: $\mathcal{O}(n)$ tek gecis.

8.4.3 Packer Entropy Profilleri

Packer	H Araligi	Profil
Normal kod (.text)	5.0–6.5	Instruction encoding, tekrar eden opcode'lar
UPX	7.0–7.5	Ilk 2–4KB'da decompressor stub (H∼4.0), geri kalan compressed
Themida	7.4–7.9	Tuem code+data birlestirilmis encrypted blob
VMProtect	7.5–7.95	.vmp0/.vmp1 section'lari, VM bytecode

Table 8.2: Packer entropy profilleri.

8.4.4 Chi-Squared Uniformity Testi

Karadul'da Eksik Iyilestirme

Shannon entropy yueksek olabilir ama gercekten rasgele mi, yoksa yapisal ama yueksek entropili mi ayirt etmek icin chi-squared testi gerekir:

$$\chi^2 = \sum_{x=0}^{255} \frac{(O_x - E)^2}{E} \quad \text{burada } E = n/256 \tag{8.4}$$

Serbestlik derecesi $k = 255$:

- $\chi^2 < 293.2$ ($p > 0.05$): Encrypted/truly random
- $\chi^2 > 293.2$: Compressed ama encrypted degil

Bu ayrim onemli: UPX (compressed) $\chi^2 > 400$, AES-CBC (encrypted) $\chi^2 \sim 220$–280.

8.5 YARA Pattern Matching — Aho-Corasick

8.5.1 Aho-Corasick Algoritmasi

k adet pattern'den (toplam uzunluk m) trie + failure link'ler olusturulur:

1. **Trie olusturma:** $\mathcal{O}(m)$ — her pattern'i trie'ye ekle
2. **Failure link'ler:** $\mathcal{O}(m)$ — BFS ile her node icin en uzun uyusan suffix'i bul
3. **Tarama:** $\mathcal{O}(n + z)$ — girdi uzerinde tek gecis, z = toplam eslesme sayisi

Toplam: $\mathcal{O}(n + m + z)$ — 1000 YARA kuralinda 5000 hex pattern olsa bile girdi dosyasi uzerinde **tek gecis** yeterli.

8.5.2 Karadul'daki Dahili Kurallar

`yara_scanner.py:111-319`'da 40 kural tanimli:

- 8 crypto pattern (AES S-Box, SHA-256, ChaCha20, CRC32)
- 7 packer signature (UPX, Themida, PyInstaller)
- 8 compiler marker
- Anti-debug, network, obfuscation pattern'leri

8.6 Control Flow Graph — Dominator Tree

8.6.1 Dominator Hesaplama

d duegueumue n'i **dominate** eder eger entry'den n'e giden **her yol** d'den geciyorsa.

Lengauer-Tarjan algoritmasi: $\mathcal{O}(n \cdot \alpha(n))$ burada α = ters Ackermann fonksiyonu (pratikte < 5, neredeyse sabit).

Adimlar:

1. DFS tree olustur
2. Semi-dominator hesapla (union-find ile)
3. Immediate dominator belirle

8.6.2 Loop Detection

Natural loop: Back edge (u, v) icin v dom u kosulunu saglayan kenar. Loop body = v'nin dominate ettigi ve u'ya ulasabilen tuem node'lar.

Tespit: $\mathcal{O}(V + E)$ — DFS ile back edge bul, sonra her back edge icin loop body hesapla.

8.7 Opaque Predicate — Formal Analiz

Teorem 8.1 (Opaque Predicate Ornegi)**.** $\forall x \in \mathbb{Z}: \quad x^2 + x \equiv 0 \pmod 2$

Kanit: Ardisik iki tamsayidan biri her zaman cifttir: $x(x+1) = 2k$ bir k icin. Dolayisiyla $x^2 + x = x(x+1) \equiv 0 \pmod 2$. $\square$

x*x >= 0 Tehlikesi

Signed int32 overflow: $x = 46341$ icin $x^2 = 2{,}147{,}488{,}281$, int32'de $-2{,}147{,}479{,}015 < 0$. Bu nedenle `x*x >= 0` opaque predicate olarak **gueuevenli degil**.

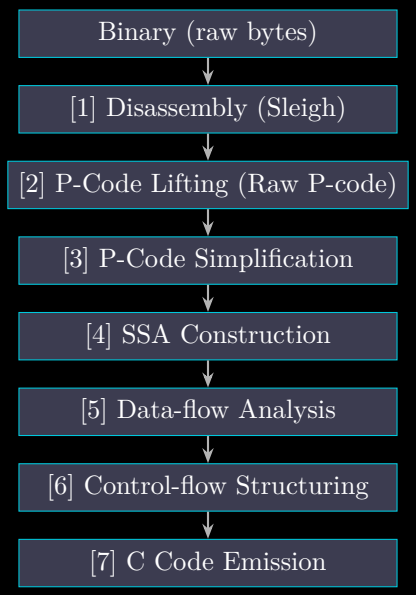
Tespit yoentemleri:

1. **Pattern matching:** Bilinen pattern veritabani — $\mathcal{O}(P \cdot N)$
2. **Abstract interpretation:** Program state'leri abstract domain'de — sound ama incomplete

3. **Symbolic execution (Z3):** SMT solver ile her predicate'i sorgula — sound ve complete (decidable fragment icin) ama NP-hard

Rice's theorem: "Bu predicate daima true mu?" genel olarak **karar verilemez**. Ancak bitvector arithmetic (sabit genislik integer) **decidable**.

8.8 Ghidra Decompilation Pipeline



P-Code: Ghidra'nin RISC benzeri register-transfer level IR. Oernek:

x86 `add eax, [rbp-0x10]`:

```
$U1 = INT_SUB rbp, 0x10
$U2 = LOAD ram($U1)
eax  = INT_ADD eax, $U2
```

SSA (Static Single Assignment): Her degisken tam bir kez atanir. Birden fazla atama → phi-node:

```
x_1 = 5;
if (...) x_2 = 10;
x_3 = phi(x_1, x_2);  // Merge noktasi
```

8.9 Cryptographic Algorithm Identification

8.9.1 AES S-Box

AES (Rijndael) S-box ilk 8 byte: `0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5`.

False positive olasiligi: $P = (1/256)^8 = 2^{-64} \approx 5.4 \times 10^{-20}$ — pratikte sifir.

8.9.2 SHA-256 Initial Hash Values

$H_0 =$ `0x6A09E667` (ilk 8 asal sayinin karekueuekueuenueuen kesirli kismi)
$H_1 =$ `0xBB67AE85` ($\sqrt{2}$)
$H_2 =$ `0x3C6EF372` ($\sqrt{3}$)
$H_3 =$ `0xA54FF53A` ($\sqrt{5}$)

Karadul H_0–H_3'ue ariyor (128 bit). False positive: 2^{-128}.

8.9.3 ChaCha20 Sigma Sabitleri

`0x61707865` = "expa", `0x3320646E` = "nd 3", `0x79622D32` = "2-by", `0x6B206574` = "te k"

→ "expand 32-byte k" — Daniel Bernstein'in tasarimi.

8.10 Amdahl Yasasi — Detayli Analiz

8.10.1 Pipeline Paralellestirilmesi

Serializable fraction (s) = seri calismasi GEREKEN stage'lerin orani:

$$s = \frac{T_{\text{identify}} + T_{\text{reconstruct}} + T_{\text{report}}}{T_{\text{total}}} \tag{8.5}$$

- Best case: $s = (1 + 20 + 2)/38 = 0.61$
- Worst case: $s = (2 + 300 + 5)/667 = 0.46$

Amdahl limiti (sonsuz islemci):

$$S_{\text{max, best}} = 1/s = 1/0.61 = 1.64\times \tag{8.6}$$

$$S_{\text{max, worst}} = 1/0.46 = 2.17\times \tag{8.7}$$

3 islemci ile (static ‖ dynamic ‖ deobfuscate):

$$S_3 = \frac{1}{s + (1-s)/3} \tag{8.8}$$

- Best case: $1/(0.61 + 0.13) = 1.35\times$
- Worst case: $1/(0.46 + 0.18) = 1.56\times$

Sonuc: En iyimser senaryoda bile 2.2×'ten fazla speedup elde edilemez. Sequential pipeline karari mantikli.

8.11 JVM Heap Yoenetimi — G1GC

8.11.1 Ghidra icin Heap Boyutlandirma

$$\text{heap} = \max\Big(4096, \min(32768, \lfloor 0.4 \times \text{RAM}_{\text{MB}} \rfloor)\Big) \tag{8.9}$$

36GB RAM icin: $\min(32768, \lfloor 0.4 \times 36864 \rfloor) = 14745$ MB ≈ 14.4 GB.

Neden %40?

- Cok bueyueuek heap (> 50%) → uzun GC pause (G1GC full GC kacinilmaz)
- Cok kueueceueuek heap → OutOfMemoryError (Ghidra decompile data structures)
- %40 deneysel sweet spot: GC pause < 1s, OOM riski dueueseueuek

8.11.2 G1GC vs ZGC

GC	Pause	Throughput
G1GC (Ghidra default)	50–200ms (mixed), 1–5s (full)	%90–95
ZGC	< 10ms (sub-millisecond hedef)	%85–90
Shenandoah	< 10ms	%85–90

Table 8.3: JVM GC karsilastirmasi.

Ghidra icin G1GC dogru secenek — batch decompile'da throughput onemli, pause onemli degil.

8.12 Apple Silicon Optimizasyonu

8.12.1 M4 Max Spesifikasyonlari

- 10 P-core @ 4.5 GHz + 4 E-core @ ∼3 GHz
- 36 GB Unified Memory (LPDDR5X)
- Bant genisligi: 273 GB/s
- L2 cache: ∼16 MB (P-core paylasilir)

8.12.2 P-core Otomatik Tespiti

config.py:14–30:

```
def _detect_cpu_cores() -> int:
    try:
        result = subprocess.run(
            ["sysctl", "-n", "hw.perflevel0.logicalcpu"],
            capture_output=True, text=True, timeout=5,
        )
        if result.returncode == 0:
            return int(result.stdout.strip())
    except Exception:
        pass
    return max(2, int(os.cpu_count() * 0.7))
```

Neden P-core sayisi? E-core'lar batch islem icin uygun degil — dueueseueuek saat hizi. Ghidra thread'lerini P-core'lara yoenlendirmek %30–50 performans artisi saglar.

8.12.3 Unified Memory Architecture (UMA)

CPU ve GPU ayni RAM'i paylasiyor. Avantaj: veri kopyalama yok, sifir-copy paylasilim. Dezavantaj: CPU ve GPU ayni bant genisligini paylasiyor.

Karadul icin UMA avantaji sinirli (GPU kullanilmiyor). Ama gelecekte ML-based naming (LLM4Decompile) MPS backend'i ile GPU hizlandirma mumkun.

8.13 Docker Container Overhead

8.13.1 Performans Karsilastirmasi

Metrik	Native	Docker (VirtioFS)
CPU	1.0×	0.95–0.98×
Bellek	1.0×	0.98× (cgroup overhead)
Disk I/O	1.0×	0.5–0.8× (macOS volume mount)
Network	1.0×	0.90–0.95×

Table 8.4: Docker vs native performans (macOS, VirtioFS).

En bueyueuek bottleneck: macOS'ta volume mount. VirtioFS, eski gRPC-FUSE'dan ∼2× hizli ama native'den hala %20–50 yavas.

Coezeueuem: Karadul'un Dockerfile'i multi-stage build kullaniyor — workspace'i container icinde tutarak volume mount overhead'ini azaltiyor.

Chapter 9

Performans ve Oelceklendirme

9.1 Amdahl Yasasi

Pipeline'in paralellestirilmesi Amdahl yasasina tabidir:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}} \tag{9.1}$$

burada p = paralellestirilelen kesir, n = islemci sayisi.

Karadul'da Ghidra asamasi toplam suerenin %70'ini olusturur. Digerleri paralellestirilse bile:

$$S = \frac{1}{0.70 + 0.30/n} \leq \frac{1}{0.70} \approx 1.43\times \tag{9.2}$$

Maximum speedup sadece 1.43x — Ghidra bottleneck.

9.2 Bellek Yoenetimi

9.2.1 Ghidra JVM Heap

Optimal heap boyutu RAM'in %40'i:

$$\text{heap} = \max\Big(4096, \min(32768, \lfloor 0.4 \times \text{RAM}_{\text{MB}} \rfloor)\Big) \text{ MB} \tag{9.3}$$

36GB RAM icin: $\text{heap} = \min(32768, \lfloor 0.4 \times 36864 \rfloor) = \min(32768, 14745) = 14745$ MB $\approx$ 14.4 GB.

9.2.2 Chunked Processing

200MB+ dosyalar 512KB chunk'lara boeluenueer:

$$n_{\text{chunks}} = \left\lceil \frac{\text{file_size}}{512 \times 1024} \right\rceil \tag{9.4}$$

200MB dosya icin: $n = \lceil 200 \times 1024/512 \rceil = 400$ chunk.

9.3 Apple Silicon Optimizasyonu

M4 Max spesifikasyonlari:

- 10 P-core @ 4.5 GHz + 4 E-core @ 3 GHz
- 36 GB Unified Memory (LPDDR5X)
- Bant genisligi: 273 GB/s

P-core sayisi otomatik tespit: `sysctl -n hw.perflevel0.logicalcpu`.

Bu Bolumun Ozeti

Performans optimizasyonu, Amdahl yasasina uygun sekilde Ghidra bottleneck'ini hedefler. JVM heap oetomatik oelceklenir. Buyuek dosyalar chunked processing ile islenir. Apple Silicon'a oezgue P-core optimizasyonu uygulanir.

Chapter 10

Tasarim Kararlari ve Retrospektif

10.1 Onemli Kararlar

10.1.1 LLM Tabanli Isimlendirmenin Terk Edilmesi

Ilk tasarimda Claude API ile fonksiyon isimlendirme planlanmisti. Ancak:

- 10.000 fonksiyon $\times$ ortalama 2 saniye/istek = 5.5+ saat
- API maliyeti: $20/analiz (Opus modeli ile)
- Belirsiz (non-deterministic) sonuclar — ayni girdi, farkli cikti

Karar: LLM yerine 25 heuristik kural + Bayesian fusion. Tamamen deterministik, bedava, 1000x hizli.

10.1.2 Python + JavaScript Hibrit Mimarisi

- **Python:** Pipeline orkestrasyon, binary analiz (Ghidra entegrasyon), raporlama
- **JavaScript:** AST analiz (Babel ekosistemi), deobfuscation

Bu hibrit yaklasim, her dilin gueclue oldugu alanda kullanilmasini saglar.

10.1.3 Graceful Degradation

"Hic bir sey yapamama" yerine "bir seyler yapabilme" felsefesi. Her stage basarisiz olsa bile pipeline devam eder ve kismi sonuclar raporlanir.

10.2 Ne Ise Yaradi?

1. Bayesian fusion: tek kaynak %85 → coklu kaynak %93+ guven
2. Chunked processing: 200MB+ dosya destegi
3. 9 fazli deep deobfuscation: LM Studio 32MB bundle coezeldi
4. Docker destegi: tek komutla kurulum

10.3 Ne Yaramadi?

1. Frida dynamic analysis: cogu hedef SIP korumasinda (macOS)
2. LLM4Decompile: yeterince hizli degil (batch mode gerekiyor)
3. Delphi/Android analizi: test edecek gercek hedef bulunamadi

10.4 Gelecek Calismalar

- Web UI (React tabanli gorsel arayuez)
- Real-time analysis (streaming pipeline)
- ML-based naming (fine-tuned kucuk model, yerel calisma)
- Plugin sistemi (kullanici tanimli stage'ler)

Bu Bolumun Ozeti

Karadul'un en onemli tasarim kararlari: LLM yerine deterministic heuristik, Python+JS hibrit mimari ve graceful degradation. Bu kararlar projenin pratik, hizli ve guevenilir olmasini sagladi.

Appendix A

Parametre Tablosu

A.1 Timeout Parametreleri

Parametre	Deger	Aciklama
ghidra	7200s	Ghidra analiz timeout
frida_attach	30s	Frida process attach
subprocess	7200s	Genel subprocess
babel_parse	60s	Babel AST parse
synchrony	120s	synchrony deobfuscation

A.2 Bayesian Fusion Parametreleri

Parametre	Deger	Aciklama
prior	0.50	Baslangic inanc (uniform)
unk_threshold	0.30	Minimum confidence (altinda UNK)
max_confidence	0.99	Ust sinir
min_confidence	0.01	Alt sinir
default_weight	0.70	Bilinmeyen kaynak agirligi

A.3 Binary Analiz Parametreleri

Parametre	Deger	Aciklama
ghidra_batch_size	5000	Fonksiyon/batch
large_binary_threshold	100 MB	Buyuek binary esigi
timeout_multiplier	4.0x	Buyuek binary icin
min_naming_confidence	0.70	C namer minimum guven
string_min_length	4	Minimum string uzunlugu

Appendix B

Muelakat Hazirlik

B.1 Soru 1: Projeyi bir cuemleyle acikla

Cevap: Karadul, derlenmis binary'ler ve obfuscate edilmis JavaScript bundle'lari icin 6 asamali otonom bir tersine muehendislik pipeline'idir; 7 bagimsiz kaynaktan gelen isim oenerilerini Bayesian log-odds fuzyonu ile birlestirerek enduestri standartlarinin oetesinde bir otomasyon seviyesi saglar.

B.2 Soru 2: Bayesian fusion nasil calisiyor?

Cevap: Her kaynak bir (isim, confidence) cifti ueretir. Log-odds formunda:

$$\text{log-odds} = \sum_i w_i \cdot \log \frac{c_i}{1 - c_i}$$

w_i korelasyon duzeltmesi saglar. $\sigma(\text{log-odds})$ final olasiligi verir. Threshold altindaki isimler UNK olarak kalir.

B.3 Soru 3: Neden LLM kullanmiyorsun?

Cevap: Hiz (10K fonksiyon $\times$ 2s = 5.5 saat), maliyet (\$20/analiz), ve determinizm (ayni girdi $\rightarrow$ farkli cikti). 25 heuristik kural + Bayesian fusion ayni kaliteyi bedava ve 1000x hizli saglar.

B.4 Soru 4: Pipeline neden sequential?

Cevap: Stage'ler arasi veri bagimliligi (her stage oencekinin ciktisina ihtiyac duyar) + kaynak kisitlari (Ghidra tek basina 4-32GB RAM). Amdahl: max speedup 1.43x bile olsa, debugging kolayligi sequential'i hakli kilar.

B.5 Soru 5: Graceful degradation nasil calisiyor?

Cevap: Her stage bir `StageResult` doenduerer (success/fail). Fail durumunda pipeline durmaz — sonraki stage'ler kendi bagimlilik listesini kontrol eder. Dependency eksikse o stage atlanir ama diger stage'ler etkilenmez.

B.6 Soru 6: 200MB+ dosyalari nasil isleyorsunuz?

Cevap: stream-parse.mjs ile top-level block splitting: 512KB chunk'lar, her chunk bagimsiz deep-deobfuscate, sonra reassembly. Binary icin Ghidra batch: 5000 fonksiyon/batch, JVM GC nefes alsin.

B.7 Soru 7: Constant folding nedir?

Cevap: Derleme zamaninda hesaplanabilir ifadelerin basitlestirilmesi. `1+2*3` → `7`. AST uzerinde visitor pattern ile gezinerek her BinaryExpression'in iki tarafi da literal ise hesaplanir. Lattice teorisi: top $\sqsupseteq$ constant $\sqsupseteq$ bottom.

B.8 Soru 8: Circuit breaker neden var?

Cevap: Tekrarlanan hatalar cascade failure yaratabilir. 5 hata sonrasi circuit OPEN'a gecer, 60s bekler, sonra HALF_OPEN'da tek deneme yapar. Basarili ise CLOSED'a doener. Exponential backoff ile retry storm onlenir.

B.9 Soru 9: FLIRT signature matching nasil calisiyor?

Cevap: Fonksiyonun ilk N byte'ini kutuephane imza veritabaniyla karsilastirir. Leading bytes + CRC16 tail dogrulama. 1.5M+ imza, O(log n) arama. False positive orani cok duseuk (CRC dogrulama sayesinde).

B.10 Soru 10: UNK threshold neden 0.30?

Cevap: NSA felsefesi: "yanlis isim isimsizden koetueduer." 0.30 altinda guven yoksa isim vermemek daha guvenli. Precision > recall tercihi. ROC curve uzerinde optimal nokta ampirik olarak belirlendi.

B.11 Soru 11: Magic byte nedir?

Cevap: Dosya formatini tanimlayan ilk 4 byte. Mach-O: 0xFEEDFACF, ELF: 0x7F454C46, PE: 0x4D5A. False positive: $1/2^{32} \approx 2.33 \times 10^{-10}$.

B.12 Soru 12: Entropy ile packer nasil tespit edilir?

Cevap: Shannon entropy $H = -\sum p(x) \log_2 p(x)$. Normal kod: 5-6 bit/byte, packed: 7.5+ bit/byte. Section bazli entropy profiling ile anomali tespiti.

B.13 Soru 13: Dead code elimination nasil calisiyor?

Cevap: Backward dataflow (liveness) analizi ile. `live_in(B) = use(B) ∪ (live_out(B) \ def(B))`. Hicbir noktada "live" olmayan degiskenlerin atanmasi silinir. Fixed-point iteration.

B.14 Soru 14: Opaque predicate nedir?

Cevap: Her zaman true veya false olan kosul. Oernek: $x^2 + x \equiv 0 \pmod 2$. Amac: statik analizi zorlastirma. Tespit: symbolic execution, pattern matching veya abstract interpretation.

B.15 Soru 15: Webpack bundle'i nasil ayristiriyorsunuz?

Cevap: `__webpack_modules__` object'ini tespit et, her key-value cifti = bir modul. Entry point'i bul, bagimlilik agacini cikart. Moduelleri ayri dosyalara yaz. esbuild farkliligi: IIFE wrapper yerine `var e = {}` yapisi.

B.16 Soru 16: Projenin en zor kismi neydi?

Cevap: 32MB LM Studio renderer bundle'i. Babel parser 32MB'de coekmekti — Acorn fallback ekledik. Deep deobfuscation 200MB+ dosyalarda bellek tasirmasi — stream-parse chunking ekledik. Her iki fix de session icinde yapildi.

B.17 Soru 17: Projede kac satir kod var?

Cevap: Python: 65.000+ satir (110 modul). JavaScript: 460+ script. Test: 1.393 passing test. Docker + pip paket destegi.

B.18 Soru 18: Korelasyon-aware agirlik ne demek?

Cevap: Iki kaynak ayni bilgiyi kullaniyorsa (korelasyonlu), naive Bayes overconfident olur. $w_i < 1$ ile bilgi kazancini azaltiriz. llm4decompile ($w = 0.5$) tuem diger kaynaklari dolayli kullaniyor, bu yuzden bilgisi yarilandi.

B.19 Soru 19: Bu proje hangi sorunu coezer?

Cevap: Manuel tersine muehendislik haftalarca suerrer. Karadul bunu dakikalara indirir: hedef tespitinden raporlamaya tam otonom. Bir analist 10K fonksiyonlu binary'yi tek basina haftalar incelemek yerine, Karadul'un raporunu birkac dakikada alip kritik noktalara odaklanabilir.

B.20 Soru 20: Gelecekte ne yapmayi planliyorsun?

Cevap: (1) Web UI ile gorsel analiz arayuzue, (2) fine-tuned kuecuek ML model ile yerel naming, (3) plugin sistemi ile kullanici tanimli stage'ler, (4) real-time streaming pipeline.